Started on	Thursday, 28 January 2021, 8:13 AM
State	Finished
Completed on	Thursday, 28 January 2021, 8:52 AM
Time taken	39 mins 38 secs
Grade	**6.05** out of 10.00 (**61**%)

Question **1**

Complete

Mark 1.60 out of 2.50

Ce este Data Space? Ce se regăsește în interiorul său? Dați exemplu de două instrucțiuni care îl pot accesa.

Data Space reprezinta spatiul de date (memoria de date) si este un SRAM de tip pipeline (sincron, 128 bytes). Memoria de date este mapata inttre 0x0040 si 0x00BF. Adresele sunt pe 7 biti, iar datele au 8 biti (aceasta fiind dimensiunea unui registru).
Procesorul lucreaza cu 16 registre de 8 biti (exista totusi 32 registre), folosindu-le doar pe cele din R16-R31.

Exemple de instructiuni: ADD, MOV

Comment:
Instructiunile sunt de load sau de store, add si mov nu au treaba. De ce ai mentionat si registrele?

Question **2**

Complete

Mark 1.50 out of 2.50

Să presupunem că pe un microcontroller ATtiny20 a fost activată întreruperea de output compare match B a Timer/Counter0 și întreruperile globale sunt active. Cum putem face astfel încât la întâlnirea unei astfel de întreruperi să fie executată o funcție numită “handle_irq”?

Dupa executia unei instructiuni, procesorul va verifica IRQ, adica linia de cereri de intreruperi, vazand daca este activa. Daca da, inseamna ca exista o intrerupere care trebuie tratata. Apoi, verifica numarul intreruperii, adica IRQNO, ca sa determine modul de tratare a intreruperii. Dupa ce afla numarul, in tabela vectorilor de intrerupere, la indexul respectiv, va gasi rutina de tratatre a intreruperii respective, apoi o executa (executa astfel si functia "handle_irq").

Comment:
Nu ai zis concret cum se executa functia (rjmp handle_irq) si nici nu ai zis care e indexul vectorului de intrerupere.

Question **3**

Complete

Mark 0.50 out of 2.50

Folosind setul de instrucțiuni AVR, scrieți o funcție care salvează în R31 cea mai mică dintre valorile din registrele R27, R8, R29 și R30. Scrieți un program care apelează funcția cu R27 = 12, R8 = 13, R29 = 24 și R30 = 99.

```
funct_smth:
  ldi R27, 12
  ldi R8, 13
  ldi R29, 24
  ldi R30, 99

  mov R31, R27
  breq R31
```

Comment:
Nu ai implementat functia, nu ai terminat de apelat functia. In R8 nu poti incarca un imediat cu ldi, trebuie sa il incarci mai intai intr-un registru R16-R31, apoi sa il muti din el in R8.

Question **4**

Complete

Mark 2.45 out of 2.50

Folosind setul de instrucțiuni AVR, scrieți o secvență de cod care:
- Configurează portul A al unui microcontroller ATtiny20 astfel: PA0, PA1, PA2, PA3, PA4 și PA5 ca pini de ieșire, PA6 și PA7 ca pini de intrare
- Setează valoarea pinilor PA0 și PA4 să fie 1 (High)
- Setează valoarea celorlalți pini să fie 0 (Low)

Ciorna:

// ddra -> 0b00111111

// porta -> 0b00010001

Secventa cod:

```
DDRA equ 0x01

PORTA equ 0x02


ldi R16, 0b00111111

out DDRA, R16


ldi R16, 0b00010001

out PORTA, R16

breq some_tag


sbi PORTA, 0

some_tag:
```

Comment:
Ce ai vrut sa faci incepand cu breq? Subiectul era rezolvat deja deasupra lui.

◄ Cursul 2

Jump to...

Examen - Partea I ►

Started on	Friday, 19 February 2021, 8:19 AM
State	Finished
Completed on	Friday, 19 February 2021, 8:50 AM
Time taken	30 mins 35 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Bill dorește să construiască un cip de memorie de 512K x 64, dar are doar cipuri de 16K x 8. De câte cipuri are nevoie Bill?

- a. 16
- b. 256 ✔
- c. 128
- d. 64
- e. 8

(512K / 16K) * (64 / 8) = 256

The correct answer is: 256

Question **2**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 5ns iar rata de hit locală este 80%
- Latența de acces din L1 în L2 este de 12ns iar rata de hit locală este de 90%
- Latența de acces din L2 în memoria RAM este de 400ns

Care este timpul mediu de acces la memorie?

- a. 1540ns
- b. 15.4ns ✔
- c. 0.154ns
- d. 1.54ns
- e. 154ns

0.8 * 5ns + 0.2 * (0.9 * 17ns + 0.1 * 417ns) = 15.4ns

The correct answer is: 15.4ns

Question **3**

Correct

Mark 1.00 out of 1.00

Un sistem cu cache L1 si L2 are un hit rate de 98% din L1 și de 99% din L2. Care este miss rate-ul global?

- a. 0.02% ✔
- b. 9.8%
- c. 98%
- d. 2%
- e. 20%

1 - 0.98 - 0.02 * 0.99 = 0.0002

The correct answer is: 0.02%

Question **4**

Correct

Mark 1.00 out of 1.00

Parcurgerea unui vector exploatează:

- a. Nu se poate determina
- b. Localitatea temporală
- c. Localitatea spațială ✔
- d. Ambele localități
- e. Niciuna

Vectorii sunt ținuți contiguu în memorie iar parcugerea unui vector rezultă în acces secvențial. Asta înseamnă că, dacă am accesat o adresă din vector, este probabil să vrem să accesăm și adresele următoare sau precedente.

The correct answer is: Localitatea spațială

Question **5**

Correct

Mark 1.00 out of 1.00

O memorie cache cu linii de 32 octeți lungime este organizată în 32 de seturi a câte 32 linii. Dacă adresarea se face la nivel de octet, cu adrese pe 32 de biți, care este dimensiunea în biți a tag-ului pentru fiecare linie de cache?

- a. 19
- b. 17
- c. 18
- d. 21
- e. 20 ✔

~~32 - log2(32 * 32 * 32) = 17~~

32 - log2(32 * 32) = 22. Cum nu ati primit aceasta varianta, orice raspuns e corect!

The correct answers are: 20, 21, 19, 18, 17

Question **6**

Correct

Mark 1.00 out of 1.00

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before:

```
for (i = 0; i < N; i++)
 c[i] = a[i] + b[i];
```

After:

```
int P = 1;
for (i = 0; i < N; i++) {
        prefetch(&a[i+P]);
        prefetch(&b[i+P]);
        prefetch(&c[i+P]);
        c[i] = a[i] + b[i];
}
```

- a. Da
- b. Nu ✔

Your answer is correct.

Prima variantă este mai rapidă fiindcă vectorii a, b și c sunt accesați în ordinea în care sunt stocați in memorie. În varianta a doua ei sunt accesați în aceeași ordine, elementele lor vor putea fi găsite în cache, deci apelurile către prefetch vor adăuga doar overhead.

The correct answer is:
Nu

Question **7**

Correct

Mark 1.00 out of 1.00

Problema cu un cache ce folosește o politică Write-Back este:

- a. Nu menține consistența datelor din memoriile unui sistem multi-procesor ✔
- b. Scade performanța unui cache
- c. Riscă să ocupe toată lățimea de bandă a magistralei procesor-memorie
- d. Nicio opțiune
- e. Necesită un TLB separat pentru a asigura transferul de date

Cum datele modificate în cache nu sunt modificate imediat și în RAM, într-un sistem multi-procesor trebuie sa implementăm un protocol de coerență a datelor dacă folosim un cache Write-Back.

The correct answer is: Nu menține consistența datelor din memoriile unui sistem multi-procesor

Question **8**

Correct

Mark 1.00 out of 1.00

Care funcție de mapare cache are cele mai puține șanse să producă thrashing (adică oferă șanse minime ca două blocuri să concureze la stocarea în aceeași linie)?

- a. Complet-asociativă ✔
- b. Set-asociativă
- c. Toate trei
- d. Nu se poate determina
- e. Directă

Cum orice linie de memorie poate intra în orice linie de cache, un cache complet-asociativ oferă cea mai bună șansă de a evita thrashing.

The correct answer is: Complet-asociativă

Question **9**

Correct

Mark 1.00 out of 1.00

Magistrala de comunicație serială sincronă SPI:

- a. Permite interconectarea mai multor dispozitive master
- b. Are un baud rate de maxim 115200kbps
- c. Permite selectarea dispozitivului slave printr-un semnal de chip-select ✔
- d. Alocă o adresă de 16 biți pentru orice dispozitiv slave
- e. Este folosită in retelele 5G

SPI este un protocol de comunicare single-master, multi-slave, unde slave-ul activ este selectat printr-un semnal numit chip-select.

The correct answer is: Permite selectarea dispozitivului slave printr-un semnal de chip-select

Question **10**

Correct

Mark 1.00 out of 1.00

O locație de memorie și una mapată într-un periferic pot fi accesate simultan de către un procesor.

- a. Adevărat
- b. Fals ✔

Perifericele și memoria sunt mapate în același spațiu de adresă, deci nu pot fi accesate simultan.

The correct answer is: Fals

◄ Colocviu

Jump to...

Examen - Partea II ►

Started on	Friday, 19 February 2021, 8:19 AM
State	Finished
Completed on	Friday, 19 February 2021, 8:51 AM
Time taken	32 mins 11 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Bill dorește să construiască un cip de memorie de 256K x 32, dar are doar cipuri de 32K x 8. De câte cipuri are nevoie Bill?

- a. 64
- b. 16
- c. 32 ✔
- d. 8
- e. 128

(256K / 32K) * (32 / 8) = 32

The correct answer is: 32

Question **2**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 4ns iar rata de hit locală este 90%
- Latența de acces din L1 în L2 este de 10ns iar rata de hit locală este de 95%
- Latența de acces din L2 în memoria RAM este de 500ns

Care este timpul mediu de acces la memorie?

- a. 0.075ns
- b. 7.5ns ✔
- c. 0.75ns
- d. 75ns
- e. 750ns

0.9 * 4ns + 0.1 * (0.95 * 14ns + 0.05 * 514ns) = 7.5ns

The correct answer is: 7.5ns

Question **3**

Correct

Mark 1.00 out of 1.00

Un sistem cu cache L1 si L2 are un hit rate de 80% din L1 și de 90% din L2. Care este miss rate-ul global?

- a. 20%
- b. 0.02%
- c. 9.8%
- d. 2% ✔
- e. 98%

1 - 0.8 - 0.2 * 0.9 = 0.02

The correct answer is: 2%

Question **4**

Correct

Mark 1.00 out of 1.00

Incrementarea unei variabile exploatează:

- a. Niciuna
- b. Nu se poate determina
- c. Ambele localități
- d. Localitatea spațială
- e. Localitatea temporală ✔

Incrementarea unei variabile presupune scrieri repetate la aceeași adresă. Deci, dacă am accesat acea adresă este probabil că vom dori să o accesăm din nou în curând.

The correct answer is: Localitatea temporală

Question **5**

Correct

Mark 1.00 out of 1.00

O memorie cache cu linii de 32 octeți lungime este organizată în 32 de seturi a câte 32 linii. Dacă adresarea se face la nivel de octet, cu adrese pe 32 de biți, care este dimensiunea în biți a tag-ului pentru fiecare linie de cache?

- a. 18
- b. 17
- c. 21
- d. 19
- e. 20 ✔

~~32 - log2(32 * 32 * 32) = 17~~

32 - log2(32 * 32) = 22. Cum nu ati primit aceasta varianta, orice raspuns e corect!

The correct answers are: 20, 21, 19, 18, 17

Question **6**

Correct

Mark 1.00 out of 1.00

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before

```
for (i = 0; i < N; i++)
 a[i] = b[i] * c[i];
for (i = 0; i < N; i++)
 d[i] = a[i] * c[i];
```

After

```
for (i = 0; i < N; i++) {
 a[i] = b[i] * c[i];
 d[i] = a[i] * c[i];
}
```

- a. Nu
- b. Da ✔

Your answer is correct.

A doua variantă este mai rapidă fiindcă are mai puține bucle.

The correct answer is:
Da

Question **7**

Correct

Mark 1.00 out of 1.00

Presupuneți că acum 20 de ani ați scris un cod pentru o arhitectura x86 care petrecea jumătate din timp cu accese la memorie și cealaltă jumătate făcând calcule. Dacă am rula același cod astăzi pe o arhitectură x86-compatible, cel mai probabil el ar:

- a. Menține aceeași proporție 50-50% a timpilor de rulare
- b. Petrece mai mult timp făcând calcule
- c. Exporta calculele in Cloud
- d. Petrece mai mult timp cu accesul memoriei ✔
- e. Nu se poate determina

Viteza procesoarelor a crescut mai mult decât viteza memoriilor.

The correct answer is: Petrece mai mult timp cu accesul memoriei

Question **8**

Correct

Mark 1.00 out of 1.00

Avem un sistem de calcul cu două niveluri de cache (L1 și L2). Dacă înlăturăm nivelul L2 de cache, cum se va modifica L1 hit time?

- a. Nu se modifică ✔
- b. Crește
- c. Depinde de L2 hit time
- d. Scade
- e. Nu se poate calcula

L1 hit time este timpul de acces la o adresă ce este deja in L1 cache, deci nu este influențat de L2 cache.

The correct answer is: Nu se modifică

Question **9**

Correct

Mark 1.00 out of 1.00

Magistrala de comunicație serială RS232:

- a. Are o schemă de adresare pe 8 biți
- b. Implementează un mecanism CSMA-CD
- c. Permite conectarea punct-la-punct a două dispozitive de calcul ✔
- d. Stă la baza tuturor rețelelor de calculatoare
- e. Folosește baud rate pentru a calcula distanța optimă de transmisie

RS232 este un protocol de comunicare punct-la-punct.

The correct answer is: Permite conectarea punct-la-punct a două dispozitive de calcul

Question **10**

Correct

Mark 1.00 out of 1.00

Procesorul poate să citească din RAM în timp ce RAM-ul este reîmprospătat.

- a. Fals ✔
- b. Adevărat

RAM-ul este inaccesibil în timpul unui refresh.

The correct answer is: Fals

◄ Colocviu

Jump to...

Examen - Partea II ►

Started on	Friday, 19 February 2021, 8:19 AM
State	Finished
Completed on	Friday, 19 February 2021, 8:52 AM
Time taken	33 mins 26 secs
Grade	**9.00** out of 10.00 (**90**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Bill dorește să construiască un cip de memorie de 512K x 64, dar are doar cipuri de 16K x 8. De câte cipuri are nevoie Bill?

- a. 8
- b. 256 ✔
- c. 16
- d. 64
- e. 128

(512K / 16K) * (64 / 8) = 256

The correct answer is: 256

Question **2**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 4ns iar rata de hit locală este 90%
- Latența de acces din L1 în L2 este de 10ns iar rata de hit locală este de 95%
- Latența de acces din L2 în memoria RAM este de 500ns

Care este timpul mediu de acces la memorie?

- a. 0.75ns
- b. 7.5ns ✔
- c. 0.075ns
- d. 750ns
- e. 75ns

0.9 * 4ns + 0.1 * (0.95 * 14ns + 0.05 * 514ns) = 7.5ns

The correct answer is: 7.5ns

Question **3**

Correct

Mark 1.00 out of 1.00

Un sistem cu cache L1 si L2 are un hit rate de 80% din L1 și de 90% din L2. Care este miss rate-ul global?

- a. 9.8%
- b. 0.02%
- c. 20%
- d. 2% ✔
- e. 98%

1 - 0.8 - 0.2 * 0.9 = 0.02

The correct answer is: 2%

Question **4**

Incorrect

Mark 0.00 out of 1.00

Decrementarea unui pointer și accesarea memoriei către care acesta pointează exploatează:

- a. Localitatea temporală
- b. Nu se poate determina
- c. Localitatea spațială ✖
- d. Ambele localități
- e. Niciuna

Your answer is incorrect.

Decrementarea presupune scrieri succesive la aceeași adresă. Asta înseamnă că, dacă am accesat o adresă, este probabil că vom dori să o accesăm din nou în curând. De aici localizarea temporală.

Accesarea memorie printr-un pointer pe care îl decrementăm rezultă într-un acces secvențial. Asta înseamnă că, dacă am accesat o adresă, este probabil că vom dori să accesăm și adresele precedente. De aici localizarea spațială.

Deci sunt ambele tipuri de localizare!

The correct answer is:
Ambele localități

Question **5**

Correct

Mark 1.00 out of 1.00

O memorie cache cu linii de 8 octeți lungime este organizată în 128 de seturi a câte 4 linii. Dacă adresarea se face la nivel de octet, cu adrese pe 32 de biți, care este dimensiunea în biți a tag-ului pentru fiecare linie de cache?

- a. 21
- b. 23
- c. 20
- d. 22 ✔
- e. 19

~~32 - log2(128 * 4 * 8) = 20~~

32 - log2(128 * 8) = 22. Cum facusem o greseala initial, orice raspuns va fi punctat!

The correct answers are: 21, 20, 19, 22, 23

Question **6**

Correct

Mark 1.00 out of 1.00

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before:

```
for (i = 0; i < N; i++)
 for (j = 0; j < N; j++)
  B[j*N+i] = A[i*N+j];
```

After:

```
int BS = 8;
for (i = 0; i < N; i+=BS)
 for (j = 0; j < N; j++)
  for (k = 0; k < BS; k++)
    B[j*N+(i+k)] = A[(i+k)*N+j];
```

- a. Nu ✔
- b. Da

Your answer is correct.

Prima variantă este mai rapidă fiindcă matricea A este accesată complet în ordinea în care este stocată în memorie. În a doua variantă niciunăă dintre matrici nu este accesată în aceasta ordine.

The correct answer is:
Nu

Question **7**

Correct

Mark 1.00 out of 1.00

Presupuneți că acum 20 de ani ați scris un cod pentru o arhitectura x86 care petrecea jumătate din timp cu accese la memorie și cealaltă jumătate făcând calcule. Dacă am rula același cod astăzi pe o arhitectură x86-compatible, cel mai probabil el ar:

- a. Petrece mai mult timp cu accesul memoriei ✔
- b. Petrece mai mult timp făcând calcule
- c. Exporta calculele in Cloud
- d. Menține aceeași proporție 50-50% a timpilor de rulare
- e. Nu se poate determina

Viteza procesoarelor a crescut mai mult decât viteza memoriilor.

The correct answer is: Petrece mai mult timp cu accesul memoriei

Question **8**

Correct

Mark 1.00 out of 1.00

Avem un sistem de calcul cu un singur nivel de cache (L1). Dacă adăugam un al doilea nivel de cache (L2), cum se vor modifica următorii parametri de funcționare: L1 hit time, L1 miss rate, L1 miss penalty?

- a. L1 hit time scade, L1 miss rate crește, L1 miss penalty scade
- b. L1 hit time crește, L1 miss rate scade, L1 miss penalty scade
- c. L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty crește
- d. L1 hit time scade, L1 miss rate neschimbat, L1 miss penalty scade
- e. L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty scade ✔

Viteza de acces în primul nivel de cache rămâne neschimbată, deci L1 hit time rămâne neschimbat. Dimensiunea și funcționarea primului nivel de cache rămân neschimbate, deci miss rate rămâne neschimbat. Cum accesul la al doilea nivel de cache este mai rapid decât accesul la RAM, miss penalty scade.

The correct answer is: L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty scade

Question **9**

Correct

Mark 1.00 out of 1.00

Magistrala de comunicație serială RS232:

- a. Implementează un mecanism CSMA-CD
- b. Folosește baud rate pentru a calcula distanța optimă de transmisie
- c. Permite conectarea punct-la-punct a două dispozitive de calcul ✔
- d. Are o schemă de adresare pe 8 biți
- e. Stă la baza tuturor rețelelor de calculatoare

RS232 este un protocol de comunicare punct-la-punct.

The correct answer is: Permite conectarea punct-la-punct a două dispozitive de calcul

Question **10**

Correct

Mark 1.00 out of 1.00

Procesorul poate să citească din RAM în timp ce RAM-ul este reîmprospătat.

- a. Adevărat
- b. Fals ✔

RAM-ul este inaccesibil în timpul unui refresh.

The correct answer is: Fals

◄ Colocviu

Jump to...

Examen - Partea II ►

Started on	Friday, 19 February 2021, 8:19 AM
State	Finished
Completed on	Friday, 19 February 2021, 8:52 AM
Time taken	33 mins 15 secs
Grade	**8.00** out of 10.00 (**80**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Bill dorește să construiască un cip de memorie de 1M x 64, dar are doar cipuri de 128K x 32. De câte cipuri are nevoie Bill?

- a. 32
- b. 8
- c. 64
- d. 128
- e. 16 ✔

(1M / 128K) * (64 / 32) = 16

The correct answer is: 16

Question **2**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 5ns iar rata de hit locală este 80%
- Latența de acces din L1 în L2 este de 12ns iar rata de hit locală este de 90%
- Latența de acces din L2 în memoria RAM este de 400ns

Care este timpul mediu de acces la memorie?

- a. 0.154ns
- b. 1.54ns
- c. 154ns
- d. 1540ns
- e. 15.4ns ✔

0.8 * 5ns + 0.2 * (0.9 * 17ns + 0.1 * 417ns) = 15.4ns

The correct answer is: 15.4ns

Question **3**

Correct

Mark 1.00 out of 1.00

Un sistem cu cache L1 si L2 are un hit rate de 98% din L1 și de 99% din L2. Care este miss rate-ul global?

- a. 2%
- b. 20%
- c. 0.02% ✔
- d. 9.8%
- e. 98%

1 - 0.98 - 0.02 * 0.99 = 0.0002

The correct answer is: 0.02%

Question **4**

Incorrect

Mark 0.00 out of 1.00

Decrementarea unui pointer și accesarea memoriei către care acesta pointează exploatează:

- a. Nu se poate determina
- b. Ambele localități
- c. Niciuna
- d. Localitatea spațială ✖
- e. Localitatea temporală

Your answer is incorrect.

Decrementarea presupune scrieri succesive la aceeași adresă. Asta înseamnă că, dacă am accesat o adresă, este probabil că vom dori să o accesăm din nou în curând. De aici localizarea temporală.

Accesarea memorie printr-un pointer pe care îl decrementăm rezultă într-un acces secvențial. Asta înseamnă că, dacă am accesat o adresă, este probabil că vom dori să accesăm și adresele precedente. De aici localizarea spațială.

Deci sunt ambele tipuri de localizare!

The correct answer is:
Ambele localități

Question **5**

Correct

Mark 1.00 out of 1.00

Un cache mapat direct este organizat intern pe 4096 linii cu lățime de 32 octeți. Adresele cu care lucrează memoria cache sunt pe 32 de biți. Care este lungimea în biți a tag-ului?

- a. 15 ✔
- b. 17
- c. 16
- d. 14
- e. 18

32 - log2(4096 * 32) = 15

The correct answer is: 15

Question **6**

Incorrect

Mark 0.00 out of 1.00

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before

```
for (i = 0; i < M; i++) {
 for (j = 0; j < N; j++) {
        x[i*N+j] = 2 * x[i*N+j];
 }
}
```

After

```
for (j = 0; j < N; j++) {
 for (i = 0; i < M; i++) {
        x[i*N+j] = 2 * x[i*N+j];
 }
}
```

- a. Nu
- b. Da ✖

Your answer is incorrect.

Prima variantă este mai rapidă fiindcă accesează matricea x în ordinea în care este stocată în memorie.

The correct answer is:
Nu

Question **7**

Correct

Mark 1.00 out of 1.00

Care dintre următoarele rămâne neschimbat la translatarea unei adrese virtuale într-o adresă fizică?

- a. Page offset ✔
- b. Toate variantele
- c. Cache tag
- d. Page number
- e. TLB tag

Offset-ul în cadrul paginii rămâne același, chiar dacă adresa virtuală a paginii este diferită față de adresa fizică.

The correct answer is: Page offset

Question **8**

Correct

Mark 1.00 out of 1.00

Avem un sistem de calcul cu un singur nivel de cache (L1). Dacă adăugam un al doilea nivel de cache (L2), cum se vor modifica următorii parametri de funcționare: L1 hit time, L1 miss rate, L1 miss penalty?

- a. L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty crește
- b. L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty scade ✔
- c. L1 hit time scade, L1 miss rate neschimbat, L1 miss penalty scade
- d. L1 hit time scade, L1 miss rate crește, L1 miss penalty scade
- e. L1 hit time crește, L1 miss rate scade, L1 miss penalty scade

Viteza de acces în primul nivel de cache rămâne neschimbată, deci L1 hit time rămâne neschimbat. Dimensiunea și funcționarea primului nivel de cache rămân neschimbate, deci miss rate rămâne neschimbat. Cum accesul la al doilea nivel de cache este mai rapid decât accesul la RAM, miss penalty scade.

The correct answer is: L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty scade

Question **9**

Correct

Mark 1.00 out of 1.00

Magistrala de comunicație serială sincronă SPI:

- a. Permite interconectarea mai multor dispozitive master
- b. Permite selectarea dispozitivului slave printr-un semnal de chip-select ✔
- c. Are un baud rate de maxim 115200kbps
- d. Alocă o adresă de 16 biți pentru orice dispozitiv slave
- e. Este folosită in retelele 5G

SPI este un protocol de comunicare single-master, multi-slave, unde slave-ul activ este selectat printr-un semnal numit chip-select.

The correct answer is: Permite selectarea dispozitivului slave printr-un semnal de chip-select

Question **10**

Correct

Mark 1.00 out of 1.00

O locație de memorie și una mapată într-un periferic pot fi accesate simultan de către un procesor.

- a. Adevărat
- b. Fals ✔

Perifericele și memoria sunt mapate în același spațiu de adresă, deci nu pot fi accesate simultan.

The correct answer is: Fals

◄ Colocviu

Jump to...

Examen - Partea II ►

Started on	Friday, 19 February 2021, 8:19 AM
State	Finished
Completed on	Friday, 19 February 2021, 8:52 AM
Time taken	32 mins 51 secs
Grade	**9.00** out of 10.00 (**90**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Bill dorește să construiască un cip de memorie de 128K x 32, dar are doar cipuri de 32K x 16. De câte cipuri are nevoie Bill?

- a. 16
- b. 32
- c. 8 ✔
- d. 128
- e. 64

(128K / 32K) * (32 / 16) = 8

The correct answer is: 8

Question **2**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 3ns iar rata de hit locală este 95%
- Latența de acces din L1 în L2 este de 9ns iar rata de hit locală este de 98%
- Latența de acces din L2 în memoria RAM este de 600ns

Care este timpul mediu de acces la memorie?

- a. 405ns
- b. 40.5ns
- c. 4.05ns ✔
- d. 0.405ns
- e. 4050ns

0.95 * 3ns + 0.05 * (0.98 * 12ns + 0.02 * 612ns) = 4.05ns

The correct answer is: 4.05ns

Question **3**

Correct

Mark 1.00 out of 1.00

Un sistem cu cache L1 si L2 are un hit rate de 80% din L1 și de 90% din L2. Care este miss rate-ul global?

- a. 0.02%
- b. 9.8%
- c. 2% ✔
- d. 98%
- e. 20%

1 - 0.8 - 0.2 * 0.9 = 0.02

The correct answer is: 2%

Question **4**

Incorrect

Mark 0.00 out of 1.00

Accesul la o stivă exploatează:

- a. Niciuna
- b. Localitatea temporală ✖
- c. Localitatea spațială
- d. Nu se poate determina
- e. Ambele localități

Stivele sunt ținute contiguu în memorie și sunt accesate, în general, secvențial. Asta înseamnă că, dacă am accesat o adresă din stivă, este probabil să vrem să accesăm și adresele următoare sau precedente.

The correct answer is: Localitatea spațială

Question **5**

Correct

Mark 1.00 out of 1.00

O memorie cache cu linii de 8 octeți lungime este organizată în 128 de seturi a câte 4 linii. Dacă adresarea se face la nivel de octet, cu adrese pe 32 de biți, care este dimensiunea în biți a tag-ului pentru fiecare linie de cache?

- a. 20
- b. 22 ✔
- c. 21
- d. 19
- e. 23

~~32 - log2(128 * 4 * 8) = 20~~

32 - log2(128 * 8) = 22. Cum facusem o greseala initial, orice raspuns va fi punctat!

The correct answers are: 21, 20, 19, 22, 23

Question **6**

Correct

Mark 1.00 out of 1.00

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before:

```
for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            sum = 0.0;
            for (k = 0; k < n; k++)
                sum += a[i][k] * b[k][j];
             c[i][j] = sum;
        }
    }
```

After:

```
    for (k = 0; k < n; k++) {
        for (i = 0; i < n; i++) {
            r = a[i][k];
            for (j = 0; j < n; j++)
                c[i][j] += r * b[k][j];
        }
    }
```

- a. Da ✔
- b. Nu

Your answer is correct.

A doua variantă este mai rapidă fiindcă face mai puține accesuri la vectorul a.

The correct answer is:
Da

Question **7**

Correct

Mark 1.00 out of 1.00

Problema cu un cache ce folosește o politică Write-Back este:

- a. Riscă să ocupe toată lățimea de bandă a magistralei procesor-memorie
- b. Nicio opțiune
- c. Scade performanța unui cache
- d. Necesită un TLB separat pentru a asigura transferul de date
- e. Nu menține consistența datelor din memoriile unui sistem multi-procesor ✔

Cum datele modificate în cache nu sunt modificate imediat și în RAM, într-un sistem multi-procesor trebuie sa implementăm un protocol de coerență a datelor dacă folosim un cache Write-Back.

The correct answer is: Nu menține consistența datelor din memoriile unui sistem multi-procesor

Question **8**

Correct

Mark 1.00 out of 1.00

Cum o să fie afectați parametrii hit time, miss rate și miss penalty dacă adăugăm capabilitatea de prefetching unui cache?

- a. Hit time creste, miss rate crește, miss penalty crește
- b. Hit time neschimbat, miss rate crește, miss penalty crește
- c. Hit time scade, miss rate crește, miss penalty neschimbat
- d. Hit time scade, miss rate scade, miss penalty crește
- e. Hit time neschimbat, miss rate scade, miss penalty neschimbat ✔

Viteza de acces rămâne neschimbată, deci hit time si miss penalty ramân neschimbate. Cum funcționalitatea de prefetching va aduce date în cache înainte de a fi folosite, miss rate va scădea.

The correct answer is: Hit time neschimbat, miss rate scade, miss penalty neschimbat

Question **9**

Correct

Mark 1.00 out of 1.00

Magistrala de comunicație serială RS232:

- a. Implementează un mecanism CSMA-CD
- b. Are o schemă de adresare pe 8 biți
- c. Folosește baud rate pentru a calcula distanța optimă de transmisie
- d. Stă la baza tuturor rețelelor de calculatoare
- e. Permite conectarea punct-la-punct a două dispozitive de calcul ✔

RS232 este un protocol de comunicare punct-la-punct.

The correct answer is: Permite conectarea punct-la-punct a două dispozitive de calcul

Question **10**

Correct

Mark 1.00 out of 1.00

Procesorul poate să citească din RAM în timp ce RAM-ul este reîmprospătat.

- a. Fals ✔
- b. Adevărat

RAM-ul este inaccesibil în timpul unui refresh.

The correct answer is: Fals

◄ Colocviu

Jump to...

Examen - Partea II ►

Started on	Friday, 19 February 2021, 8:17 AM
State	Finished
Completed on	Friday, 19 February 2021, 8:54 AM
Time taken	36 mins 33 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Bill dorește să construiască un cip de memorie de 256K x 64, dar are doar cipuri de 16K x 8. De câte cipuri are nevoie Bill?

- a. 8
- b. 16
- c. 32
- d. 64
- e. 128 ✔

(256K / 16K) * (64 / 8) = 128

The correct answer is: 128

Question **2**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 2ns iar rata de hit locală este 93%
- Latența de acces din L1 în L2 este de 8ns iar rata de hit locală este de 94%
- Latența de acces din L2 în memoria RAM este de 300ns

Care este timpul mediu de acces la memorie?

- a. 0.382ns
- b. 3820ns
- c. 3.82ns ✔
- d. 382ns
- e. 38.2ns

0.93 * 2ns + 0.07 * (0.94 * 10ns + 0.06 * 310ns) = 3.82ns

The correct answer is: 3.82ns

Question **3**

Correct

Mark 1.00 out of 1.00

Un sistem cu cache L1 si L2 are un hit rate de 98% din L1 și de 99% din L2. Care este miss rate-ul global?

- a. 20%
- b. 0.02% ✔
- c. 2%
- d. 98%
- e. 9.8%

1 - 0.98 - 0.02 * 0.99 = 0.0002

The correct answer is: 0.02%

Question **4**

Correct

Mark 1.00 out of 1.00

Incrementarea unei variabile exploatează:

- a. Localitatea temporală ✔
- b. Niciuna
- c. Ambele localități
- d. Localitatea spațială
- e. Nu se poate determina

Incrementarea unei variabile presupune scrieri repetate la aceeași adresă. Deci, dacă am accesat acea adresă este probabil că vom dori să o accesăm din nou în curând.

The correct answer is: Localitatea temporală

Question **5**

Correct

Mark 1.00 out of 1.00

O memorie cache cu linii de 32 octeți lungime este organizată în 32 de seturi a câte 32 linii. Dacă adresarea se face la nivel de octet, cu adrese pe 32 de biți, care este dimensiunea în biți a tag-ului pentru fiecare linie de cache?

- a. 18
- b. 21
- c. 19
- d. 17
- e. 20 ✔

~~32 - log2(32 * 32 * 32) = 17~~

32 - log2(32 * 32) = 22. Cum nu ati primit aceasta varianta, orice raspuns e corect!

The correct answers are: 20, 21, 19, 18, 17

Question **6**

Correct

Mark 1.00 out of 1.00

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before

```
for (i = 0; i < M; i++) {
 for (j = 0; j < N; j++) {
        x[i*N+j] = 2 * x[i*N+j];
 }
}
```

After

```
for (j = 0; j < N; j++) {
 for (i = 0; i < M; i++) {
        x[i*N+j] = 2 * x[i*N+j];
 }
}
```

- a. Da
- b. Nu ✔

Your answer is correct.

Prima variantă este mai rapidă fiindcă accesează matricea x în ordinea în care este stocată în memorie.

The correct answer is:
Nu

Question **7**

Correct

Mark 1.00 out of 1.00

Care dintre următoarele rămâne neschimbat la translatarea unei adrese virtuale într-o adresă fizică?

- a. Page offset ✔
- b. Cache tag
- c. Page number
- d. Toate variantele
- e. TLB tag

Offset-ul în cadrul paginii rămâne același, chiar dacă adresa virtuală a paginii este diferită față de adresa fizică.

The correct answer is: Page offset

Question **8**

Correct

Mark 1.00 out of 1.00

Avem un sistem de calcul cu două niveluri de cache (L1 și L2). Dacă înlăturăm nivelul L2 de cache, cum se va modifica L1 hit time?

- a. Depinde de L2 hit time
- b. Nu se poate calcula
- c. Crește
- d. Nu se modifică ✔
- e. Scade

L1 hit time este timpul de acces la o adresă ce este deja in L1 cache, deci nu este influențat de L2 cache.

The correct answer is: Nu se modifică

Question **9**

Correct

Mark 1.00 out of 1.00

Magistrala de comunicație serială sincronă SPI:

- a. Are un baud rate de maxim 115200kbps
- b. Alocă o adresă de 16 biți pentru orice dispozitiv slave
- c. Permite interconectarea mai multor dispozitive master
- d. Este folosită in retelele 5G
- e. Permite selectarea dispozitivului slave printr-un semnal de chip-select ✔

SPI este un protocol de comunicare single-master, multi-slave, unde slave-ul activ este selectat printr-un semnal numit chip-select.

The correct answer is: Permite selectarea dispozitivului slave printr-un semnal de chip-select

Question **10**

Correct

Mark 1.00 out of 1.00

Translatarea adreselor din memoria virtuală este utilă chiar și atunci când dimensiunea totală a memoriei virtuale este mai mică decât dimensiunea memoriei fizice

- a. Adevărat ✔
- b. Fals

Memoria virtuală este utilă chiar și atunci când spațiul de adresă virtual este mai mic decât spațiul de adresă fizic, spre exemplu, pentru separația spațiului de adresă între procese.

The correct answer is: Adevărat

◄ Colocviu

Jump to...

Examen - Partea II ►

Started on	Friday, 19 February 2021, 8:19 AM
State	Finished
Completed on	Friday, 19 February 2021, 8:53 AM
Time taken	33 mins 36 secs
Grade	**9.00** out of 10.00 (**90**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Bill dorește să construiască un cip de memorie de 128K x 32, dar are doar cipuri de 32K x 16. De câte cipuri are nevoie Bill?

- a. 64
- b. 16
- c. 128
- d. 8 ✔
- e. 32

(128K / 32K) * (32 / 16) = 8

The correct answer is: 8

Question **2**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 4ns iar rata de hit locală este 90%
- Latența de acces din L1 în L2 este de 10ns iar rata de hit locală este de 95%
- Latența de acces din L2 în memoria RAM este de 500ns

Care este timpul mediu de acces la memorie?

- a. 750ns
- b. 0.75ns
- c. 7.5ns ✔
- d. 0.075ns
- e. 75ns

0.9 * 4ns + 0.1 * (0.95 * 14ns + 0.05 * 514ns) = 7.5ns

The correct answer is: 7.5ns

Question **3**

Correct

Mark 1.00 out of 1.00

Un sistem cu cache L1 si L2 are un hit rate de 80% din L1 și de 90% din L2. Care este miss rate-ul global?

- a. 98%
- b. 0.02%
- c. 9.8%
- d. 20%
- e. 2% ✔

1 - 0.8 - 0.2 * 0.9 = 0.02

The correct answer is: 2%

Question **4**

Correct

Mark 1.00 out of 1.00

Citirea secvențială dintr-un fișier exploatează:

- a. Ambele localități
- b. Localitatea temporală
- c. Nu se poate determina
- d. Localitatea spațială ✔
- e. Niciuna

Citire secvențială înseamnă că, dacă am accesat o adresă, este probabil că vom dori să accesăm și adresele următoare.

The correct answer is: Localitatea spațială

Question **5**

Correct

Mark 1.00 out of 1.00

Un cache mapat direct este organizat intern pe 4096 linii cu lățime de 32 octeți. Adresele cu care lucrează memoria cache sunt pe 32 de biți. Care este lungimea în biți a tag-ului?

- a. 14
- b. 16
- c. 18
- d. 17
- e. 15 ✔

32 - log2(4096 * 32) = 15

The correct answer is: 15

Question **6**

Incorrect

Mark 0.00 out of 1.00

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before

```
int i = 0;
while (i < 4*num) {
 o_functie(i);
 i++;
}
```

After

```
int i = 0;
while (i < 4*num) {
 o_functie(i);
 o_functie(i+1);
 o_functie(i+2);
 o_functie(i+3);
 i += 4;
}
```

- a. Da ✖
- b. Nu

Your answer is incorrect.

Prima variantă este mai rapidă fiindcă nu trebuie să calculăm rezultatele temporare i + 1, i + 2 si i + 3.

The correct answer is:
Nu

Question **7**

Correct

Mark 1.00 out of 1.00

Care dintre următoarele rămâne neschimbat la translatarea unei adrese virtuale într-o adresă fizică?

- a. Cache tag
- b. Page offset ✔
- c. TLB tag
- d. Page number
- e. Toate variantele

Offset-ul în cadrul paginii rămâne același, chiar dacă adresa virtuală a paginii este diferită față de adresa fizică.

The correct answer is: Page offset

Question **8**

Correct

Mark 1.00 out of 1.00

Un TLB:

- a. Ajută la reducerea timpului de acces la o locație din memoria principală ✔
- b. Nicio opțiune
- c. Are un număr limitat de acționări pe ciclu de memorie
- d. Nu poate fi folosit în sisteme multi-procesor, pentru că produce inconsistențe la accesul memoriei partajate
- e. Poate fi folosit fără probleme și pentru accesul la locații de I/O

TLB-ul ne ajută să facem mai repede translatarea dintr-o adresă virtuală într-o adresă fizică, deci face accesul la memoria principală mai rapid.

The correct answer is: Ajută la reducerea timpului de acces la o locație din memoria principală

Question **9**

Correct

Mark 1.00 out of 1.00

Magistrala de comunicație SPI:

- a. Are o topologie multi-master, multi-slave
- b. Este o magistrală sincronă de transfer a datelor ✔
- c. Are o schemă de adresare pe 8 biți
- d. Permite comunicația la distanțe mari
- e. Are o viteză de transmisie de maxim 9600 biți pe secundă

SPI este un protocol de comunicare sincron (i.e. folosește un semnal de ceas).

The correct answer is: Este o magistrală sincronă de transfer a datelor

Question **10**

Correct

Mark 1.00 out of 1.00

Translatarea adreselor din memoria virtuală este utilă chiar și atunci când dimensiunea totală a memoriei virtuale este mai mică decât dimensiunea memoriei fizice

- a. Fals
- b. Adevărat ✔

Memoria virtuală este utilă chiar și atunci când spațiul de adresă virtual este mai mic decât spațiul de adresă fizic, spre exemplu, pentru separația spațiului de adresă între procese.

The correct answer is: Adevărat

◄ Colocviu

Jump to...

Examen - Partea II ►

Started on	Friday, 19 February 2021, 8:02 AM
State	Finished
Completed on	Friday, 19 February 2021, 8:41 AM
Time taken	38 mins 58 secs
Grade	**5.00** out of 10.00 (**50**%)

Question **1**

Incorrect

Mark 0.00 out of 1.00

Bill dorește să construiască un cip de memorie de 256K x 32, dar are doar cipuri de 32K x 8. De câte cipuri are nevoie Bill?

- a. 32
- b. 8
- c. 64
- d. 16 ✖
- e. 128

(256K / 32K) * (32 / 8) = 32

The correct answer is: 32

Question **2**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 5ns iar rata de hit locală este 80%
- Latența de acces din L1 în L2 este de 12ns iar rata de hit locală este de 90%
- Latența de acces din L2 în memoria RAM este de 400ns

Care este timpul mediu de acces la memorie?

- a. 0.154ns
- b. 15.4ns ✔
- c. 1540ns
- d. 154ns
- e. 1.54ns

0.8 * 5ns + 0.2 * (0.9 * 17ns + 0.1 * 417ns) = 15.4ns

The correct answer is: 15.4ns

Question **3**

Correct

Mark 1.00 out of 1.00

Un sistem cu cache L1 si L2 are un hit rate de 95% din L1 și de 98% din L2. Care este miss rate-ul global?

- a. 99%
- b. 0.1% ✔
- c. 1%
- d. 0.01%
- e. 0.99%

1 - 0.95 - 0.05 * 0.98 = 0.001

The correct answer is: 0.1%

Question **4**

Incorrect

Mark 0.00 out of 1.00

Decrementarea unui pointer și accesarea memoriei către care acesta pointează exploatează:

- a. Ambele localități
- b. Nu se poate determina
- c. Niciuna
- d. Localitatea temporală
- e. Localitatea spațială ✖

Your answer is incorrect.

Decrementarea presupune scrieri succesive la aceeași adresă. Asta înseamnă că, dacă am accesat o adresă, este probabil că vom dori să o accesăm din nou în curând. De aici localizarea temporală.

Accesarea memorie printr-un pointer pe care îl decrementăm rezultă într-un acces secvențial. Asta înseamnă că, dacă am accesat o adresă, este probabil că vom dori să accesăm și adresele precedente. De aici localizarea spațială.

Deci sunt ambele tipuri de localizare!

The correct answer is:
Ambele localități

Question **5**

Incorrect

Mark 0.00 out of 1.00

Un cache mapat direct este organizat intern pe 4096 linii cu lățime de 32 octeți. Adresele cu care lucrează memoria cache sunt pe 32 de biți. Care este lungimea în biți a tag-ului?

- a. 18
- b. 15
- c. 14
- d. 17 ✖
- e. 16

32 - log2(4096 * 32) = 15

The correct answer is: 15

Question **6**

Incorrect

Mark 0.00 out of 1.00

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before:

```
for (i = 0; i < N; i++)
 for (j = 0; j < N; j++)
  B[j*N+i] = A[i*N+j];
```

After:

```
int BS = 8;
for (i = 0; i < N; i+=BS)
 for (j = 0; j < N; j++)
  for (k = 0; k < BS; k++)
    B[j*N+(i+k)] = A[(i+k)*N+j];
```

- a. Da ✖
- b. Nu

Your answer is incorrect.

Prima variantă este mai rapidă fiindcă matricea A este accesată complet în ordinea în care este stocată în memorie. În a doua variantă niciuna dintre matrici nu este accesată în aceasta ordine.

The correct answer is:
Nu

Question **7**

Correct

Mark 1.00 out of 1.00

Problema cu un cache ce folosește o politică Write-Through este:

- a. Nu poate fi folosită în sisteme multi-procesor, chiar dacă CPU-urile fac snooping pentru un update
- b. Scade performanța cache-ului ✔
- c. Nici o variantă
- d. Necesită un overhead suplimentar în cache pentru a contabiliza blocurile care au fost modificate
- e. Riscă să lase memoria principală invalidă pentru o perioada lungă de timp

Faptul că la fiecare scriere cache-ul trebuie să facă o scriere și în RAM înseamnă că nu poate fi la fel de rapid ca un cache care nu trebuie să facă asta.

The correct answer is: Scade performanța cache-ului

Question **8**

Incorrect

Mark 0.00 out of 1.00

Avem un sistem de calcul cu un singur nivel de cache (L1). Dacă adăugam un al doilea nivel de cache (L2), cum se vor modifica următorii parametri de funcționare: L1 hit time, L1 miss rate, L1 miss penalty?

- a. L1 hit time scade, L1 miss rate crește, L1 miss penalty scade
- b. L1 hit time scade, L1 miss rate neschimbat, L1 miss penalty scade
- c. L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty crește ✖
- d. L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty scade
- e. L1 hit time crește, L1 miss rate scade, L1 miss penalty scade

Viteza de acces în primul nivel de cache rămâne neschimbată, deci L1 hit time rămâne neschimbat. Dimensiunea și funcționarea primului nivel de cache rămân neschimbate, deci miss rate rămâne neschimbat. Cum accesul la al doilea nivel de cache este mai rapid decât accesul la RAM, miss penalty scade.

The correct answer is: L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty scade

Question **9**

Correct

Mark 1.00 out of 1.00

Magistrala de comunicație SPI:

- a. Are o schemă de adresare pe 8 biți
- b. Permite comunicația la distanțe mari
- c. Este o magistrală sincronă de transfer a datelor ✔
- d. Are o topologie multi-master, multi-slave
- e. Are o viteză de transmisie de maxim 9600 biți pe secundă

SPI este un protocol de comunicare sincron (i.e. folosește un semnal de ceas).

The correct answer is: Este o magistrală sincronă de transfer a datelor

Question **10**

Correct

Mark 1.00 out of 1.00

În toate sistemele multiprocesor, timpul de acces la memorie este identic pentru fiecare procesor.

- a. Fals ✔
- b. Adevărat

În cazul sistemelor NUMA (Non-Uniform Memory Access) timpul de acces la memorie diferă de la un procesor la altul.

The correct answer is: Fals

◄ Colocviu

Jump to...

Examen - Partea II ►

Started on	Friday, 19 February 2021, 8:19 AM
State	Finished
Completed on	Friday, 19 February 2021, 8:45 AM
Time taken	26 mins 15 secs
Grade	**8.00** out of 10.00 (**80**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Bill dorește să construiască un cip de memorie de 256K x 32, dar are doar cipuri de 32K x 8. De câte cipuri are nevoie Bill?

- a. 8
- b. 32 ✔
- c. 64
- d. 128
- e. 16

(256K / 32K) * (32 / 8) = 32

The correct answer is: 32

Question **2**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 4ns iar rata de hit locală este 90%
- Latența de acces din L1 în L2 este de 10ns iar rata de hit locală este de 95%
- Latența de acces din L2 în memoria RAM este de 500ns

Care este timpul mediu de acces la memorie?

- a. 75ns
- b. 7.5ns ✔
- c. 750ns
- d. 0.075ns
- e. 0.75ns

0.9 * 4ns + 0.1 * (0.95 * 14ns + 0.05 * 514ns) = 7.5ns

The correct answer is: 7.5ns

Question **3**

Correct

Mark 1.00 out of 1.00

Un sistem cu cache L1 si L2 are un hit rate de 98% din L1 și de 99% din L2. Care este miss rate-ul global?

- a. 9.8%
- b. 0.02% ✔
- c. 2%
- d. 20%
- e. 98%

1 - 0.98 - 0.02 * 0.99 = 0.0002

The correct answer is: 0.02%

Question **4**

Incorrect

Mark 0.00 out of 1.00

Accesul la o stivă exploatează:

- a. Niciuna
- b. Ambele localități
- c. Localitatea temporală ✖
- d. Localitatea spațială
- e. Nu se poate determina

Stivele sunt ținute contiguu în memorie și sunt accesate, în general, secvențial. Asta înseamnă că, dacă am accesat o adresă din stivă, este probabil să vrem să accesăm și adresele următoare sau precedente.

The correct answer is: Localitatea spațială

Question **5**

Correct

Mark 1.00 out of 1.00

Un cache mapat direct este organizat intern pe 16384 linii cu lățime de 64 octeți. Adresele cu care lucrează memoria cache sunt pe 32 de biți. Care este lungimea în biți a tag-ului?

- a. 15
- b. 12 ✔
- c. 13
- d. 14
- e. 11

32 - log2(16384 * 64) = 12

The correct answer is: 12

Question **6**

Incorrect

Mark 0.00 out of 1.00

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before

```
int i = 0;
while (i < 4*num) {
 o_functie(i);
 i++;
}
```

After

```
int i = 0;
while (i < 4*num) {
 o_functie(i);
 o_functie(i+1);
 o_functie(i+2);
 o_functie(i+3);
 i += 4;
}
```

- a. Da ✖
- b. Nu

Your answer is incorrect.

Prima variantă este mai rapidă fiindcă nu trebuie să calculăm rezultatele temporare i + 1, i + 2 si i + 3.

The correct answer is:
Nu

Question **7**

Correct

Mark 1.00 out of 1.00

Problema cu un cache ce folosește o politică Write-Through este:

- a. Scade performanța cache-ului ✔
- b. Riscă să lase memoria principală invalidă pentru o perioada lungă de timp
- c. Necesită un overhead suplimentar în cache pentru a contabiliza blocurile care au fost modificate
- d. Nici o variantă
- e. Nu poate fi folosită în sisteme multi-procesor, chiar dacă CPU-urile fac snooping pentru un update

Faptul că la fiecare scriere cache-ul trebuie să facă o scriere și în RAM înseamnă că nu poate fi la fel de rapid ca un cache care nu trebuie să facă asta.

The correct answer is: Scade performanța cache-ului

Question **8**

Correct

Mark 1.00 out of 1.00

Avem un sistem de calcul cu un singur nivel de cache (L1). Dacă adăugam un al doilea nivel de cache (L2), cum se vor modifica următorii parametri de funcționare: L1 hit time, L1 miss rate, L1 miss penalty?

- a. L1 hit time crește, L1 miss rate scade, L1 miss penalty scade
- b. L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty scade ✔
- c. L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty crește
- d. L1 hit time scade, L1 miss rate neschimbat, L1 miss penalty scade
- e. L1 hit time scade, L1 miss rate crește, L1 miss penalty scade

Viteza de acces în primul nivel de cache rămâne neschimbată, deci L1 hit time rămâne neschimbat. Dimensiunea și funcționarea primului nivel de cache rămân neschimbate, deci miss rate rămâne neschimbat. Cum accesul la al doilea nivel de cache este mai rapid decât accesul la RAM, miss penalty scade.

The correct answer is: L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty scade

Question **9**

Correct

Mark 1.00 out of 1.00

O magistrala de comunicație serială:

- a. Permite doar comunicarea punct la punct
- b. Are limitări puternice de lățime de bandă
- c. Necesita un control mai complex decât o magistrală paralelă ✔
- d. Necesită în general un număr mare de linii de comunicație
- e. Poate transfera date doar asincron

Un protocol de comunicare serial necesită un control mai complex pentru, spre exemplu, determinarea biților transmiși și a numărului acestora.

The correct answer is: Necesita un control mai complex decât o magistrală paralelă

Question **10**

Correct

Mark 1.00 out of 1.00

O locație de memorie și una mapată într-un periferic pot fi accesate simultan de către un procesor.

- a. Fals ✔
- b. Adevărat

Perifericele și memoria sunt mapate în același spațiu de adresă, deci nu pot fi accesate simultan.

The correct answer is: Fals

◄ Colocviu

Jump to...

Examen - Partea II ►

Started on	Friday, 19 February 2021, 8:19 AM
State	Finished
Completed on	Friday, 19 February 2021, 8:50 AM
Time taken	30 mins 40 secs
Grade	**9.00** out of 10.00 (**90**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Bill dorește să construiască un cip de memorie de 128K x 32, dar are doar cipuri de 8K x 16. De câte cipuri are nevoie Bill?

- a. 128
- b. 16
- c. 256
- d. 32 ✔
- e. 64

(128K / 8K) * (32 / 16) = 32

The correct answer is: 32

Question **2**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 2ns iar rata de hit locală este 93%
- Latența de acces din L1 în L2 este de 8ns iar rata de hit locală este de 94%
- Latența de acces din L2 în memoria RAM este de 300ns

Care este timpul mediu de acces la memorie?

- a. 0.382ns
- b. 382ns
- c. 38.2ns
- d. 3.82ns ✔
- e. 3820ns

0.93 * 2ns + 0.07 * (0.94 * 10ns + 0.06 * 310ns) = 3.82ns

The correct answer is: 3.82ns

Question **3**

Correct

Mark 1.00 out of 1.00

Un sistem cu cache L1 si L2 are un hit rate de 80% din L1 și de 90% din L2. Care este miss rate-ul global?

- a. 9.8%
- b. 0.02%
- c. 20%
- d. 2% ✔
- e. 98%

1 - 0.8 - 0.2 * 0.9 = 0.02

The correct answer is: 2%

Question **4**

Correct

Mark 1.00 out of 1.00

Parcurgerea unei liste înlănțuite alocate contiguu în memorie exploatează:

- a. Niciuna
- b. Localitatea spațială ✔
- c. Localitatea temporală
- d. Nu se poate determina
- e. Ambele localități

Lista este alocată contiguu în memorie, iar parcurgere rezultă în acces secvențial. Asta înseamnă că, dacă am accesat o adresă din listă, este probabil să vrem să accesăm și adresele următoare sau precedente.

The correct answer is: Localitatea spațială

Question **5**

Correct

Mark 1.00 out of 1.00

Un cache mapat direct este organizat intern pe 16384 linii cu lățime de 64 octeți. Adresele cu care lucrează memoria cache sunt pe 32 de biți. Care este lungimea în biți a tag-ului?

- a. 11
- b. 12 ✔
- c. 13
- d. 14
- e. 15

32 - log2(16384 * 64) = 12

The correct answer is: 12

Question **6**

Incorrect

Mark 0.00 out of 1.00

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before:

```
for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            sum = 0.0;
            for (k = 0; k < n; k++)
                sum += a[i][k] * b[k][j];
             c[i][j] = sum;
        }
    }
```

After:

```
    for (k = 0; k < n; k++) {
        for (i = 0; i < n; i++) {
            r = a[i][k];
            for (j = 0; j < n; j++)
                c[i][j] += r * b[k][j];
        }
    }
```

- a. Da
- b. Nu ✖

Your answer is incorrect.

A doua variantă este mai rapidă fiindcă face mai puține accesuri la vectorul a.

The correct answer is:
Da

Question **7**

Correct

Mark 1.00 out of 1.00

Care dintre următoarele este adevărată?

- a. Un cache set-asociativ cu o singură linie per set este un cache mapat direct ✔
- b. Un cache set-asociativ cu un singur set este un cache mapat direct
- c. Un cache set-asociativ cu o singură linie per set este un cache complet-asociativ
- d. Un cache set-asociativ cu două seturi este un cache complet-asociativ
- e. Toate variantele sunt adevărate

Între seturi maparea se face în mod direct. Dacă seturile au o singură linie, înseamnă ca avem mapare directă între toate liniile, deci este un cache mapat direct.

The correct answer is: Un cache set-asociativ cu o singură linie per set este un cache mapat direct

Question **8**

Correct

Mark 1.00 out of 1.00

Avem un sistem de calcul cu un singur nivel de cache (L1). Dacă adăugam un al doilea nivel de cache (L2), cum se vor modifica următorii parametri de funcționare: L1 hit time, L1 miss rate, L1 miss penalty?

- a. L1 hit time scade, L1 miss rate neschimbat, L1 miss penalty scade
- b. L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty scade ✔
- c. L1 hit time scade, L1 miss rate crește, L1 miss penalty scade
- d. L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty crește
- e. L1 hit time crește, L1 miss rate scade, L1 miss penalty scade

Viteza de acces în primul nivel de cache rămâne neschimbată, deci L1 hit time rămâne neschimbat. Dimensiunea și funcționarea primului nivel de cache rămân neschimbate, deci miss rate rămâne neschimbat. Cum accesul la al doilea nivel de cache este mai rapid decât accesul la RAM, miss penalty scade.

The correct answer is: L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty scade

Question **9**

Correct

Mark 1.00 out of 1.00

Magistrala de comunicație serială sincronă SPI:

- a. Are un baud rate de maxim 115200kbps
- b. Este folosită in retelele 5G
- c. Permite selectarea dispozitivului slave printr-un semnal de chip-select ✔
- d. Alocă o adresă de 16 biți pentru orice dispozitiv slave
- e. Permite interconectarea mai multor dispozitive master

SPI este un protocol de comunicare single-master, multi-slave, unde slave-ul activ este selectat printr-un semnal numit chip-select.

The correct answer is: Permite selectarea dispozitivului slave printr-un semnal de chip-select

Question **10**

Correct

Mark 1.00 out of 1.00

O locație de memorie și una mapată într-un periferic pot fi accesate simultan de către un procesor.

- a. Adevărat
- b. Fals ✔

Perifericele și memoria sunt mapate în același spațiu de adresă, deci nu pot fi accesate simultan.

The correct answer is: Fals

◄ Colocviu

Jump to...

Examen - Partea II ►

Started on	Friday, 19 February 2021, 8:17 AM
State	Finished
Completed on	Friday, 19 February 2021, 8:51 AM
Time taken	33 mins 49 secs
Grade	**9.00** out of 10.00 (**90**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Bill dorește să construiască un cip de memorie de 128K x 32, dar are doar cipuri de 8K x 16. De câte cipuri are nevoie Bill?

- a. 32 ✔
- b. 128
- c. 256
- d. 64
- e. 16

(128K / 8K) * (32 / 16) = 32

The correct answer is: 32

Question **2**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 2ns iar rata de hit locală este 93%
- Latența de acces din L1 în L2 este de 8ns iar rata de hit locală este de 94%
- Latența de acces din L2 în memoria RAM este de 300ns

Care este timpul mediu de acces la memorie?

- a. 38.2ns
- b. 382ns
- c. 0.382ns
- d. 3.82ns ✔
- e. 3820ns

0.93 * 2ns + 0.07 * (0.94 * 10ns + 0.06 * 310ns) = 3.82ns

The correct answer is: 3.82ns

Question **3**

Correct

Mark 1.00 out of 1.00

Un sistem cu cache L1 si L2 are un hit rate de 80% din L1 și de 90% din L2. Care este miss rate-ul global?

- a. 20%
- b. 0.02%
- c. 98%
- d. 9.8%
- e. 2% ✔

1 - 0.8 - 0.2 * 0.9 = 0.02

The correct answer is: 2%

Question **4**

Correct

Mark 1.00 out of 1.00

Incrementarea unei variabile exploatează:

- a. Localitatea temporală ✔
- b. Nu se poate determina
- c. Localitatea spațială
- d. Ambele localități
- e. Niciuna

Incrementarea unei variabile presupune scrieri repetate la aceeași adresă. Deci, dacă am accesat acea adresă este probabil că vom dori să o accesăm din nou în curând.

The correct answer is: Localitatea temporală

Question **5**

Correct

Mark 1.00 out of 1.00

Un cache mapat direct este organizat intern pe 4096 linii cu lățime de 32 octeți. Adresele cu care lucrează memoria cache sunt pe 32 de biți. Care este lungimea în biți a tag-ului?

- a. 16
- b. 15 ✔
- c. 17
- d. 14
- e. 18

32 - log2(4096 * 32) = 15

The correct answer is: 15

Question **6**

Incorrect

Mark 0.00 out of 1.00

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before:

```
for (i = 0; i < N; i++)
 for (j = 0; j < N; j++)
  B[j*N+i] = A[i*N+j];
```

After:

```
int BS = 8;
for (i = 0; i < N; i+=BS)
 for (j = 0; j < N; j++)
  for (k = 0; k < BS; k++)
    B[j*N+(i+k)] = A[(i+k)*N+j];
```

- a. Da ✖
- b. Nu

Your answer is incorrect.

Prima variantă este mai rapidă fiindcă matricea A este accesată complet în ordinea în care este stocată în memorie. În a doua variantă niciuna dintre matrici nu este accesată în aceasta ordine.

The correct answer is:
Nu

Question **7**

Correct

Mark 1.00 out of 1.00

Problema cu un cache ce folosește o politică Write-Through este:

- a. Nici o variantă
- b. Necesită un overhead suplimentar în cache pentru a contabiliza blocurile care au fost modificate
- c. Scade performanța cache-ului ✔
- d. Riscă să lase memoria principală invalidă pentru o perioada lungă de timp
- e. Nu poate fi folosită în sisteme multi-procesor, chiar dacă CPU-urile fac snooping pentru un update

Faptul că la fiecare scriere cache-ul trebuie să facă o scriere și în RAM înseamnă că nu poate fi la fel de rapid ca un cache care nu trebuie să facă asta.

The correct answer is: Scade performanța cache-ului

Question **8**

Correct

Mark 1.00 out of 1.00

Avem un sistem de calcul cu un singur nivel de cache (L1). Dacă adăugam un al doilea nivel de cache (L2), cum se vor modifica următorii parametri de funcționare: L1 hit time, L1 miss rate, L1 miss penalty?

- a. L1 hit time scade, L1 miss rate crește, L1 miss penalty scade
- b. L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty crește
- c. L1 hit time scade, L1 miss rate neschimbat, L1 miss penalty scade
- d. L1 hit time crește, L1 miss rate scade, L1 miss penalty scade
- e. L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty scade ✔

Viteza de acces în primul nivel de cache rămâne neschimbată, deci L1 hit time rămâne neschimbat. Dimensiunea și funcționarea primului nivel de cache rămân neschimbate, deci miss rate rămâne neschimbat. Cum accesul la al doilea nivel de cache este mai rapid decât accesul la RAM, miss penalty scade.

The correct answer is: L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty scade

Question **9**

Correct

Mark 1.00 out of 1.00

Magistrala de comunicație serială RS232:

- a. Are o schemă de adresare pe 8 biți
- b. Implementează un mecanism CSMA-CD
- c. Folosește baud rate pentru a calcula distanța optimă de transmisie
- d. Stă la baza tuturor rețelelor de calculatoare
- e. Permite conectarea punct-la-punct a două dispozitive de calcul ✔

RS232 este un protocol de comunicare punct-la-punct.

The correct answer is: Permite conectarea punct-la-punct a două dispozitive de calcul

Question **10**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul ce are doar un L1 cache de 128kB este **întotdeauna** mai rapid decât un sistem ce are L1 și L2 cache cu dimensiunea totală de 128kB.

a. Adevărat

b. Fals ✔

Nu putem zice că un sistem cu un nivel de cache este **întotdeauna** mai rapid ca un sistem cu două niveluri de cache. Chiar dacă primul sistem are mai mult cache rapid (L1), există cazuri în care asta nu îl ajută să atingă o performanță mai mare ca al doilea sistem. Spre exemplu, un program poate să nu acceseze memoria deloc, caz în care cele două sisteme sunt la fel de rapide.

The correct answer is: Fals

◄ Colocviu

Jump to...

Examen - Partea II ►

I. Subiecte examen CN - 21.01.2017

Teorie

1. Care este tipul de cache cel mai potrivit pentru bus watching si care este folosit si in cadrul sistemelor multiprocesor?

a) Write-through (sigur?foarte)
b) write-back

E write-back, de ce ar fi write-through. Daca s-ar folosi write-through s-ar misca foarte greu procesoarele cand ar fi vorba de scris in memorie, plus ca bus-ul ar fi mult mai ocupat.
Protocoalele MSI si MESI sunt pentru cache-uri write-back cu snooping(adica bus watching).

si eu zic b: “Automat FSM pentru un protocol de coerență cache ce folosește cache write-back”

// Poate cineva sa explice de ce e asa ?
// Eu am vazut ca se foloseste mult wb in protocoalele MSI si MESI :D
//http://electronics.stackexchange.com/questions/285434/which-cache-type-is-better-for-bus-watching
// cnv a intrebat asta azi :))

2. Pe perioada RAM refresh, CPU nu poate scrie in RAM.

a) Adevarat
b) Fals

Nu cred, pentru ca daca memoria avea valoarea 1 si noi voiam sa scriem 0 fix cand se facea refresh faceam chiar scurt la nivelul circuitelor (aveam 0 de o parte si 1 de cealalta).
//nu ar astepta in cazul asta pana termina refresh-ul ? si dupa sa il actualizeze ?

Pai atunci raspunsul ar fi fals la intrebare, adica CPU trebuie sa astepte RAM sa termine.
Si nu cred ca ai putea sa faci refresh la o coloana si dupa sa scrii valoarea si dupa sa continui refresh-ul pentru ca ai risca sa pierzi valorile din memorie.

- **Extended data output** (EDO) DRAM is the leading type of DRAM used in mid-1990s. It is faster than FPM DRAM because it allows the CPU to access data while the refresh cycle is being set up. The typical access time for EDO DRAM is 60 ns. As the CPU speed increases beyond 200 MHz, however, the popularity of EDO DRAM gives way to the faster SDRAM.

Sursa:
https://books.google.ro/books?id=qhHsSlazGrQC&pg=PA325&lpg=PA325&dq=cpu+can+write+while+ram+is+refreshing&source=bl&ots=TiSr3ErgGu&sig=iz7zBluMIJld16YXGifUU0AcfV4&hl=ro&sa=X&ved=0ahUKEwjW1PyA1YPSAhVIcRQKHU4wCS4Q6AEIVTAI#v=onepage&q=CPU%20write%20while%20refresh&f=false

!!!!!! While a refresh cycle is occurring the memory is not available for normal read and write operations, but in modern memory this "overhead" time is not large enough to significantly slow down memory operation. https://en.wikipedia.org/wiki/Memory_refresh

3. Daca avem un procesor care necesita 10ns pentru un acces la memorie in cazul unui hit si 40ns pentru un acces la memorie in cazul unui miss, iar hit rate-ul este de 75%, care este timpul mediu de acces la memorie?

a) 7.5ns
b) 40ns
c) 17.5ns
d) 32.5ns

// cum se calculeaza?
// 10 x 0.75 + 40 x 0.25
Average Memory Access Time = Hit time + miss_rate * miss_penalty
Dar miss penalty din problema cred ca este 40-10=30ns
Deci ar veni 10+30*0.25=17.5 (ceea ce e echivalent cu ce e calculat sus pentru ca AMAT este si Hit_time * hit_rate + miss _rate * (hit_time + miss_penalty)
// Deci timpul de acces la memorie in cazul unui miss e (hit time + miss penalty)? De unde?

4. Care tip de cache e cel mai putin predispus la thrashing (mai multe blocuri din memorie sunt mapate in aceiasi linie)?

a) complet asociativ
b) set asociativ
c) mapat direct
Prosti:
// de ce?
// https://www.quora.com/What-is-cache-thrashing - al doilea raspuns
// Eu din raspuns am inteles ca cel mai putin predispus e set-asociativ
// Citisem gresit intrebarea, scz :)

5. Presupunem un sistem multiprocesor care implementeaza coerenta cache-ului folosind protocolul MESI. Procesorul A are in cache o linie care se afla in starea E(exclusive). Daca procesorul B aduce aceiasi linie in cache, in ce stare va trece procesorul A: //(Probabil voia sa spuna in ce stare trece linia)

a) Exclusive (cred. Ar fi fost Shared daca procesorul B ar fi pus linia in alt cache)
b) Shared (B cere linia, A trece pe shared si ii paseaza lui B linia - care tot shared va fi)
c) Invalid
d) Modified because writing to main memory is much slower than writing to cpu cache, and the data might be short during (means might change again sooner, and no need to put the old version into memory). It's complex, but more sophisticated, most memory in modern cpu use this policy. – Eric Wang Aug 8 '16 at 4:32

add a comment

e) nu se poate preciza

Wikipedia:

Exclusive (E)

The cache line is present only in the current cache, but is *clean* - it matches main memory. It may be changed to the **Shared** state at any time, in response to a read request. Alternatively, it may be changed to the Modified state when writing to it.

- Dacă o linie este în starea M atunci nici un alt cache nu poate să aibă o copie a linei!
  - Memoria rămâne coerentă, nu pot exista copii diferite

// Presupun ca se aplica si pentru starea E (amandoua sunt exclusive)

Eu cred ca este Shared, pentru ca atunci cand se face request de catre B, A vede cererea, si (pentru ca nu e o valoare noua) va trece in shared. De asemenea, memoria ii va da valoarea cred lui B, nu cache A. Cache-urile isi dau valorile doar daca era vorba de starea M (din ce am inteles din ppt-ul pus pe cs.curs)

Adevarat ce scrie mai sus. De pe wikipedia: In this step, a BusRd is posted on the bus and the snooper on P1 senses this. It then flushes the data and changes its state to shared. The block on P3 also changes its state to shared as it has received data from another cache. There is no main memory access here. (Unde P1 = A si P3 = B)

6. Care este cel mai lent tip de cache (necesita cel mai lung timp de cautare)?

a) complet asociativ
b) set asociativ
c) mapat direct

7. O problema care poate aparea in cazul cache-urilor write-through este:

a) poate duce la perioade lungi in care memoria e indisponibila ***//***I guess
b) nu poate fi folosit in cadrul sistemelor multiprocesor, chiar daca se face snooping pentru update-uri
c) mareste timpul de scriere in memorie ← curs 2 slide 41
d) nicio varianta de mai sus

// Prin memorie se refera la cache sau la DRAM?

// Eu am intuitia ca raspunsul e **a)**, pentru ca timpul de scriere nu e afectat, ci doar DRAM-ul devine indisponibil => cache indisponibil pt ca asteapta (de aia se apeleaza la buffering)

//Nu cred ca e c), tot atat dureaza sa scrii in memorie, de ce ar dura mai mult daca folosesti Write through, in loc de write back, sa zicem? Doar ca la write back nu scrii si in RAM de fiecare data cand modifici in Cache. Dar timpul de scriere ramane acelasi. Prin urmare, merg pe **a).**

// clar nu e c). in cursul 2, slide 41, scrie "Write-through încetinește memoria cache pentru a permite memoriei principale să scrie datele ", ceea ce nu coincide cu raspunsul asta.

ESTE c). Daca faci write-through, trebuie sa astepti ca valoarea sa se scrie in memorie (ceea ce dureaza mult mai mult decat daca ai scrie in cache direct). Plus ca la a), memoria e "indisponibila" doar cat se face un write (ceea ce nu e o perioada lunga). E adevarat, ca va creste numarul de transferuri pe bus catre memorie.

^ I don't think that's true. Zice timpul de scriere in MEMORIE, nu in cache. Cred ca e d).

8. Care este avantajul primar al DMA in comparatie cu Interrupt-driven I/O?

Nu blocheaza CPU (si nu polueaza cache-ul), pentru ca transfera direct datele de la I/O la memorie.

// Nici interrupt driven I/O nu blocheaza CPU:

> An alternative scheme for dealing with I/O is the interrupt-driven method. Here the CPU works on its given tasks continuously. When an input is available, such as when someone types a key on the keyboard, then the CPU is interrupted from its work to take care of the input data. In our example, the basketball player would

https://www.calvin.edu/academic/rit/webBook/chapter2/design/hardware/io/interrupts.htm

Pai il blocheaza mai putin decat pooling-ul(unde procesorul verifica mereu daca sunt date disponibile), dar aici tot procesorul trebuie sa ia datele. DMA le scrie singur.

Din curs7 am mai gasit: salveaza latime de banda si are performanta marita pt ca permite CPU si I/O in paralel.

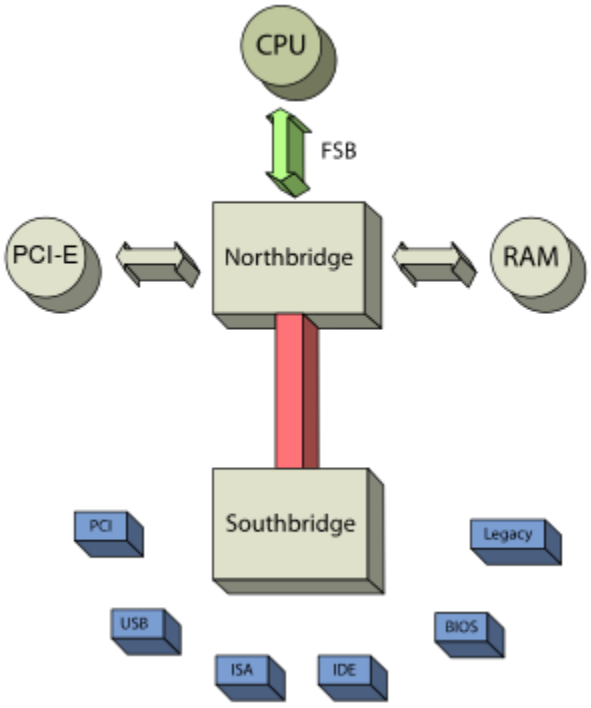
9. Dati doua sau trei exemple de probleme care pot aparea atunci cand conectam mai multe dispozitive la o magistrala.

1. **Latimea de banda a magistralei poate limita I/O throughput**
2. **Magistrala trebuie sa permita functionarea unor dispozitive cu latente si viteze de transfer diferite**
3. **Coliziuni**

10. Dati exemplu de un modul conectat la Northbridge si un modul conectat la Southbridge in calculatorul vostru.

Northbridge: RAM, PCIe

Southbridge: USB, PCI, SATA (hard diskurile)



11.

```
for(j=0; j < N; j++) {
    for(i=0; i < M; i++) {
        x[i][j] = 2 * x[i][j];
    }
}
```



```
for(i=0; i < M; i++) {
    for(j=0; j < N; j++) {
        x[i][j] = 2 * x[i][j];
    }
}
```

Ce fel de localitate exploatează schimbarea de sus?

S-a prins cineva si poate da o explicatie?

Localitate spatiala -> daca folosesti mai intai elementele de pe linii, care sunt contigue, se vor aduce in memoria cache (cel mai probabil) si blocurile urmatoare de memorie => timp mai mic pentru fetch (proprietate: cand se acceseaza o locatie de memorie este foarte probabil ca dupa sa se acceseze si locatiile vecine -> in cazul parcurgerii mai intai pe coloane, nu se indeplineste proprietatea)

La fel si al asta:

```
for(i=0; i < N; i++)
      a[i] = b[i] * c[i];

for(i=0; i < N; i++)
     d[i] = a[i] * c[i];
```



```
for(i=0; i < N; i++)
{
       a[i] = b[i] * c[i];
       d[i] = a[i] * c[i];
}
```

Dar schimbarea de sus?

Localitate temporala -> daca o locatie de memorie e accesata, e foarte probabil ca aceeasi locatie de memorie sa fie accesata si in viitorul apropiat
punem ca avem un proceso

Probleme

1. Presur cu frecventa de 2GHz care are CPI = 1.0 in cazul in care avem doar hit-uri. Avem un singur nivel de cache, care are miss rate de 3% si un timp de acces la memoria RAM de 200ns. De cate ori devine mai rapid procesorul daca introducem si un cache de nivel 2, cu miss rate de 0.2% si timp de acces la date de 2ns?

amat = hit_time + miss_rate * miss_penalty

hit_time1 = 1 / (2 * 10^9) = 0.5 ns
miss_rate1 = 3%
miss_penalty1 = 200 ns
=> amat1 = hit_time1 + miss_rate1 * miss_penalty1 = 6.5 ns

miss_rate12 = 0.2% din 3%
miss_penalty12 = 200 ns (dureaza la fel de mult sa acceseze memoria RAM)

hit_time2 = 2ns //de unde le scoti?(din enunt :)) xD //nu e miss penalty pt cache2?(cache2 la miss cauta tot in ram ca si cache1(fara 2) deci penalty-ul e acelasi..nici eu nu sunt f sigur dar cred ca prin timp de access se ref la hit_time2, altfel nici nu vad cum l-ai putea calcula din ce iti da pe acolo)
hit_time12 = 97% hit_time1 + 99.8% din 3% hit_time2 // 97% L1, 99.8% din rest L2
=> amat12 = hit_time12 + miss_rate12 * miss_penalty12

Speedup = amat12/amat1 * 100

2. Avem un benchmark care ruleaza in 100ns, din care 90ns sunt operatii pe CPU, iar in celelalte 10ns sunt operatii de I/O. In urmatorii 5 ani, performantele CPU s-au imbunatatit cu 50% la fiecare an, iar performantele I/O raman neschimbate.

a) Cu cat s-a imbunatatit performanta CPU dupa cei 5 ani?
b) Cu cat s-a imbunatatit timpul de rulare al benchmark-ului?
a) (3/2)^5
b) 1/ (0.9/(3/2)^5 + 0.1)

//de unde (3/2)^5?
Vezi problema din curs 09. Era bine cu 3/2 la a 5-a. In curs e cu 10 la a 4-a sau ceva de genul.
(3/2)^5 + 0.1)
//de unde (3/2)^5 -> (150% de 5 ori-> (150/100) * (150/100) * … -> (150/100) ^5 -> (3/2)^5)
// si atunci nu ar trebuii modificat si a) ? gata :)
De unde 150%?!?!?!?

3. Avem urmatoarele 2 secvente de cod:

Loop A:

```
int sum = 0;
for(int i = 0; i < 128; i++)
   for(int j = 0; j < 64; j++)
      sum += a[i][j];
```

Loop B:

```
int sum = 0;
for(int j = 0; j < 64; j++)
   for(int i = 0; i < 128; i++)
      sum += a[i][j];
```

Consideram ca avem un sistem cu cache pe 4KB si linii care ocupa 32 de octeti si au 8 cuvinte. De asemenea, vom considera ca procesorul citeste din memorie doar valorile tabloului **a**, celelalte variable fiind permanent stocate in registri.
Tabloul **a** este format din intregi (pe 4 octeti) si este stocat continuu in memorie, adica:

-> la adresa 0, A[0][0],
-> la adresa 4, A[0][1],
-> la adresa 8, A[0][2],
-> …..
-> la adresa 256, A[1][0] // sigur e ok?
-> …
-> la adresa (4 * 64 * 128), A[63][127] // aici e A[127][63] mai mult ca sigur :)

a) Calculati numarul de miss-uri pentru Loop A si Loop B daca cache-ul este mapat direct.

Considerand ca matricea A are 128 linii si 64 coloane si stocam in memorie linie cu linie din matrice. Vom avea:

Loop A:

For-ul Interior: 8 miss-uri ~~2 miss-uri (in cache avem 16 linii a cate 32 octeti)~~

For-ul Exterior: 128 miss-uri (la fiecare linie noua dam request + la 16 iteratii incepem sa suprascriem -- not that it would matter now)

Miss-uri Totale: 1024 (128 * 64 / 8, pentru ca la fiecare 8 accesari avem un miss)

***** Linia are 32 de bytes, dar un element din matrice ocupa 4 bytes, nu 1 byte**

Cred ca e asa:
i = 0
j = 0 miss aduce linia cu elementele (0,1,2,3,4,5,6,7) - adresele (0-31) // linie de 32B
j = 1-7 => 7 hits
j = 8 iar miss
j = 9-15 => 7 hits

deci pentru tot forul cu j face 64/8 cicluri => 8 miss + 8 * 7 hits
pentru tot forul cu i, face 128(8miss + 8*7 hits)

Loop B:

For-ul Interior: 128 miss-uri

For-ul Exterior: 64 miss-uri

Miss-uri Totale: 128 x 64

Nu cred ca e corect la LoopB. Pt ca in cache incap 128 de linii a cate 8 elem.
Cand am j=0 pun in cache primele 8 elem de pe fiecare linie.
Cand j=1, eu am deja in cache a[0][1], a[1][1],...,a[127][1]
Si o sa fie hit-uri de la j=1 pana la j=7 inclusiv.
La j=8 iar o sa am 128 de miss-uri. Dar care ma asigura pana la j=16. ETC…
Missuri Totale = 128 * (64/8) = 128 * 8
E ciudat ca da la fel ca LoopA si ma asteptam sa fie mai multe.

// Nu asa functioneaza, cache-ul incarca o linie din matrice ca e zona contigua de memorie. Nu se aplica si la coloane.

In momentul in care ceri a[0][0], cache incarca linia a[0][0]->a[0][7] (este zona contigua de mem)
Apoi cand ceri a[1][0], incarca linia a[1][0]->a[1][7]
Deci pt j=0, cache o sa incarce 128 de linii cu elementele a[linie][0]->a[linie][7]

b) Care este numarul minim de linii pentru cache astfel incat sa nu avem niciun miss atunci cand se executa Loop A (exceptand miss-urile obligatorii de la inceput) ? Dar pentru Loop B?

Loop A:

Cache: Trebuie sa avem 8 linii, fiecare a cate 64 octeti pentru a evita miss-uri pe for-ul interior.
Nu cumva doar o linie? (adica nu iti mai pasa de datele vechi) // ideea e sa ai linii de minim 64 de octeti. Cum dimensiunea cache-ului e data si fixa, nu poti controla si nr de linii din cache.
Latimea e fixa. Nu poti sa modifici dimensiunea unei linii.

Loop B:

Cache: Trebuie sa avem 128 linii, fiecare a cate 1 octet pentru a evita un numar maxim de miss-uri pe for-ul interior. Pe for-ul exterior vom avea in continuare 64 miss-uri.
Nu cumva 128 * 64 * 4 / 32? (ca sa nu incarci de doua ori aceeasi valoare) (numarul total de octeti necesari / numarul de octeti pe linie)

c) Care este numarul de miss-uri pentru Loop A si pentru Loop B daca cache-ul este complet asociativ, iar politica de inlocuire este FIFO?

Loop A:

Din anii trecuti

II. Teorie

0. Avem o memorie cache cu cuvinte de 1 octet, linii de 8 octeti lungime organizata in 32 de seturi de cate 4 linii. Care este dimensiunea utila a memoriei cache, in octeti? Daca adresarea se face pe 32 de biti, care este dimensiunea tag-ului pentru fiecare linie de cache?
1KB
5 biti set index
3 biti de offset
32 - (5 + 3) = 24

1. daca ai linii de 2 ori mai lungi in cache spune ce se intampla cu miss obligatoriu, miss de conflict si miss de capacitate

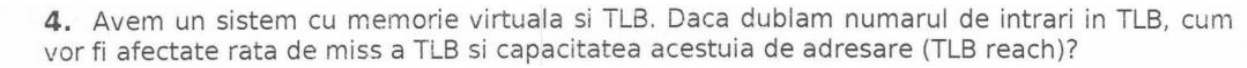
Miss obligatoriu - scade
Miss de capacitate - scade
Miss de conflict - creste (daca numarul de linii se injumatateste)

2. daca ai linii de 2 ori mai scurte (cache e la jumate din capacitate pt ca se pastreaza asociativitatea si inca ceva) ce se intampla cu hit time, miss rate si miss penalty

Hit time - scade
Miss rate - creste
Miss penalty - scade

3. Se da urmatoarea secventa de cod:
add.d f0, f1, f8
add.d f2, f3, f8
add.d f4, f5, f8
add.d f6, f7, f8
Care din urmatoarele tehnici va aduce o imbunatatire pentru codul de mai sus:
-branch prediction (n-ai instructiuni de branch)
-out of order execution cu register renaming (register renaming nu trebuie si nici out-of order nu aduce vreun beneficiu pentru ca e fix aceeasi instructiune)
-superscalar (da, pentru ca se pot executa simultan toate 4)

4. daca ai pagini de 2 ori mai mari ce se intampla cu rata de miss TLB si cu TLB reach??


4. Avem un sistem cu memorie virtuala si TLB. Daca dublam numarul de intrari in TLB, cum vor fi afectate rata de miss a TLB si capacitatea acestuia de adresare (TLB reach)?

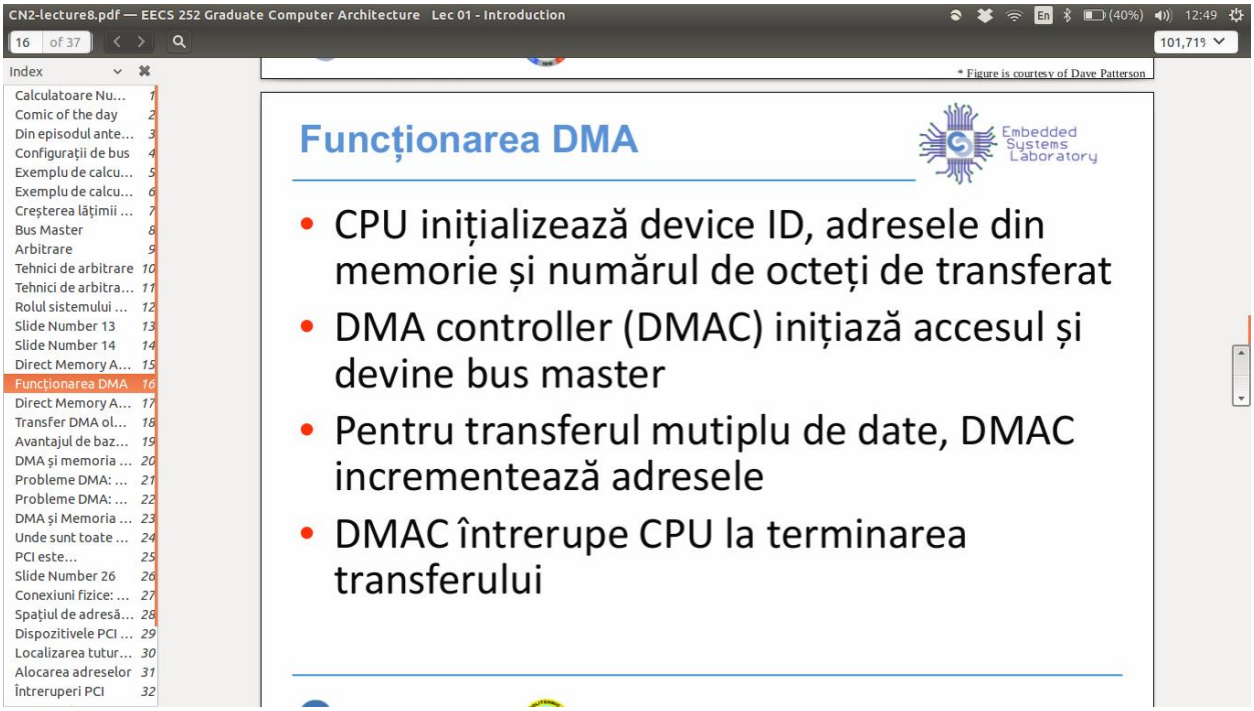
TLB reach creste. Miss la ce? // la TLB

The TLB stores the recent translations of virtual memory to physical memory and can be called an address-translation cache.
The amount of memory accessible from the TLB.
TLB Reach = (TLB Size) X (Page Size).
The working set (= un set de pagini virtuale active) of each process is stored in the TLB, Otherwise there is a high degree of page faults.

Increase the Page Size : This may lead to an increase in fragmentation as not all applications require a large page size
Provide Multiple Page Sizes : This allows applications that require larger page sizes the opportunity to use them without an increase in fragmentation

5.Care sunt componentele unei adrese de 16 biti a unui cache mapat direct, cu datele accesibile la nivel de octeti? Linia de cache are o latime de W = 4 octeti. Dimensiunea cache L=1024 linii(4kb).
Pozitia unui octet in linie este codificata pe log2(4) = 2 biti.
Adresa de index a unei linii de cache este log2(1024) = 10 biti.
Eticheta: 16 - (2 + 10) = 4 bit.

6. Descrieti, pe scurt, modul de functionare al unui controller DMA.



(Context: Direct memory access (DMA) is a feature of computer systems that allows certain hardware subsystems to access main system memory (RAM), independent of the central processing unit (CPU). Without DMA, when the CPU is using programmed input/output, it is typically fully occupied for the entire duration of the read or write operation, and is thus unavailable to perform other work. With DMA, the CPU first initiates the transfer, then it does other operations while the transfer is in progress, and it finally receives an interrupt from the DMA controller when the operation is done.)

Standard DMA, also called third-party DMA, uses a DMA controller. A DMA controller can generate memory addresses and initiate memory read or write cycles. It contains several hardware registers that can be written and read by the CPU. These include a memory address register, a byte count register, and one or more control registers. The control registers could specify the source, the destination, the direction of the transfer (reading from the I/O device or writing to the I/O device), the size of the transfer unit, and/or the number of bytes to transfer in one burst depending on what features the DMA controller provides.[1]

To carry out an input, output or memory-to-memory operation, the host processor initializes the DMA controller with a count of the number of words to transfer, and the memory address to use. The CPU then sends commands to a peripheral device to initiate transfer of data. The DMA controller then provides addresses and read/write control lines to the system memory. Each time a byte of data is ready to be transferred between the peripheral device and memory, the DMA controller increments its internal address register until the full block of data is transferred.

7. enumera 3 magistrale si la ce folosesc

Magistrala Procesor-Memorie : trebuie doar sa conecteze sistemul de memorie
Magistrala I/O : acomodeaza o gama larga de dispozitive I/O
Backplane Bus : permite procesoarelor, memoriei si dispozitivelor I/O sa coexiste

8. in ce consta consistenta secventiala (asta era la multiprocesoare)
APD ftw
^ didn't you mean *fml*? ← INDEED!

9. Explicati cum sunt afectate cele 3 tipuri de miss(obligatoriu, de conflict si de capacitate) daca adaugam capabilitatea de prefetching unui cache?
Obligatoriu scade, capacitate ramane la fel, conflict (s-ar putea sa scada sau ramane la fel)
^ nu creste missul de conflict? e posibil sa poluezi cache-ul.
// si capacitatea si conflicturi sunt afectate negativ din cauza poluarii

10.Cum sunt afectati parametrii de Hit Time, Miss Rate si Miss Penalty daca dublam asociativitatea unui cache set-asociativ(dar pastram capacitatea si dimensiunea liniilor constante)?
Hit_time creste, miss rate scade, miss_penalty creste

11.Care dintre cele 3 abordari furnizeaza cea mai mare imbunatatire a performantei la rularea urmatorului cod:
LD R1 0(R2) #cache miss
ADD R2 R1 R1
LD R1 0(R3) #cache hit
LD R3 0(R4) #cache hit
ADD R3 R1 R3
ADD R1 R2 R3
Variante:

-branch predictioni// (care-i faza de e recurenta treaba cu branch prediction?)
-out of order execution cu register renaming (nu prea cred ca ar imbunatati mult)
-superscalar (https://en.wikipedia.org/wiki/Superscalar_processor)

12. Avem un sistem cu memorie virtuala si TLB. Daca dublam numarul de intrari in TLB, cum vor fi afectate rata de miss a TLB si capacitatea acestuia de adresare(TLB reach)?

(vezi si ex 4 de mai sus tot cu TLB)
TLB Reach = (TLB Size) X (Page Size).
TLB Size creste => TLB Reach va creste => mai multa memorie va fi accesibila din TLB
Cum mai multa memorie e accesibila => sunt sanse mai mari ca adresa pe care o cautam sa existe in TLB, deci rata de miss ar trebui sa scada.

13.

5. Monica este studenta in anul 1 la Calculatoare si doreste sa imbunatateasca viteza de rulare a codului ei. Ea modifica in felul urmator programul ei:

Before

```
for (i=0; i<N; i++)
 for (j=0; j<N; j++)
  B[j*N+i] = A[i*N+j];
```

After

```
int BS=8;
for (i=0; i<N; i+=BS)
 for (j=0; j<N; j++)
  for (k=0; k<BS; k++)
   B[j*N+(i+k)] = A[(i+k)*N+j];
```

A procedat Monica corect?

Monica: Pare ca Monica a facut cache tiling, deci a procedat corect.

14.

6. Prietenul Monicai, Irinel, are aceeasi problema:

Before

```
for (i=0; i<N; i++)
 c[i] = a[i] + b[i];
```

After

```
int P=1;
for (i=0; i<N; i++) {
        prefetch(&a[i+P]);
        prefetch(&b[i+P]);
        prefetch(&c[i+P]);
        c[i] = a[i] + b[i];
}
```

Se executa mai rapid codul dupa optimizarea lui Irinel?

Da, adica face prefetch.

15.

7. Avem un sistem de calcul cu un singur nivel de cache (L1) Daca adaugam un al doilea nivel de cache (L2), cum se vor modifica urmatorii parametri de functionare: L1 Hit Time, L1 Miss Rate, L1 Miss Penalty?

L1 hit time scade, l1 miss penalty scade, l1 miss rate... (ramane la fel!?) // miss rate creste, deoarece L1 este un cache mai mic

16.

8. Descrieti protocolul MESI.

17.

9. Explicati modul de functionare al unui TLB.

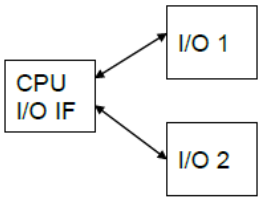
18.

10. In sistemele de calcul moderne se folosesc din ce in ce mai mult magistrale de comunicatie seriala (SATA, USB etc). De ce sunt acestea preferate magistralelor paralele (dati cel putin trei motive)?

Linii seriale punct-la-punct dedicate

Embedded Systems Laboratory

- Legăturile punct-la-punct funcționează la viteze multi-gigabit folosindcodificări avansate pentru ceas/semnale de date (necesită multă electronică la ambele capete ale comunicației)
- Low power – avem o singură legătură de date cu un singur periferic



- SATA, USB, Firewire, etc.
- Transferuri simultane multiple
- Conectori și cabluri ieftine (mai puțin cupru pentru cabluri/ mai mult siliciu pentru logica de comunicație)
- Fiecare dispozitiv are legătura lui de date, cu parametri proprii de viteză și comunicație
- Exemple: Ethernet, Infiniband, PCI Express,

19. Alex este student in anul I la Calculatoare si doreste sa imbunatatească viteza de rulare a codului lui. El modifica in felul urmator programul lui:

Before :

```
for(i = 0; i < M; i++)
        for(j = 0; j < N;j++)
                x[i*N+j] = 2*x[i*N+j];
```

After

```
for(j = 0; j < N;j++)
        for(i = 0; i < M; i++)
                x[i*N+j] = 2*x[i*N+j];
```

Unde x este un vector de intregi de 32 de biti. A procedat Alex corect?
Nu prea, ca nu se mai foloseste de localitatea spatiala

20.Prietena lui Alex, Antonia are aceeasi problema:
Before:

```
for(i = 0; i < N; i++)
        a[i] = b[i]*c[i];
for(i = 0; i < M; i++)
        d[i] = a[i]*c[i];
```

After:

```
for(i = 0; i < N; i++){
        a[i] = b[i]*c[i];
        d[i] = b[i]*c[i];
}
```

Se executa mai rapid codul dupa imbunatatirea Antoniei?

- **Loop fusion - se executa mai rapid**

II. Probleme

1. 2014

1. Un procesor are doua niveluri de cache (L1 si L2) si o memorie RAM. ~~Sistemul are un CPI=1 daca~~ Programul care ruleaza pe aceasta arhitectura are 10000 de operații load și store care au 10% hit in L1, 30% hit in L2 iar restul rezida in RAM. Timpii de acces pentru diferitele memorii sunt: t_{L1} = 2cicli, t_{L2}=5 cicli si t_{RAM}=100 cicli. Care este valoarea timpului mediu de acces la memorie? (1p)

2. Un bus sincron are o perioadă de ceas de 10ns și fiecare transmisie durează un ciclu de ceas. Alt bus asincron are nevoie de 40ns pentru fiecare handshake. Fiecare bus are 32 de biți de date. Care este lățimea de bandă a fiecărui bus pentru citiri de un cuvânt dintr-o memorie cu 500ns per read? (1p)

3. Avem un procesor cu adrese pe 8 biți și o memorie cache de 16 octeți cu lungimea liniei de 2 octeți, datele fiind accesibile la nivel de octet. Timpul de acces în cazul unui hit este $\mathbf{T_{hit} = 20\ ns}$ și timpul de acces în cazul unui miss este $\mathbf{T_{miss} = 100\ ns}$. Cât va dura urmatoarea secvență de cod și ce date se vor afla în cache dacă:

a) adresele sunt **mapate direct**;(1p)

b) adresele sunt mapate **set asociativ cu 2 cai** și politica de înlocuire a liniilor este **LRU**; (2p)

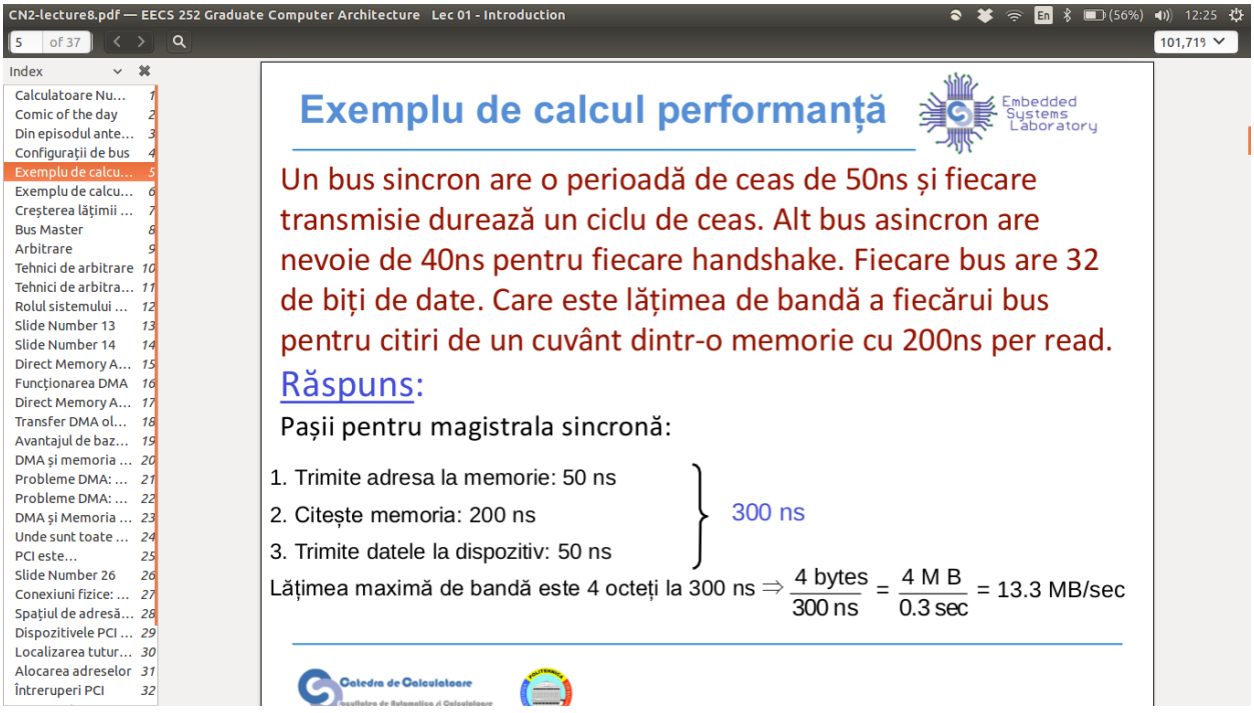
c) adresele sunt mapate **set asociativ cu 2 cai** și politica de înlocuire a liniilor este **FIFO**? (2p)

d) Ce date se vor afla în RAM la aceleași adrese (cele folosite în cod) după terminarea execuției in cele trei cazuri dacă politica de scriere în cache este **write-back**? (4p)

Se știu: RAM[8] = 0; RAM[9] = 1; RAM[16] = 2; RAM[17] = 3; RAM[20] = 4; RAM[21] = 5; RAM[32] = 6; RAM[33] = 7;

```
load R0, 8
load R1, 9
load R2, 21
load R3, 16
store 8, R1
load R0, 32
store 20, R1
```

Pentru Problema 2 -> exemplu asemanator din curs



Index
Calculatoare Nu... 1
Comic of the day 2
Din episodul ante... 3
Configurații de bus 4
Exemplu de calcu... 5
Exemplu de calcu... 6
Creșterea lățimii ... 7
Bus Master 8
Arbitrare 9
Tehnici de arbitrare 10
Tehnici de arbitra... 11
Rolul sistemului ... 12
Slide Number 13 13
Slide Number 14 14
Direct Memory A... 15
Funcționarea DMA 16
Direct Memory A... 17
Transfer DMA ol... 18
Avantajul de baz... 19
DMA și memoria ... 20
Probleme DMA: ... 21
Probleme DMA: ... 22
DMA și Memoria ... 23
Unde sunt toate ... 24
PCI este... 25
Slide Number 26 26
Conexiuni fizice: ... 27
Spațiul de adresă... 28
Dispozitivele PCI ... 29
Localizarea tutur... 30
Alocarea adreselor 31
Întreruperi PCI 32

Exemplu de calcul performanță

Embedded Systems Laboratory

Secvența de pași pentru magistrala asincronă:

1. Memory read address atunci când apare ReadReq : 40 ns
2,3,4. Data ready & handshake: max(3x40 ns, 200 ns)= 200 ns
5,6,7. Read & Ack. : 3 × 40 ns = 120 ns

} 360 ns

Lățimea maximă de bandă este 4 octeți la 360 ns $\Rightarrow \frac{4 \text{ bytes}}{360 \text{ ns}} = \frac{4 \text{ MB}}{0.36 \text{ sec}} = 11.1 \text{ MB/sec}$

Partea 1 (4p --- cica, nu stiu daca este asa cu adevarat)

1. **(cu cache, teorie si probleme)**
   - **Explicati cum sunt afectate cele 3 tipuri de miss (obligatoriu, de conflict si de capacitate) daca adaugam capabilitate de prefetching unui cache. (08.09.2014) (03.02.2014)**

--- **Miss obligatoriu scade, pentru ca sunt aduse dinainte datele.**

--- **Miss de capacitate scade**, **(in special daca face prefetch la ce trebuie si cand trebuie):** ideea e ca eu ma gandesc ca miss-ul de capacitate are loc atunci cand are loc miss indiferent de politica de inlocuire - daca eu am o politica de inlocuire perfecta si trebuie sa aduc 100MB intr-un cache de 10MB, la un moment dat tot o sa am miss, dar daca sa zicem ca nu folosesc toti 100MB unii dupa altii, e posibil sa am timp sa fac prefetch la ceva ce urmeaza si sa scap de miss-ul de capacitate.
-- Miss de capacitate -> scade. Idem Tibi. Daca avem nevoie de mai multa informatie decat poate cache-ul sa tina, tehnica de prefetch poate sa aduca informatia pe care nu am reusit sa o incarcam in cache atunci cand este nevoie.
--- **Miss de capacitate:** eu aici cred ca nu se modifica deoarece cache-ul are aceeasi capacitate si daca eu aduc niste date de care am nevoie mai devreme, inlocuind in acelasi timp alte date utile, asta nu inseamna ca per total missurile de capacitate vor scadea, ci se vor petrece mai devreme decat s-ar fi petrecut in mod obisnuit. (mie mi se pare normal sa ramana la fel, desi inteleg ce zici, Tibi, dar capacitatea cache-ului ramane aceeasi si eu cand fac prefetch doar inlocuiesc datele din cache cu alte date, cum a zis Miruna, deci mi se pare ca folosesc cache-ul cum se foloseste in mod normal si ca nu se aduce nicio imbunatatire)

--- **Miss de conflict ar putea scadea in cazul in care valorile care sunt mapate in aceeasi locatie din cache nu sunt folosite atat de rapid una dupa alta, incat sa nu aiba timp prefetch-ul sa le inlocuiasca atunci cand e nevoie de ele. In schimb, daca ele sunt folosite una dupa alta, prefetch-ul nu imbunatateste cu nimic viteza, deoarece nu se poate face prefecth intr-o locatie care e folosita in mod curent.**

   - **Daca ai linii de 2 ori mai lungi in cache spune ce se intampla cu miss obligatoriu, miss de conflict si miss de capacitate (28.01.2014)**

Ok, deci aici dimensiunea ramane constanta iar ele sunt de doua ori mai lungi.
---Deci in cazul asta scad miss urile obligatorii pentru ca aducem mai multe date,
---scad miss urile de capacitate (creste dimensiune cache). ??? nu zisesem ca dimensiunea cache ramane aceeasi? (acum am vazut ca in curs spune ca scad la reload (ce o mai fi insemnand si aia), si ca a ramas in dubiu si cu Dan Tudose de ce ele scad) aici ma gandesc ca e posibil sa fie acelasi motiv pentru care scad miss-urile de obligatorii si anume ca se aduc mai multe date => scade nevoia de a tot inlocui date in cache cu unele din RAM => mai putine sanse de a da miss.

---Aici in curs scrie ca CRESC miss urile de conflict (scuze :">).
Eu zic ca miss-ul de capacitate nu scade.(daca nu se umbla la dimensiunea chache-ului, nu cred ca se afecteaza miss-urile de capacitate) Ar putea sa creasca in cazul in care trebuie accesate adresele de forma 10k unde liniile au dimensiune 9. ca la un moment dat se umple cache-ul de linii si trebuie invalidate, dar daca liniile erau doar de un octet se umplea doar cu cate un octet, nu cu cate 9 si aveau loc toate. (totusi, cele de capacitate parca erau influentate strict de dimensiunea cache-ului. Daca eu cer adrese care imi dezorganizeaza cache-ul, nu cred ca e de capacitate)

- **Cum sunt afectati parametrii de hit time, miss rate, miss penalty daca adaugam capabilitate de prefetching unui cache? (10.02.2015)**

--- hit time ramane la fel
--- scad miss rate (1 - hit rate)
--- miss penalty ramane constant

- **Cum sunt afectati parametrii de hit time, miss rate, miss penalty daca dublam asociativitatea unui cash set-asociativ (dar pastram capacitatea si dimensiunea liniilor constante)? (08.09.2014) (03.02.2014)**

Tinda sa reduca miss uri conflictuale deci scad miss rate, creste hit time , creste miss penalty (explicatie Tibi)

ideea e ca un set are acum mai multe linii daca creste asociativitatea
full associative inseamna un set care cuprinde toate liniile, adica toate pot fi mapate oriunde asta daca folosim "set" asa cum e el denumit de fapt, nu cum a zis Dan Tudose la curs, cand se referea la set ca locul in cadrul unui index

--- creste hit time pentru ca ii ia mai mult sa caute
--- scade miss rate pentru ca scad miss-urile de conflict (bine-nteles trebuie o politica buna)
---- miss penalty creste pt ca adaugi cautarea + inlocuirea (mai faci si politica sa vezi pe care o inlocuiesti, sunt mai multe locuri in care poti inlocui) dar politica de inlocuire exista de la inceput si daca este una perfecta, atunci nu ar trebui sa dureze mai mult pana inlocuieste datele, nu?

- **Daca ai linii de 2 ori mai scurte (cache e la jumate din capacitate pt ca se pastreaza asociativitatea si inca ceva, probabil dimensiunea liniilor) ce se intampla cu hit time, miss rate si miss penalty (28.01.2014)**

- hit time: scade --------> pentru ca dureaza mai putin sa aduci datele din cache (eu aici as fi zis ca ramane la fel, ca pana la urma din cache cred ca se ia doar ce date se dorec din linia aia) si eu zic tot ca ramane la fel
- miss rate: creste ----------> mai putine date in cache
- miss penalty: scade ---------> (cred ca aici scade miss penalty deoarece se aduc din RAM calupuri de date mai mici, deci se ocupa mai putina latime de banda pe magistrala, timp de transfer mai scurt, probabil) (si eu cred ca scade, ia mai putin timp sa aduci datele din RAM) si eu zic tot ca scade

- miss de capacitate: creste
- miss obligatoriu: la fel creste ca sunt aduse mai putine instructiuni si deci trebuie facute mai multe accese la memorie obligatorii
- miss de conflict: scade? creste ca sunt mai multe linii in RAM de mapat, deci concureaza mai multe pe acelasi loc in cache

---

Inainte de probleme, poate ar fi bine sa scriem niste formule de referinta (daca sunt gresite, yell and modify :))

Structura : tag index offset

- **dimensiune linie cache = 2 ^ offset**
- **dimensiune cache = 2 ^ index linii**
- **dimensiune cache = 2 ^ offset * 2 ^ index * nr_seturi**
  = 2 ^ (offset + index) * nr_seturi

- **Avem o memorie cache cu cuvinte de un octet, linii de 8 octeti lungime organizata in 32 de seturi de cate 4 linii. Care este dimensiunea utila a memoriei cache, in octeti? Daca adresarea se face pe 32 de biti, care este dimensiunea tag-ului pt. fiecare linie de cache? (10.02.2015)**

(
fiecare linie are 8 octeti | fiecare set are 4 linii => 4*8 octeti = 32 octeti | sunt 32 de set-uri => 32*32 = 1024 octeti dimensiune totala cache

linii de 8 octeti => adresare pe 3 biti. (offset)
32 de seturi => adresare pe 5 biti (index)
=> tag-ul e de 24 de biti
)

- **Care sunt componentele unei adrese pe 16 biti a unui cache mapat direct, cu datele accesibile la nivel de octet. Linia de cache are o latime de W = 4 octeti. Dimensiunea cache L = 1024 linii (4kB)? (08.09.2014)** (28.01.2014)

Structura : tag index offset
W = 4 octeti = 4 * 8 biti = 32 biti
latime_linie = 2 ^ offset = 32 => offset = log2(32) = 5 biti pentru offset (aici datele sunt accesibile al nivel de octet => calcul cu cei 4 octeti, nu mai transformi in biti => 2 biti pt offset) da, 2 biti aici pt offset
L = 1024 linii = 2 ^ index => index = log2(1024) = 10 biti index
16 - 10 - 5 = 1 bit pentru tag (si aici cred ca ar veni: 4 biti pt tag ramasi) si eu cred ca rezultatul e 4b pt tag (model: slide 30, curs 2)

Raspuns: (2 biti offset, 10 biti index, 4 biti tag)

- **Avem un sistem de calcul cu un singur nivel de cache (L1). Daca adaugam un al doilea nivel de cache (L2), cum se vor modifica urmatorii parametri de functionare: L1 hit time, L1 miss rate, L1 miss penalty? (03.02.2014)**

Scade miss penalty (mai mic pentru a aduce date din L2),
miss si hit raman la fel (nu pot aduce mai multe date sau modifica dimensiune deci…)

In curs scrie asa: (curs 3, slide 33):

- **hit time mult mai mic (pentru ca prezenta lui L2 duce la un L1 mai mic)** **buna observatie, nu m-as fi gandit - dar trebuie specificat ca schimbi L1 daca zici asta**
- **miss rate mai mare (pentru ca L1 e mai mic)**
- **miss penalty mai mic pt L1 (nu scrie de ce, dar probabil pentru ca suporta L2 miss penalty-ul)**

- **Care din urmatoarele sunt adevarate? (10.02.2015)**
  - **un cache set-asociativ cu o linie pe set este un cache complet asociativ**
  - **un cache set-asociativ cu un singur set este un cache complet asociativ** **da da da oui** да
  - **un cache set-asociativ cu o linie pe set este un cache mapat direct** **da da da oui** да
  - **un cache set-asociativ cu un singur set este un cache mapat direct**

2. (cu cod imbunatatit sau nu dupa modificare)

- **Se modifica cod pentru a functiona mai repede. Este ok? (10.02.2015)**

Before

```
int main(int argc, char **argv) {
      int a, b;
      for(i = 0; i < NUM; i++) {
            a = rand()%100;
            b = rand()%100;
            a = (a > b) ? a : b;
            printf("%d\n", a);
      }
return 0;
}
```

After

```
int max(int a, int b) {
  return (a > b) ? a : b;
}
void rand_ab(int *a, int *b){
     *a = rand()%100;
     *b = rand()%100;
     return;
}
int main(int argc, char **argv) {
      int a, b;
      for(i = 0; i < NUM; i++) {
            rand_ab(&a, &b);
            a = max(a, b);
            printf("%d\n", a);
      }
return 0;
}
```

Nu (scade localitatea spatiala) (nu este vorba de localitatea temporala?: gen a si b din primul exemplu sunt tot folosite, pe cand in exemplul 2 se aloca alti pointeri de a si de b, cred), si pe de alta parte mai face si apeluri de stiva multe pt chemare functiilor alora => ineficient
(Cred ca accentul trebuie sa fie pus pe apelul de stiva, nu cred ca se refera neaparat la localizare temporala/spatiala) asa e, nu avem vectori, deci scade localitatea temporala

- **Se modifica cod pentru a functiona mai repede. Este ok? (10.02.2015)**

Before

```
void f(int** a, int** b) {
for(j = 0; j < ROWMAX; j += 1)
 for(i = 0; i < COLMAX; i += 1)
  a[i][j] = b[i][j] * 3330;
}
```

After

```
void f(int** a, int** b) {
for(j = 0; j < COLMAX; j += 1)
 for(i = ROWMAX-1; i >= 0; i -= 1)
  a[j][i] = b[j][i] * 3330;
}
```

Nu(prima foloseste localitate spatiala). In memoria matricele sunt reprezinta ca o zona contigua de memorie. de fapt e cum zice Andrei mai jos, a doua are localitatea sptiala mai buna
(eu pe asta nu o inteleg … adica fooseste j drept contor de linii, dar in a il scrie asa: a[i][j] … adica drept index de coloane …. chestia asta da segfault daca i si j sunt diferite … gresesc?) Nu observasem asta, dar desi trece de bariera logica a notatiilor, sunt totusi niste variabile, deci poate sa le ia si sa le numeasca cum vrea el.
Le numeste asa ca sa te induca in eroare. Pe primul parcurg matricea coloana cu coloana (ceea ce e naspa). In al doilea, parcurg rand cu rand, dar randurile le parcurg invers. Eu cred ca se imbunatateste localitatea spatiala, deci aduce o imbunatatire.

- **Se modifica cod pentru a functiona mai repede. Este ok? (08.09.2014)**

Before

```
for (i=0; i<M; i++) {
 for (j=0; j<N; j++) {
      x[i*N+j] = 2 * x[i*N+j];
      }
}
```

After

```
for (j=0; j<N; j++) {
 for (i=0; i<M; i++) {
      x[i*N+j] = 2 * x[i*N+j];
      }
}
```

Identic cu cel anterior? cred ca da

eu cred ca aici al doilea este mai bun la localitate temporala … adica in bucla din interior dubleaza de M ori aceeasi valoare din vectorul X, pe cand in primul face dublarea la intervale de N operatii;
(eu cred ca nu se aduce o imbunatarie, la primul, parcurgerea se face pe randuri, la al doilea pe coloane => localizare spatiala mai proasta la al doilea => NU este ok)
(pai, Andrei, nu parcurgi o matrice, ci un vector …. adica nu ai coloane si linii; M-ul ala este acolo sa indice de cate ori inmultesti cu 2, nu?)
Primul are localitate spatiala mai buna pentru ca in vector creste cu cate o unitate la fiecare iteratie, nu cu cate N
Subscriu ^^^^

- **Se modifica cod pentru a functiona mai repede. Este ok? (08.09.2014)**



A doua varianta mai buna. In primul caz va invalida “a” si eventual actualiza in RAM (rau). In al doilea caz il foloseste din cache. de acord se fol astfel localitatea temporala a[i] si c[i] folosite de mai multe ori да

- **Se modifica cod pentru a functiona mai repede. Este ok? (28.01.2014, dar alt cod) (03.02.2014)**



Prima varianta foloseste localitate spatiala? Se pare ca, luand in considerare documentul pus de Tibi http://csapp.cs.cmu.edu/2e/waside/waside-blocking.pdf, a doua varianta prezinta localitate temporala, so functioneaza mai repede. Si in general, orice e organizat pe blocuri prezinta o mai buna localitate temporala.
(primul e clar ca nu merge bine pentru localitatea spatiala -> al doilea e pe blocuri -> al doilea e mai bun) algoritmul de aici nu e identic cu cel pe blocuri, dar e un pas spre ele deci are localitatea temporala mai buna

- **Se modifica cod pentru a functiona mai repede. Este ok? (28.01.2014, dar alt cod) (03.02.2014)**

Before

```
for (i=0; i<N; i++)
 c[i] = a[i] + b[i];
```

After

```
int P=1;
for (i=0; i<N; i++) {
        prefetch(&a[i+P]);
        prefetch(&b[i+P]);
        prefetch(&c[i+P]);
        c[i] = a[i] + b[i];
}
```

Mai buna prima varianta. Este prea tarziu pentru prefech si va polua cache ul. Spunea Dan ca era ok daca ar fi fost facut prefetch cu ~100 instructiuni inainte. nu imi amintesc de asta cu 100 instructiuni, dar si eu mi-am notat ca nu e buna a doua varianta.
Am notat eu asta pe curs :? -> oricum mai buna prima da, clar
(prefetch-ul e costisitor, nu? Si daca eu fac prefetch la un element, e clar ca nu se va aduce doar acel element in cache, se va aduce intreaga linie, deci urmatoarele prefetch-uri sunt degeaba. Dar daca doar maresc P-ul, de ce ar fi mai bine?)

3. **(memorie virtuala, TLB: teorie sau probleme)**

- **Daca ai pagini de 2 ori mai mari, ce se intampla cu rata de miss si cu TLB reach (28.01.2014)**

Miss rate banuiesc ca scade, avand in vedere ca avem mai multa informatie la dispozitie => sanse mai mici sa dam miss

TLB reach este direct proportional cu dimensiunea paginii, deci si el creste:

TLB reach = numar_intrari_TLB * dimensiune_pagina * numar_pagini_pe_intrare? sincer nu stiu, in curs era doar prima parte
Am si eu o intreabare aici. TLB Reach spune ca este dimensiunea celui mai mare spatiu care poate fi mapat SIMULTAN. Deci daca am fi crescut doar numarul de intrari am fi fost siguri ca este contiguu si ar fi crescut TLB Reach?

- **Avem un sistem cu memorie virtuala si TLB. Daca dublam numarul de intrari in TLB, cum vor fi afectate rata de miss a TLB si capacitatea acestuia de adresare (TLB reach)? (08.09.2014) (03.02.2014)**

Rata de miss scade.

La fel ca mai sus (dimensiune pagina = numar de intrari)

Ah , este chiat aici intreabarea. Daca nu este contiguu nu cred ca putem spune sigur de TLB Reach. Gresesc? !!!! Buna observatie, Lavinia, nici eu nu stiu cum este… :(asa e, in curs scria ca pt memorie contigua, deci nu stiu ce se intampla daca nu e asa

- **Descrieti rolul pe care il are TLB intr-o arhitectura de calcul moderrna. (10.02.2015) (03.02.2014)**

TLB (Translation Lookaside Buffer) este un cache mic care contine cele mai recente translatii adresa virtuala - adresa fizica. Are rolul de a reduce numarul de accese la memorie pentru majoritatea operatiilor load/store (de la 2 cicli la 1).

- **Ati scris un program care are nevoie frecvent de cativa kB de memorie temporara. Ce este mai rapid: sa fol in mod repetat acelasi spatiu de memorie sau sa alocam un spatiu diferit de fiecare data? De ce? (10.02.2015)**

Daca datele de intrare nu difera de la o rulare la alta, atunci e mai rapid sa folosim acelasi spatiu de memorie, pentru ca datele sunt deja in cache si cum stim deja, e mai rapid sa accesam datele din cache decat din RAM sau HDD.

Eu as zice ca mereu e mai bine sa folosim in mod repetat acelasi spatiu pentru ca imbunatateste localitatea temporala.

- **Care dintre urmatoarele sunt motive valide pt. fol. adreselor virtuale? Explicati. (10.02.2015)**
  - **permite unui proces sa acceseze intregul spatiu de memorie** ---- Da, in sensul ca ii lasa impresia procesului ca are acces la toata memoria disponibila Nu, tocmai ca e limitat in spatiul pe care i-l pune la dispozitie sistemul de operare ??? (si eu as zice ca Alina … ca macar ii da impresia ca lucreaza cu toata memoria) e cum spui, Tibi, el e spatiu limitat, dar procesul crede ca lucreaza cu toata memoria
  - mareste rata de transfer a dispozitivelor I/O tind sa spun “nu”, de ce ar fi asa? (I/O-ul nu foloseste VM, nu?) Foloseste….in sensul in care umbla cu RAM-ul si HDD-ul prin DMA, dar cred ca adresele virtuale ii ingreuneaza treba … pt ca el in esenta cred ca lucreaza cu adrese fizice Nu mareste rata de transfer, poate chiar aduce un overhead prin operatii de translatare de adrese
  - **este mai dificil ca un program ce functioneaza prost sa afecteze alte programe concurente** -------- Da, pentru ca fiecare proces are propriul spatiu de memorie virtuala
  - **simplifica implementarea functiilor de memorie (malloc, new)** (unul ditre principalele scopuri) eu cred ca asta este “da”, pt ca scrie prin curs ceva de genul ca inainte de adrese virtuale, cei care scriau programe trebuiau sa fie atenti ei la alocari de memorie sa nu intre peste alte programe imi amintesc si eu ca scria asta da, simplifica pentru ca vede tot spatiul programului ca un spatiu contiguu de adrese si ii e mai usor sa il gestioneze decat daca ar avea la dispozitie toata memoria
  - **permite upgrade-ul gratuit de al Win7 la Win10 ------- clar da, nici nu exista dubiu, daca accentul e pus pe gratuit clar nu (daca nu era memorie virtuala, managementul memoriei era foarte dificil si dependend de arhitectura. Acuma, daca accentul e pus pe “gratuit” - e o prostie, asta tine de Windows, deci nu. Dar, daca accentul e pus pe “permite”, poate si memoria virtuala sa contribuie la chestia asta si as zice “da” :))** si eu as zice tot ca

da, desi nu prea are sens :D daca accentul pica pe gratuit, atunci sigur e corect, ca fiind adrese virtuale costa mai putin, adica e gratuit.

4. (cu magistrale sau ce mai este pe la I/O)

- **La care din urmatoarele magistrale este legat CPU-ul? (10.02.2015)**

a) CPU bus; b) Memory bus; c) School bus d) I/O bus e) PCI bus

Alina: a, b asta este dubioasa pt ca in curs scrie magistrala procesor-memorie … nu este scris niciodata separat … cred ca merita intrebata daca ne pica la examen exact, stiu, de-asta le-am ales pe amandoua :(

- **Dati 3 exemple la alegere de magistrale dintr-un sistem de calcul si descrieti pe scurt la ce sunt folosite. (08.09.2014) (28.01.2014)**

1. Magistrala Procesor-Memorie:
    a. conectata direct la CPU;
    b. conecteaza sistemul de memorie;
    c. maximizeaza bandwidth-ul Memorie - CPU
    d. optimizata pentru transferuri de blocuri cache
    e. scurta si de viteza mare (detaliu)

2. Magistrala I/O:
    a. trebuie sa acomodeze o gama larga de dispozitive I/O
    b. lunga si lenta (detaliu)
    c. se conecteaza la magistrala procesor-memorie sau la backplane (detaliu)

3. Backplane bus:
    a. permite procesoarelor, dispozitivelor de I/O si memoriei sa coexiste
    b. o structura de interconectare din interiorul sasiului (detaliu)

4. posibil sa mearga povestit si despre PCI

- **In sistemele de calcul moderne se folosesc din ce in ce mai mult magistrale de comunicatie seriala (SATA, USB, etc.). De ce sunt acestea preferate magistralelor paralele (dati cel putin 3 motive)? (03.02.2014)**

- functioneaza la viteze multi-gigabit
- low power => avem o singura legatura de date per periferic
- transferuri simultane multiple
- conectori si cabluri ieftine
- fiecare dispozitiv are legatura lui de date

- **Descrieti, pe scurt, modul de functionare al unui controller DMA (asta este de la I/O). (08.09.2014) (28.01.2014)**

  DMA = direct memory access
  Pasii de functionare sunt:
  1. un dispozitiv cere transferul a S octeti de la disk controller; specifica adresa A si dimensiunea S controllerului DMA;
  2. disk controller incepe transferul DMA a S octeti;
  3. disk controller initiaza bus request pentru fiecare octet de pe disk;
  4. controller DMA transfera cate un octet la adresa A, incrementeaza A si decrementeaza S;
  5. cand S == 0, controller DMA intrerupe CPU pt a indica ca transferul e complet.

5. (de prin cursul ILP si Superscalar)

- **Se da urmatoarea secventa de cod: (28.01.2014)**
  add.d f0, f1, f8
  add.d f2, f3, f8
  add.d f4, f5, f8
  add.d f6, f7, f8

  Care din urmatoarele tehnici va aduce o imbunatatire pentru codul de mai sus:
  -branch prediction
  -out of order execution cu register renaming
  -superscalar ---> ce este asta? am gasit pe wikipedia ca e un CPU care implementeaza o forma de paralelism pe un singur procesor, executand mai multe instructiuni intr-un singur clock cycle pe diferite unitati de executie de pe CPU-ul respectiv.

  Cred ca raspunsul este superscalar, pentru ca:
  - branch prediction nu are cum sa fie, ca nu avem branchuri
  - out of order execution cu register renaming nu ajuta la nimic, ca nu avem dependente de date si rearanjarea nu ar imbunatati nimic

  Deci a ramas superscalar..si ar merge sa fie executate instructiunile in paralel din moment ce nu avem dependente de date.

- **Care dintre cele 3 abordari furnizeaza cea mai mare imbunatatire a performantei la rularea urmatorului cod: (08.09.2014) 03.02.2014)**
  LD R1 0(R2) #cache miss
  ADD R2 R1 R1
  LD R1 0(R3) #cache hit
  LD R3 0(R4) #cache hit
  ADD R3 R1 R3
  ADD R1 R2 R3

Variante:

- **out-of-order execution cu Register Renaming**
- **branch prediction**
- **superscalar**

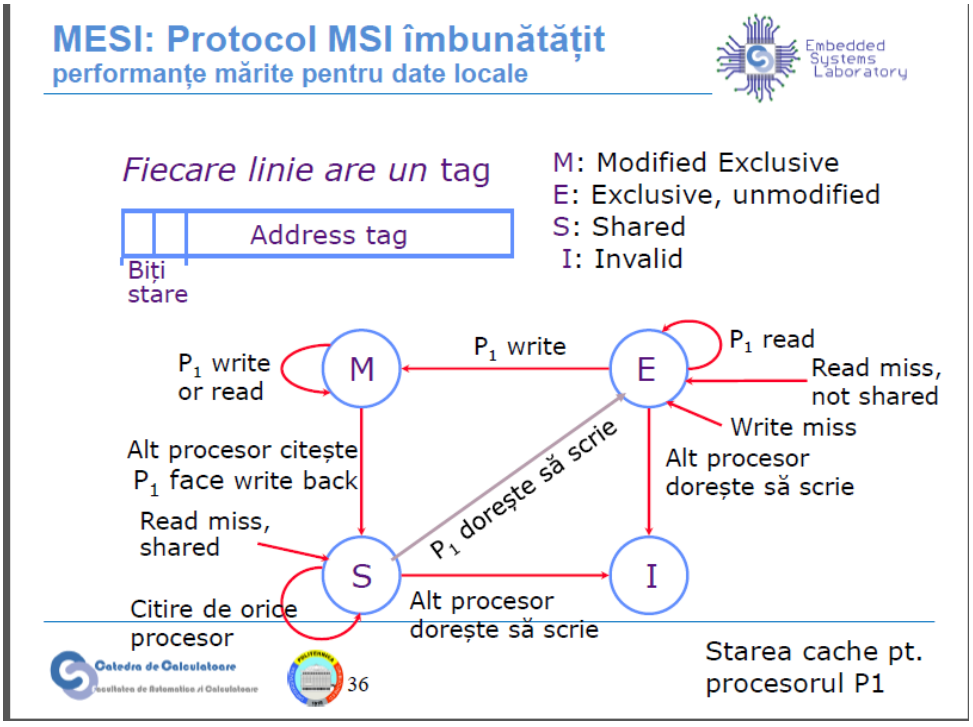
Banuiesc ca out-of-order execution, avand in vedere ca sunt dependente de date pe acolo. da, mai exact pentru ca avem WAW unde chiar e folosit register renaming.

6. (de la multiprocesor)

- **Definiti consistenta secventiala pentru un sistem multi-procesor. (08.09.2014) (28.01.2014)**

Un sistem este consistent secvential daca rezultatul oricarei executii e acelasi ca si cand operatiile tuturor procesoarelor ar fi executate secvential si operatiile fiecarui procesor individual apar in ordinea specificata in program. (Leslie Lamport)

- **Descrieti protocolul MESI. (03.02.2014) - e cam messy stiu cred ca Danut trebuie sa aiba o zi proasta ca sa o dea pe asta**



In computing, the MSI protocol - a basic cache-coherence protocol - operates in multiprocessor systems. As with other cache coherency protocols, the letters of the protocol name identify the possible states in which a cache line can be. So, for MSI, each block contained inside a cache can have one of three possible states:

- **Modified: The block has been modified in the cache. The data in the cache is then inconsistent with the backing store (e.g. memory). A cache with a block in the "M" state has the responsibility to write the block to the backing store when it is evicted.**
- **Shared: This block is unmodified and exists a read-only state in at least one cache. The cache can evict the data without writing it to the backing store.**
- **Invalid: This block is not present in the current cache, and must be fetched from memory or another cache if the block is to be stored in this cache.**

When a read request arrives at a cache for a block in the "M" or "S" states, the cache supplies the data. If the block is not in the cache (in the "I" state), it must verify that the line is not in the "M" state in any other cache. Different caching architectures handle this differently. For example, bus architectures often perform snooping, where the read request is broadcast to all of the caches. Other architectures include cache directories which have agents (directories) that know which caches last had copies of a particular cache block. If another cache has the block in the "M" state, it must write back the data to the backing store and go to the "S" or "I" states. Once any "M" line is written back, the cache obtains the block from either the backing store, or another cache with the data in the "S" state. The cache can then supply the data to the requestor. After supplying the data, the cache block is in the "S" state.

When a write request arrives at a cache for a block in the "M" state, the cache modifies the data locally. If the block is in the "S" state, the cache must notify any other caches that might contain the block in the "S" state that they must evict the block. This notification may be via bus snooping or a directory, as described above. Then the data may be locally modified. If the block is in the "I" state, the cache must notify any other caches that might contain the block in the "S" or "M" states that they must evict the block. If the block is in another cache in the "M" state, that cache must either write the data to the backing store or supply it to the requesting cache. If at this point the cache does not yet have the block locally, the block is read from the backing store before being modified in the cache. After the data is modified, the cache block is in the "M" state.

7. (de la Dan)

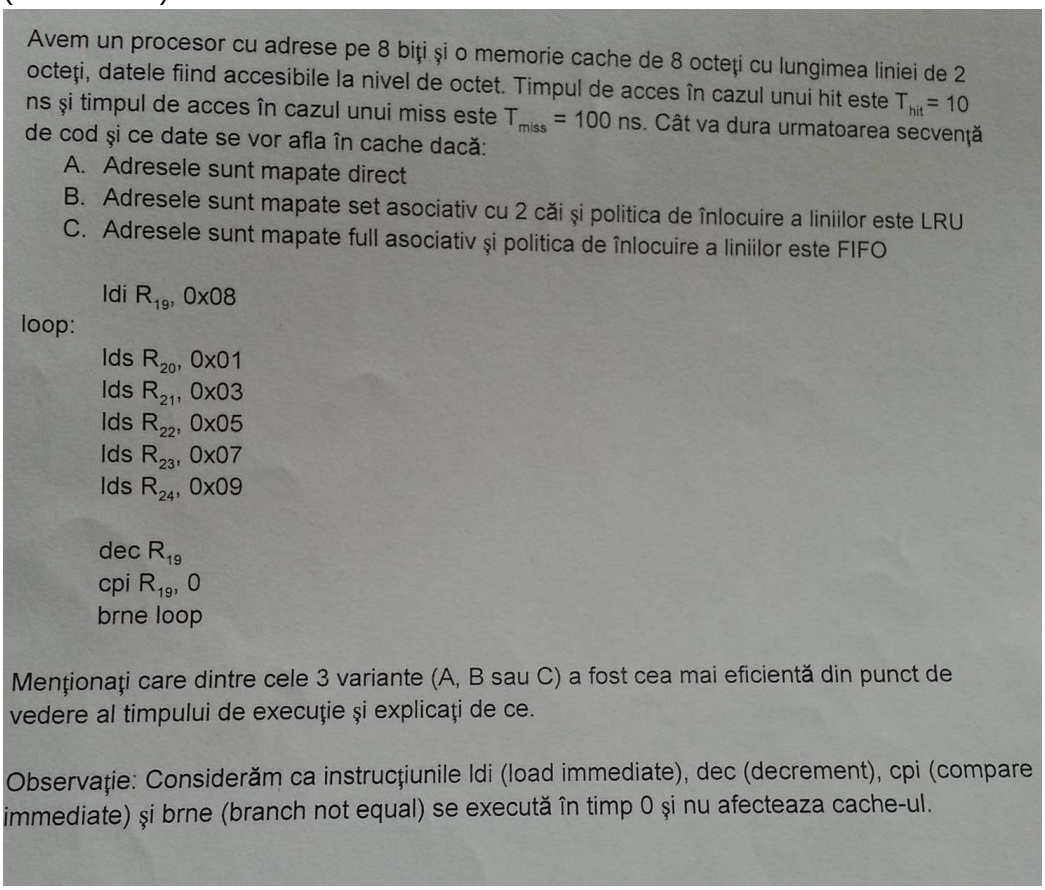
- **Care sunt principalele caracteristici ale procesorului pe care l-ati folost la lab? (10.02.2015)**

- arhitectura RISC

- banda de asamblare in 5 etape
- registri pe 8 biti
- 32 de registri generali (dar se foloseau doar 16, nu?) - noi am folosit doar 16 pentru ca multe instructiuni folosesc doar 16, exista instructiuni care pot sa ii foloseasca si pe ceilalti 16
- adresare pe 8 biti

Partea 2 (1p --- cica, nu stiu daca este asa cu adevarat)

(10.02.2015)



Avem un procesor cu adrese pe 8 biţi şi o memorie cache de 8 octeţi cu lungimea liniei de 2 octeţi, datele fiind accesibile la nivel de octet. Timpul de acces în cazul unui hit este T_{hit} = 10 ns şi timpul de acces în cazul unui miss este T_{miss} = 100 ns. Cât va dura urmatoarea secvenţă de cod şi ce date se vor afla în cache dacă:

A. Adresele sunt mapate direct
B. Adresele sunt mapate set asociativ cu 2 căi şi politica de înlocuire a liniilor este LRU
C. Adresele sunt mapate full asociativ şi politica de înlocuire a liniilor este FIFO

```
        ldi R19, 0x08
loop:
        lds R20, 0x01
        lds R21, 0x03
        lds R22, 0x05
        lds R23, 0x07
        lds R24, 0x09

        dec R19
        cpi R19, 0
        brne loop
```

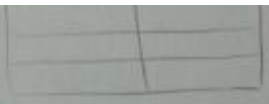
Menţionaţi care dintre cele 3 variante (A, B sau C) a fost cea mai eficientă din punct de vedere al timpului de execuţie şi explicaţi de ce.

Observaţie: Considerăm ca instrucţiunile ldi (load immediate), dec (decrement), cpi (compare immediate) şi brne (branch not equal) se execută în timp 0 şi nu afecteaza cache-ul.

la primul ldi nu este miss, greseala mea :(

$T_{hit} = 10\,ns$

$T_{miss} = 100\,ns$





1) ldi R_{17}, 0x02

loop:

2) lds R_{20}, 0x01
3) lds R_{21}, 0x03
4) lds R_{22}, 0x05
5) lds R_{23}, 0x07
6) lds R_{24}, 0x09

dec R_{17}
cpi R_{17}, 0
brne loop

Cât durează următoarea secvență de cod și ce date se află în cache, dacă:

0x00 0x01
0x08 0x09
0x00 0x01

A. adresare mapată direct

: 2 (una = din totală a lui)

~~0x08~~	~~0x09~~
0x02	0x03
0x04	0x05
0x06	0x07

- miss (compulsory) 1)
- miss (2) miss miss
- miss (3) hit hit
- miss (4) hit hit
- miss (5) hit hit
- miss (6) miss miss

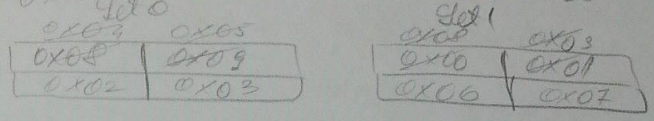
→ Timp = 6 · $T_{miss_compulsory}$ +

+ (10x08 - 1) * (2 x T_{miss} + 8 T_{hit}

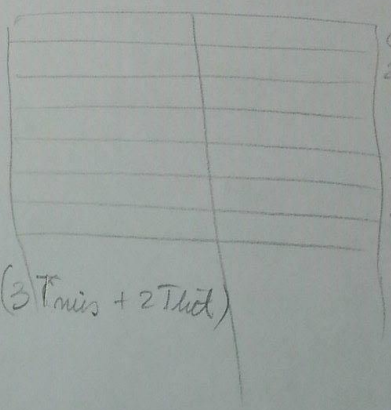
= 600 + (10x08-1) * 230 = ..

b. Adresele sunt mapate set asociativ cu 2 căi și politica de înlocuire = LRU.

:4 acces



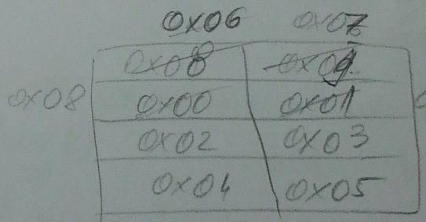
miss (1)
miss (2) miss(2) miss
miss (3) hit hit
miss(4) miss miss
miss(5) hit hit
miss(6) miss miss



$T_{total} = 6 * T_{miss} + (0x02-1) * (3T_{miss} + 2T_{hit})$

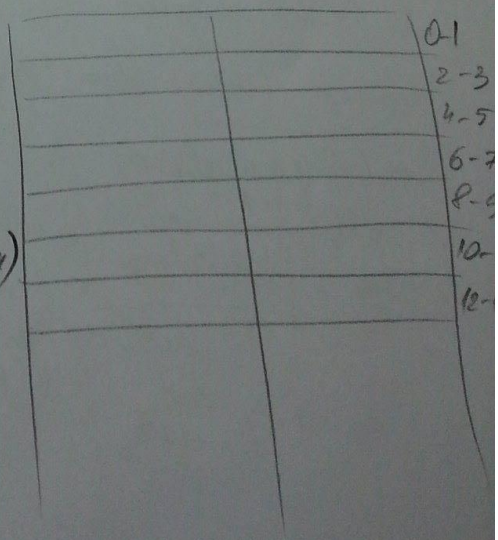
$= 600 + (0x02-1) * 320$

c) Adresele sunt mapate full asociativ și politica de înlocuire a liniilor = FIFO



(1) miss

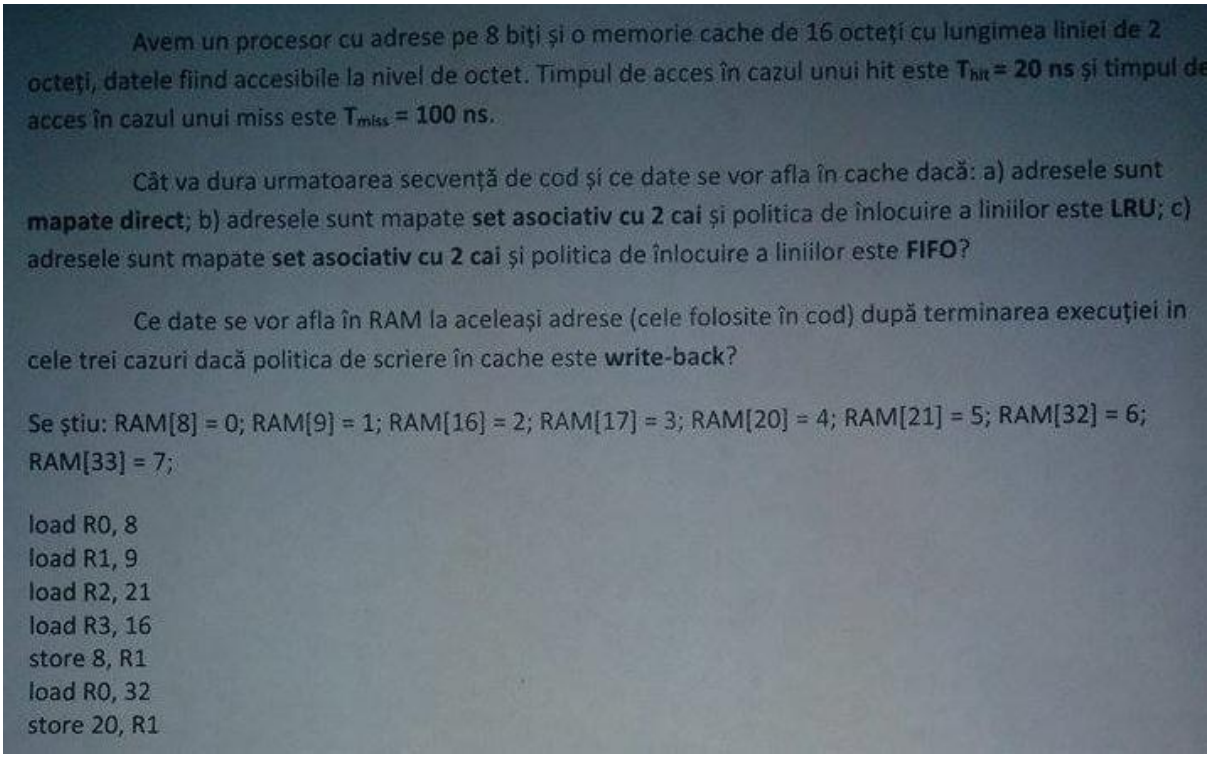
2 miss miss
3 miss miss
4 miss miss
5 miss miss
6 miss miss



$6 \cdot T_{miss} + (0x08-1) T_{miss}$

$= T_{miss} * (5 + 0x08) = 600 * (5 + 0x08)$

(08.09.2014)



Avem un procesor cu adrese pe 8 biți și o memorie cache de 16 octeți cu lungimea liniei de 2 octeți, datele fiind accesibile la nivel de octet. Timpul de acces în cazul unui hit este T_{hit} = **20 ns** și timpul de acces în cazul unui miss este T_{miss} = **100 ns**.

Cât va dura urmatoarea secvență de cod și ce date se vor afla în cache dacă: a) adresele sunt **mapate direct**; b) adresele sunt mapate **set asociativ cu 2 cai** și politica de înlocuire a liniilor este **LRU**; c) adresele sunt mapate **set asociativ cu 2 cai** și politica de înlocuire a liniilor este **FIFO**?

Ce date se vor afla în RAM la aceleași adrese (cele folosite în cod) după terminarea execuției în cele trei cazuri dacă politica de scriere în cache este **write-back**?

Se știu: RAM[8] = 0; RAM[9] = 1; RAM[16] = 2; RAM[17] = 3; RAM[20] = 4; RAM[21] = 5; RAM[32] = 6; RAM[33] = 7;

```
load R0, 8
load R1, 9
load R2, 21
load R3, 16
store 8, R1
load R0, 32
store 20, R1
```

(03.02.2014)

ATENTIE!!!

8-9 se mapeaza la 4

4 e tot miss, la 5 ar fi trebuit sa fie hit

RAM-ul nu cred ca e bine actualizat, pentru ca la write-back se actualizeaza doar linia invalidata, nu tot ce are flag “dirty” pana atunci.

Adrese mapate...
Bacuire LRU

1. Adrese mapate direct

Cache

	32	33
0	16	17
1		
2	20	21
3	1	9
4		
5		
6		
7		

RAM

0-1		
2-3		
8-9	8 1	1
16-17	2	3
20-21	4	5
32-33	6	7

1: miss
2: hit
3: miss
4: hit
5: miss
6: miss
7: hit

T = 3 hit + 4 miss

Write-back => RAM actualizat doar la invalidare linie => doar aici se actualizează

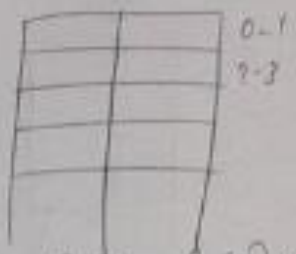
B. Adresare mapata 2-way set asociativ

Cache

16	17
20	21
8	9

32	33

RAM



1: miss
2: hit
3: miss
4: miss
5: hit
6: miss
7: hit

T = 3 hit + 4 miss

RAM - ul raspunde la fel, i.e. cu ~~hit~~ miss cand nu a fost invalidat

C. Adresare mapata full-asociativ

Cache

8	9
20	21
16	17
32	33

RAM



1: miss
2: hit
3: miss
4: miss
5: hit
6: miss
7: hit

T = 3 hit + 4 miss

RAM la fel.

1. Un procesor are doua niveluri de cache (L1 si L2) si o memorie RAM. ~~Sistemul are un CPI=1 daca~~ Programul care ruleaza pe aceasta arhitectura are 10000 de operații load și store care au 10% hit in L1, 30% hit in L2 iar restul rezida in RAM. Timpii de acces pentru diferitele memorii sunt: t_{L1} = 2cicli, t_{L2}=5 cicli si t_{RAM}=100 cicli. Care este valoarea timpului mediu de acces la memorie? (1p)

2. Un bus sincron are o perioadă de ceas de 10ns și fiecare transmisie durează un ciclu de ceas. Alt bus asincron are nevoie de 40ns pentru fiecare handshake. Fiecare bus are 32 de biți de date. Care este lățimea de bandă a fiecărui bus pentru citiri de un cuvânt dintr-o memorie cu 500ns per read? (1p)

3. Avem un procesor cu adrese pe 8 biți și o memorie cache de 16 octeți cu lungimea liniei de 2 octeți, datele fiind accesibile la nivel de octet. Timpul de acces în cazul unui hit este $\mathbf{T_{hit}}$ **= 20 ns** și timpul de acces în cazul unui miss este $\mathbf{T_{miss}}$ **= 100 ns**. Cât va dura urmatoarea secvență de cod și ce date se vor afla în cache dacă:

a) adresele sunt **mapate direct**;(1p)

b) adresele sunt mapate **set asociativ cu 2 cai** și politica de înlocuire a liniilor este **LRU**; (2p)

c) adresele sunt mapate **set asociativ cu 2 cai** și politica de înlocuire a liniilor este **FIFO**? (2p)

d) Ce date se vor afla în RAM la aceleași adrese (cele folosite în cod) după terminarea execuției in cele trei cazuri dacă politica de scriere în cache este **write-back**? (4p)

Se știu: RAM[8] = 0; RAM[9] = 1; RAM[16] = 2; RAM[17] = 3; RAM[20] = 4; RAM[21] = 5; RAM[32] = 6; RAM[33] = 7;

```
load R0, 8
load R1, 9
load R2, 21
load R3, 16
store 8, R1
load R0, 32
store 20, R1
```

1. E o combinatie intre AMAT si niveluri multiple de cache, dar nu stiu sigur cum se face

(am urmat modelul din curs)

2. Pentru bus sincron:
   - trimite adresa la memorie: 10ns
   - citeste memoria: 500ns
   - trimite date la dispozitiv: 10ns

   => in total = 500 + 10 + 10 = 520ns

   => BW = 4 octeti la 520ns => 4 / 520 = 7.69 MB/s

   Pentru bus asincron:
   - memory read: 40ns
   - data ready & hadshake: 3 * 40 + 500 = 620 ns
   - read & ack: 3 * 40 = 120 ns

   => in total = 780 ns

   => BW = 4 octeti la 780ns => 4 / 780 = 5.12 MB/s

3. e acelasi cu cel de mai sus

(28.01.2014)
Este partea de problema care consta din 3 exercitii revin cu ele cand fac rost
In principiu erau probleme cu speedup, performanta si numarat miss-uri intr-un cache.

Rezolvare of Problemă examen CN2 09.02.2015

Avem un procesor cu adrese pe 8 biți și o memorie cache de 16 octeți cu lungimea liniei de 2 octeți, datele fiind accesibile la nivel de octet. Timpul de acces în cazul unui hit este T_{hit} = 50 ns și timpul de acces în cazul unui miss este T_{miss} = 250 ns. Cât va dura urmatoarea secvență de cod și ce date se vor afla în cache dacă:

A. Adresele sunt mapate direct
B. Adresele sunt mapate set asociativ cu 2 căi și politica de înlocuire a liniilor este LRU
C. Adresele sunt mapate full asociativ și politica de înlocuire a liniilor este FIFO

Ce date se vor afla în RAM la aceleași adrese (cele folosite în cod) după terminarea execuției in cele trei cazuri dacă politica de scriere în cache este write-through?

Se știu:
RAM[0x04] = 0;
RAM[0x05] = 1;
RAM[0x0A] = 2;
RAM[0x0B] = 3;
RAM[0x1A] = 4;
RAM[0x1B] = 5;
RAM[0x40] = 6;
RAM[0x41] = 7;

1) lds R_{19}, 0x0A
2) lds R_{20}, 0x0B
3) sts 0x40, R_{20}
4) lds $R_{20,}$ 0x1A
5) lds R_{21}, 0x05
6) sts 0x05, R_{19}
7) lds R_{19}, 0x0A

Observație: Pentru fiecare subpunct scrieți ce conține memoria RAM la adresele folosite în cod, ce se află în fiecare linie de cache la finalul execuției codului și timpul total de execuție. Puteți scrie timpul necesar fiecărei instrucțiuni (se punctează parțial).

A.

lds R_{19}, 0x0A ; miss
lds R_{20}, 0x0B ; hit
sts 0x40, R_{20} ; miss
lds $R_{20,}$ 0x1A ; miss
lds R_{21}, 0x05 ; miss
sts 0x05, R_{19} ; hit
lds R_{19}, 0x0A ; miss

Cache:

0: 0x40,	0x41
1: x,	x
2: 0x04,	0x05
3: x,	x
4: x,	x
5: ~~0x0A~~ ~~0x1A~~ 0x0A,	~~0x0B~~ ~~0x1B~~ 0x0B———
6: x,	x
7: x,	x

RAM:
RAM[0x40] = 3;
RAM[0x05] = 2;

$T = 5*T_{miss} + 2*T_{hit} = 1350$

B. b

lds R_{19}, 0x0A ; miss
lds R_{20}, 0x0B ; hit
sts 0x40, R_{20} ; miss
lds $R_{20,}$ 0x1A ; miss
lds R_{21}, 0x05 ; miss
sts 0x05, R_{19} ; hit
lds R_{19}, 0x0A ; hit

Cache:
0.0: 0x40, 0x41
0.1: x, x
1.0: 0x0A, 0x0B
1.1: 0x1A, 0x1B
2.0: 0x04, 0x05
2.1: x, x———
3.0: x, x
3.1: x, x

RAM:
RAM[0x40] = 3;
RAM[0x05] = 2;

$T = 4*T_{miss} + 3*T_{hit} = 1150$

C. c

lds R_{19}, 0x0A ; miss
lds R_{20}, 0x0B ; hit
sts 0x40, R_{20} ; miss
lds $R_{20,}$ 0x1A ; miss
lds R_{21}, 0x05 ; miss
sts 0x05, R_{19} ; hit
lds R_{19}, 0x0A ; hit

Cache:
0: 0x0A, 0x0B
0: 0x40, 0x41
0: 0x1A, 0x1B
0: 0x04, 0x05
0: x, x
0: x, x———
0: x, x
0: x, x

RAM:
RAM[0x40] = 3;
RAM[0x05] = 2;

$T = 4*T_{miss} + 3*T_{hit} = 1150$

:p

Partea 1 (4p --- cica, nu stiu daca este asa cu adevarat)

1. **(cu cache, teorie si probleme)**
   - **Explicati cum sunt afectate cele 3 tipuri de miss (obligatoriu, de conflict si de capacitate) daca adaugam capabilitate de prefetching unui cache. (08.09.2014) (03.02.2014)**

--- **Miss obligatoriu scade, pentru ca sunt aduse dinainte datele.**

--- **Miss de capacitate scade**, **(in special daca face prefetch la ce trebuie si cand trebuie):** ideea e ca eu ma gandesc ca miss-ul de capacitate are loc atunci cand are loc miss indiferent de politica de inlocuire - daca eu am o politica de inlocuire perfecta si trebuie sa aduc 100MB intr-un cache de 10MB, la un moment dat tot o sa am miss, dar daca sa zicem ca nu folosesc toti 100MB unii dupa altii, e posibil sa am timp sa fac prefetch la ceva ce urmeaza si sa scap de miss-ul de capacitate.
-- Miss de capacitate -> scade. Idem Tibi. Daca avem nevoie de mai multa informatie decat poate cache-ul sa tina, tehnica de prefetch poate sa aduca informatia pe care nu am reusit sa o incarcam in cache atunci cand este nevoie.
--- **Miss de capacitate:** eu aici cred ca nu se modifica deoarece cache-ul are aceeasi capacitate si daca eu aduc niste date de care am nevoie mai devreme, inlocuind in acelasi timp alte date utile, asta nu inseamna ca per total missurile de capacitate vor scadea, ci se vor petrece mai devreme decat s-ar fi petrecut in mod obisnuit. (mie mi se pare normal sa ramana la fel, desi inteleg ce zici, Tibi, dar capacitatea cache-ului ramane aceeasi si eu cand fac prefetch doar inlocuiesc datele din cache cu alte date, cum a zis Miruna, deci mi se pare ca folosesc cache-ul cum se foloseste in mod normal si ca nu se aduce nicio imbunatatire)

--- **Miss de conflict ar putea scadea in cazul in care valorile care sunt mapate in aceeasi locatie din cache nu sunt folosite atat de rapid una dupa alta, incat sa nu aiba timp prefetch-ul sa le inlocuiasca atunci cand e nevoie de ele. In schimb, daca ele sunt folosite una dupa alta, prefetch-ul nu imbunatateste cu nimic viteza, deoarece nu se poate face prefecth intr-o locatie care e folosita in mod curent.**

   - **Daca ai linii de 2 ori mai lungi in cache spune ce se intampla cu miss obligatoriu, miss de conflict si miss de capacitate (28.01.2014)**

Ok, deci aici dimensiunea ramane constanta iar ele sunt de doua ori mai lungi.
---Deci in cazul asta scad miss urile obligatorii pentru ca aducem mai multe date,
---scad miss urile de capacitate (creste dimensiune cache). ??? nu zisesem ca dimensiunea cache ramane aceeasi? (acum am vazut ca in curs spune ca scad la reload (ce o mai fi insemnand si aia), si ca a ramas in dubiu si cu Dan Tudose de ce ele scad) aici ma gandesc ca e posibil sa fie acelasi motiv pentru care scad miss-urile de obligatorii si anume ca se aduc mai multe date => scade nevoia de a tot inlocui date in cache cu unele din RAM => mai putine sanse de a da miss.
---Aici in curs scrie ca CRESC miss urile de conflict (scuze :">).
Eu zic ca miss-ul de capacitate nu scade.(daca nu se umbla la dimensiunea chache-ului, nu cred ca se afecteaza miss-urile de capacitate) Ar putea sa creasca in cazul in care trebuie

accesate adresele de forma 10k unde liniile au dimensiune 9. ca la un moment dat se umple cache-ul de linii si trebuie invalidate, dar daca liniile erau doar de un octet se umplea doar cu cate un octet, nu cu cate 9 si aveau loc toate. (totusi, cele de capacitate parca erau influentate strict de dimensiunea cache-ului. Daca eu cer adrese care imi dezorganizeaza cache-ul, nu cred ca e de capacitate)

- **Cum sunt afectati parametrii de hit time, miss rate, miss penalty daca adaugam capabilitate de prefetching unui cache? (10.02.2015)**

--- hit time ramane la fel
--- scad miss rate (1 - hit rate)
--- miss penalty ramane constant

- **Cum sunt afectati parametrii de hit time, miss rate, miss penalty daca dublam asociativitatea unui cash set-asociativ (dar pastram capacitatea si dimensiunea liniilor constante)? (08.09.2014) (03.02.2014)**

Tinda sa reduca miss uri conflictuale deci scad miss rate, creste hit time , creste miss penalty (explicatie Tibi)

ideea e ca un set are acum mai multe linii daca creste asociativitatea
full associative inseamna un set care cuprinde toate liniile, adica toate pot fi mapate oriunde asta daca folosim "set" asa cum e el denumit de fapt, nu cum a zis Dan Tudose la curs, cand se referea la set ca locul in cadrul unui index

--- creste hit time pentru ca ii ia mai mult sa caute
--- scade miss rate pentru ca scad miss-urile de conflict (bine-nteles trebuie o politica buna)
---- miss penalty creste pt ca adaugi cautarea + inlocuirea (mai faci si politica sa vezi pe care o inlocuiesti, sunt mai multe locuri in care poti inlocui) dar politica de inlocuire exista de la inceput si daca este una perfecta, atunci nu ar trebui sa dureze mai mult pana inlocuieste datele, nu?

- **Daca ai linii de 2 ori mai scurte (cache e la jumate din capacitate pt ca se pastreaza asociativitatea si inca ceva, probabil dimensiunea liniilor) ce se intampla cu hit time, miss rate si miss penalty (28.01.2014)**

- hit time: scade --------> pentru ca dureaza mai putin sa aduci datele din cache (eu aici as fi zis ca ramane la fel, ca pana la urma din cache cred ca se ia doar ce date se dorec din linia aia) si eu zic tot ca ramane la fel
- miss rate: creste ----------> mai putine date in cache
- miss penalty: scade ---------> (cred ca aici scade miss penalty deoarece se aduc din RAM calupuri de date mai mici, deci se ocupa mai putina latime de banda pe magistrala, timp de transfer mai scurt, probabil) (si eu cred ca scade, ia mai putin timp sa aduci datele din RAM) si eu zic tot ca scade
- miss de capacitate: creste
- miss obligatoriu: la fel creste ca sunt aduse mai putine instructiuni si deci trebuie facute mai multe accese la memorie obligatorii
- miss de conflict: scade? creste ca sunt mai multe linii in RAM de mapat, deci concureaza mai multe pe acelasi loc in cache

---

Inainte de probleme, poate ar fi bine sa scriem niste formule de referinta (daca sunt gresite, yell and modify :))

Structura : tag index offset

- **dimensiune linie cache = 2 ^ offset**
- **dimensiune cache = 2 ^ index linii**
- **dimensiune cache = 2 ^ offset * 2 ^ index * nr_seturi**
  = 2 ^ (offset + index) * nr_seturi

- **Avem o memorie cache cu cuvinte de un octet, linii de 8 octeti lungime organizata in 32 de seturi de cate 4 linii. Care este dimensiunea utila a memoriei cache, in octeti? Daca adresarea se face pe 32 de biti, care este dimensiunea tag-ului pt. fiecare linie de cache? (10.02.2015)**

(
fiecare linie are 8 octeti | fiecare set are 4 linii => 4*8 octeti = 32 octeti | sunt 32 de set-uri => 32*32 = 1024 octeti dimensiune totala cache

linii de 8 octeti => adresare pe 3 biti. (offset)
32 de seturi => adresare pe 5 biti (index)
=> tag-ul e de 24 de biti
)

- **Care sunt componentele unei adrese pe 16 biti a unui cache mapat direct, cu datele accesibile la nivel de octet. Linia de cache are o latime de W = 4 octeti. Dimensiunea cache L = 1024 linii (4kB)? (08.09.2014)** (28.01.2014)

Structura : tag index offset
W = 4 octeti = 4 * 8 biti = 32 biti
latime_linie = 2 ^ offset = 32 => offset = log2(32) = 5 biti pentru offset (aici datele sunt accesibile al nivel de octet => calcul cu cei 4 octeti, nu mai transformi in biti => 2 biti pt offset) da, 2 biti aici pt offset
L = 1024 linii = 2 ^ index => index = log2(1024) = 10 biti index
16 - 10 - 5 = 1 bit pentru tag (si aici cred ca ar veni: 4 biti pt tag ramasi) si eu cred ca rezultatul e 4b pt tag (model: slide 30, curs 2)

Raspuns: (2 biti offset, 10 biti index, 4 biti tag)

- **Avem un sistem de calcul cu un singur nivel de cache (L1). Daca adaugam un al doilea nivel de cache (L2), cum se vor modifica urmatorii parametri de functionare: L1 hit time, L1 miss rate, L1 miss penalty? (03.02.2014)**

Scade miss penalty (mai mic pentru a aduce date din L2),
miss si hit raman la fel (nu pot aduce mai multe date sau modifica dimensiune deci…)

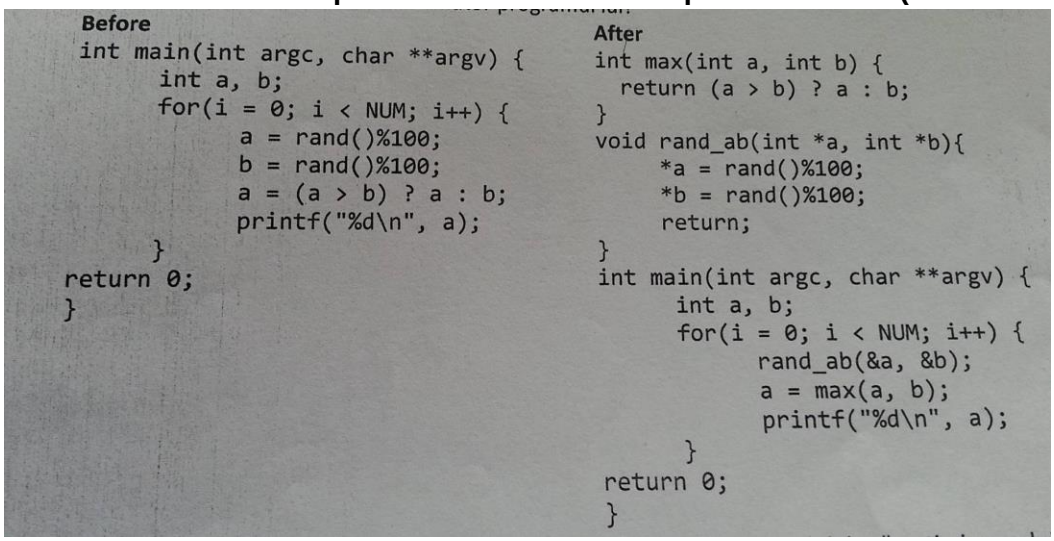
In curs scrie asa: (curs 3, slide 33):

- **hit time mult mai mic (pentru ca prezenta lui L2 duce la un L1 mai mic)** **buna observatie, nu m-as fi gandit - dar trebuie specificat ca schimbi L1 daca zici asta**
- **miss rate mai mare (pentru ca L1 e mai mic)**
- **miss penalty mai mic pt L1 (nu scrie de ce, dar probabil pentru ca suporta L2 miss penalty-ul)**

- **Care din urmatoarele sunt adevarate? (10.02.2015)**
  - **un cache set-asociativ cu o linie pe set este un cache complet asociativ**
  - **un cache set-asociativ cu un singur set este un cache complet asociativ** **da da da oui** да
  - **un cache set-asociativ cu o linie pe set este un cache mapat direct** **da da da oui** да
  - **un cache set-asociativ cu un singur set este un cache mapat direct**

2. (cu cod imbunatatit sau nu dupa modificare)

- **Se modifica cod pentru a functiona mai repede. Este ok? (10.02.2015)**



Before

```
int main(int argc, char **argv) {
      int a, b;
      for(i = 0; i < NUM; i++) {
            a = rand()%100;
            b = rand()%100;
            a = (a > b) ? a : b;
            printf("%d\n", a);
      }
return 0;
}
```

After

```
int max(int a, int b) {
  return (a > b) ? a : b;
}
void rand_ab(int *a, int *b){
     *a = rand()%100;
     *b = rand()%100;
     return;
}
int main(int argc, char **argv) {
      int a, b;
      for(i = 0; i < NUM; i++) {
            rand_ab(&a, &b);
            a = max(a, b);
            printf("%d\n", a);
      }
return 0;
}
```

Nu (scade localitatea spatiala) (nu este vorba de localitatea temporala?: gen a si b din primul exemplu sunt tot folosite, pe cand in exemplul 2 se aloca alti pointeri de a si de b, cred), si pe de alta parte mai face si apeluri de stiva multe pt chemare functiilor alora => ineficient (Cred ca accentul trebuie sa fie pus pe apelul de stiva, nu cred ca se refera neaparat la localizare temporala/spatiala) asa e, nu avem vectori, deci scade localitatea temporala

- **Se modifica cod pentru a functiona mai repede. Este ok? (10.02.2015)**

Before

```
void f(int** a, int** b) {
for(j = 0; j < ROWMAX; j += 1)
 for(i = 0; i < COLMAX; i += 1)
  a[i][j] = b[i][j] * 3330;
}
```

After

```
void f(int** a, int** b) {
for(j = 0; j < COLMAX; j += 1)
 for(i = ROWMAX-1; i >= 0; i -= 1)
  a[j][i] = b[j][i] * 3330;
}
```

Nu(prima foloseste localitate spatiala). In memoria matricele sunt reprezinta ca o zona contigua de memorie. de fapt e cum zice Andrei mai jos, a doua are localitatea sptiala mai buna

(eu pe asta nu o inteleg … adica fooseste j drept contor de linii, dar in a il scrie asa: a[i][j] … adica drept index de coloane …. chestia asta da segfault daca i si j sunt diferite … gresesc?)

Nu observasem asta, dar desi trece de bariera logica a notatiilor, sunt totusi niste variabile, deci poate sa le ia si sa le numeasca cum vrea el.

Le numeste asa ca sa te induca in eroare. Pe primul parcurg matricea coloana cu coloana (ceea ce e naspa). In al doilea, parcurg rand cu rand, dar randurile le parcurg invers. Eu cred ca se imbunatateste localitatea spatiala, deci aduce o imbunatatire.

- **Se modifica cod pentru a functiona mai repede. Este ok? (08.09.2014)**

Before

```
for (i=0; i<M; i++) {
 for (j=0; j<N; j++) {
      x[i*N+j] = 2 * x[i*N+j];
      }
}
```

After

```
for (j=0; j<N; j++) {
 for (i=0; i<M; i++) {
      x[i*N+j] = 2 * x[i*N+j];
      }
}
```

Identic cu cel anterior? cred ca da

eu cred ca aici al doilea este mai bun la localitate temporala … adica in bucla din interior dubleaza de M ori aceeasi valoare din vectorul X, pe cand in primul face dublarea la intervale de N operatii;

(eu cred ca nu se aduce o imbunatarie, la primul, parcurgerea se face pe randuri, la al doilea pe coloane => localizare spatiala mai proasta la al doilea => NU este ok)

(pai, Andrei, nu parcurgi o matrice, ci un vector …. adica nu ai coloane si linii; M-ul ala este acolo sa indice de cate ori inmultesti cu 2, nu?)

Primul are localitate spatiala mai buna pentru ca in vector creste cu cate o unitate la fiecare iteratie, nu cu cate N

Subscriu ^^^^

- **Se modifica cod pentru a functiona mai repede. Este ok? (08.09.2014)**

Before

```
for (i=0; i<N; i++)
 a[i] = b[i] * c[i];
for (i=0; i<N; i++)
 d[i] = a[i] * c[i];
```

After

```
for (i=0; i<N; i++) {
 a[i] = b[i] * c[i];
 d[i] = a[i] * c[i];
}
```

A doua varianta mai buna. In primul caz va invalida “a” si eventual actualiza in RAM (rau). In al doilea caz il foloseste din cache. de acord se fol astfel localitatea temporala a[i] si c[i] folosite de mai multe ori да

- **Se modifica cod pentru a functiona mai repede. Este ok? (28.01.2014, dar alt cod) (03.02.2014)**

Before

```
for (i=0; i<N; i++)
 for (j=0; j<N; j++)
  B[j*N+i] = A[i*N+j];
```

After

```
int BS=8;
for (i=0; i<N; i+=BS)
 for (j=0; j<N; j++)
  for (k=0; k<BS; k++)
   B[j*N+(i+k)] = A[(i+k)*N+j];
```

Prima varianta foloseste localitate spatiala? Se pare ca, luand in considerare documentul pus de Tibi http://csapp.cs.cmu.edu/2e/waside/waside-blocking.pdf, a doua varianta prezinta localitate temporala, so functioneaza mai repede. Si in general, orice e organizat pe blocuri prezinta o mai buna localitate temporala.
(primul e clar ca nu merge bine pentru localitatea spatiala -> al doilea e pe blocuri -> al doilea e mai bun) algoritmul de aici nu e identic cu cel pe blocuri, dar e un pas spre ele deci are localitatea temporala mai buna

- **Se modifica cod pentru a functiona mai repede. Este ok? (28.01.2014, dar alt cod) (03.02.2014)**

Before

```
for (i=0; i<N; i++)
 c[i] = a[i] + b[i];
```

After

```
int P=1;
for (i=0; i<N; i++) {
        prefetch(&a[i+P]);
        prefetch(&b[i+P]);
        prefetch(&c[i+P]);
        c[i] = a[i] + b[i];
}
```

Mai buna prima varianta. Este prea tarziu pentru prefech si va polua cache ul. Spunea Dan ca era ok daca ar fi fost facut prefetch cu ~100 instructiuni inainte. nu imi amintesc de asta cu 100 instructiuni, dar si eu mi-am notat ca nu e buna a doua varianta.
Am notat eu asta pe curs :? -> oricum mai buna prima da, clar
(prefetch-ul e costisitor, nu? Si daca eu fac prefetch la un element, e clar ca nu se va aduce doar acel element in cache, se va aduce intreaga linie, deci urmatoarele prefetch-uri sunt degeaba. Dar daca doar maresc P-ul, de ce ar fi mai bine?)

3. **(memorie virtuala, TLB: teorie sau probleme)**

- **Daca ai pagini de 2 ori mai mari, ce se intampla cu rata de miss si cu TLB reach (28.01.2014)**

Miss rate banuiesc ca scade, avand in vedere ca avem mai multa informatie la dispozitie => sanse mai mici sa dam miss

TLB reach este direct proportional cu dimensiunea paginii, deci si el creste:

TLB reach = numar_intrari_TLB * dimensiune_pagina * numar_pagini_pe_intrare? sincer nu stiu, in curs era doar prima parte

Am si eu o intreabare aici. TLB Reach spune ca este dimensiunea celui mai mare spatiu care poate fi mapat SIMULTAN. Deci daca am fi crescut doar numarul de intrari am fi fost siguri ca este contiguu si ar fi crescut TLB Reach?

- **Avem un sistem cu memorie virtuala si TLB. Daca dublam numarul de intrari in TLB, cum vor fi afectate rata de miss a TLB si capacitatea acestuia de adresare (TLB reach)? (08.09.2014) (03.02.2014)**

  Rata de miss scade.

  La fel ca mai sus (dimensiune pagina = numar de intrari)

Ah , este chiat aici intreabarea. Daca nu este contiguu nu cred ca putem spune sigur de TLB Reach. Gresesc? !!!! Buna observatie, Lavinia, nici eu nu stiu cum este… :(asa e, in curs scria ca pt memorie contigua, deci nu stiu ce se intampla daca nu e asa

- **Descrieti rolul pe care il are TLB intr-o arhitectura de calcul moderrna. (10.02.2015) (03.02.2014)**

TLB (Translation Lookaside Buffer) este un cache mic care contine cele mai recente translatii adresa virtuala - adresa fizica. Are rolul de a reduce numarul de accese la memorie pentru majoritatea operatiilor load/store (de la 2 cicli la 1).

- **Ati scris un program care are nevoie frecvent de cativa kB de memorie temporara. Ce este mai rapid: sa fol in mod repetat acelasi spatiu de memorie sau sa alocam un spatiu diferit de fiecare data? De ce? (10.02.2015)**

Daca datele de intrare nu difera de la o rulare la alta, atunci e mai rapid sa folosim acelasi spatiu de memorie, pentru ca datele sunt deja in cache si cum stim deja, e mai rapid sa accesam datele din cache decat din RAM sau HDD.

Eu as zice ca mereu e mai bine sa folosim in mod repetat acelasi spatiu pentru ca imbunatateste localitatea temporala.

- **Care dintre urmatoarele sunt motive valide pt. fol. adreselor virtuale? Explicati. (10.02.2015)**
  - **permite unui proces sa acceseze intregul spatiu de memorie** ---- Da, in sensul ca ii lasa impresia procesului ca are acces la toata memoria disponibila Nu, tocmai ca e limitat in spatiul pe care i-l pune la dispozitie sistemul de operare ??? (si eu as zice ca Alina … ca macar ii da impresia ca lucreaza cu toata memoria) e cum spui, Tibi, el e spatiu limitat, dar procesul crede ca lucreaza cu toata memoria
  - mareste rata de transfer a dispozitivelor I/O tind sa spun “nu”, de ce ar fi asa? (I/O-ul nu foloseste VM, nu?) Foloseste….in sensul in care umbla cu RAM-ul si HDD-ul prin DMA, dar cred ca adresele virtuale ii ingreuneaza treba … pt ca el in esenta cred ca lucreaza cu adrese fizice Nu mareste rata de transfer, poate chiar aduce un overhead prin operatii de translatare de adrese

- **este mai dificil ca un program ce functioneaza prost sa afecteze alte programe concurente** -------- Da, pentru ca fiecare proces are propriul spatiu de memorie virtuala
- **simplifica implementarea functiilor de memorie (malloc, new)** (unul ditre principalele scopuri) eu cred ca asta este “da”, pt ca scrie prin curs ceva de genul ca inainte de adrese virtuale, cei care scriau programe trebuiau sa fie atenti ei la alocari de memorie sa nu intre peste alte programe imi amintesc si eu ca scria asta da, simplifica pentru ca vede tot spatiul programului ca un spatiu contiguu de adrese si ii e mai usor sa il gestioneze decat daca ar avea la dispozitie toata memoria
- **permite upgrade-ul gratuit de al Win7 la Win10 ------- clar da, nici nu exista dubiu, daca accentul e pus pe gratuit clar nu (daca nu era memorie virtuala, managementul memoriei era foarte dificil si dependend de arhitectura. Acuma, daca accentul e pus pe “gratuit” - e o prostie, asta tine de Windows, deci nu. Dar, daca accentul e pus pe “permite”, poate si memoria virtuala sa contribuie la chestia asta si as zice “da” :))** si eu as zice tot ca da, desi nu prea are sens :D daca accentul pica pe gratuit, atunci sigur e corect, ca fiind adrese virtuale costa mai putin, adica e gratuit.

4. (cu magistrale sau ce mai este pe la I/O)

- **La care din urmatoarele magistrale este legat CPU-ul? (10.02.2015)**

a) CPU bus; b) Memory bus; c) School bus d) I/O bus e) PCI bus

Alina: a, b asta este dubioasa pt ca in curs scrie magistrala procesor-memorie … nu este scris niciodata separat … cred ca merita intrebata daca ne pica la examen exact, stiu, de-asta le-am ales pe amandoua :(

- **Dati 3 exemple la alegere de magistrale dintr-un sistem de calcul si descrieti pe scurt la ce sunt folosite. (08.09.2014) (28.01.2014)**

1. Magistrala Procesor-Memorie:
   a. conectata direct la CPU;
   b. conecteaza sistemul de memorie;
   c. maximizeaza bandwidth-ul Memorie - CPU
   d. optimizata pentru transferuri de blocuri cache
   e. scurta si de viteza mare (detaliu)

2. Magistrala I/O:
   a. trebuie sa acomodeze o gama larga de dispozitive I/O
   b. lunga si lenta (detaliu)
   c. se conecteaza la magistrala procesor-memorie sau la backplane (detaliu)

3. Backplane bus:
   a. permite procesoarelor, dispozitivelor de I/O si memoriei sa coexiste
   b. o structura de interconectare din interiorul sasiului (detaliu)

4. posibil sa mearga povestit si despre PCI

- **In sistemele de calcul moderne se folosesc din ce in ce mai mult magistrale de comunicatie seriala (SATA, USB, etc.). De ce sunt acestea preferate magistralelor paralele (dati cel putin 3 motive)? (03.02.2014)**

- functioneaza la viteze multi-gigabit
- low power => avem o singura legatura de date per periferic
- transferuri simultane multiple
- conectori si cabluri ieftine
- fiecare dispozitiv are legatura lui de date

- **Descrieti, pe scurt, modul de functionare al unui controller DMA (asta este de la I/O). (08.09.2014) (28.01.2014)**

DMA = direct memory access
Pasii de functionare sunt:

1. un dispozitiv cere transferul a S octeti de la disk controller; specifica adresa A si dimensiunea S controllerului DMA;
2. disk controller incepe transferul DMA a S octeti;
3. disk controller initiaza bus request pentru fiecare octet de pe disk;
4. controller DMA transfera cate un octet la adresa A, incrementeaza A si decrementeaza S;
5. cand S == 0, controller DMA intrerupe CPU pt a indica ca transferul e complet.

5. (de prin cursul ILP si Superscalar)

- **Se da urmatoarea secventa de cod: (28.01.2014)**
  add.d f0, f1, f8
  add.d f2, f3, f8
  add.d f4, f5, f8
  add.d f6, f7, f8

  Care din urmatoarele tehnici va aduce o imbunatatire pentru codul de mai sus:
  -branch prediction
  -out of order execution cu register renaming
  -superscalar ---> ce este asta? am gasit pe wikipedia ca e un CPU care implementeaza o forma de paralelism pe un singur procesor, executand mai multe instructiuni intr-un singur clock cycle pe diferite unitati de executie de pe CPU-ul respectiv.

  Cred ca raspunsul este superscalar, pentru ca:
  - branch prediction nu are cum sa fie, ca nu avem branchuri
  - out of order execution cu register renaming nu ajuta la nimic, ca nu avem dependente de date si rearanjarea nu ar imbunatati nimic

Deci a ramas superscalar..si ar merge sa fie executate instructiunile in paralel din moment ce nu avem dependente de date.

- **Care dintre cele 3 abordari furnizeaza cea mai mare imbunatatire a performantei la rularea urmatorului cod: (08.09.2014) 03.02.2014)**
  LD R1 0(R2) #cache miss
  ADD R2 R1 R1
  LD R1 0(R3) #cache hit
  LD R3 0(R4) #cache hit
  ADD R3 R1 R3
  ADD R1 R2 R3

  Variante:
  - **out-of-order execution cu Register Renaming**
  - **branch prediction**
  - **superscalar**

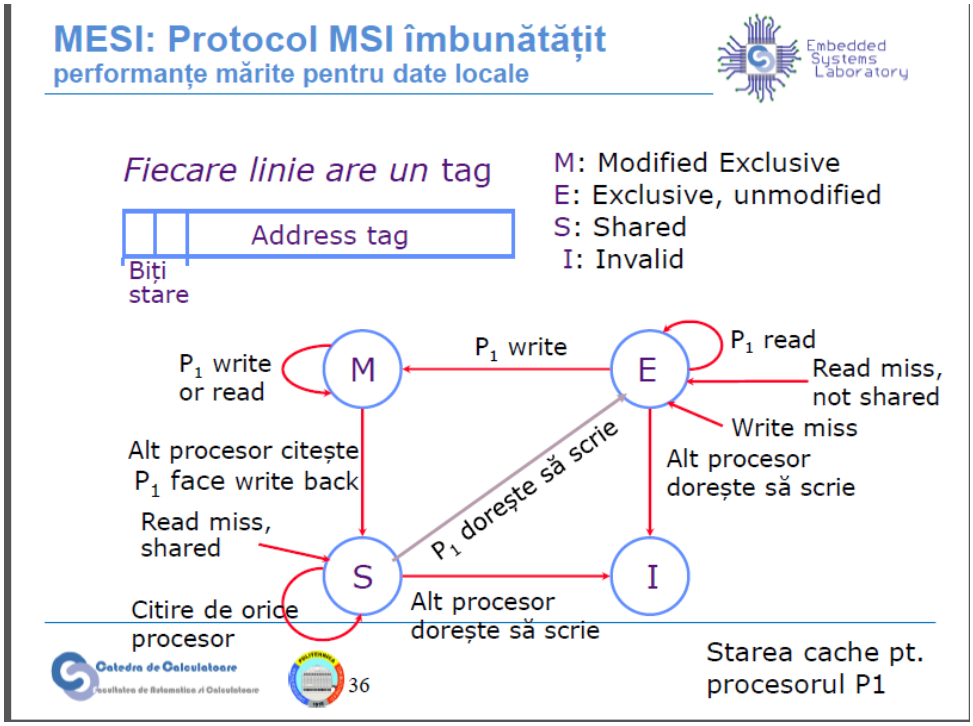
Banuiesc ca out-of-order execution, avand in vedere ca sunt dependente de date pe acolo. da, mai exact pentru ca avem WAW unde chiar e folosit register renaming.

6. (de la multiprocesor)

- **Definiti consistenta secventiala pentru un sistem multi-procesor. (08.09.2014) (28.01.2014)**

Un sistem este consistent secvential daca rezultatul oricarei executii e acelasi ca si cand operatiile tuturor procesoarelor ar fi executate secvential si operatiile fiecarui procesor individual apar in ordinea specificata in program. (Leslie Lamport)

- **Descrieti protocolul MESI. (03.02.2014) - e cam messy stiu cred ca Danut trebuie sa aiba o zi proasta ca sa o dea pe asta**



In computing, the MSI protocol - a basic cache-coherence protocol - operates in multiprocessor systems. As with other cache coherency protocols, the letters of the protocol name identify the possible states in which a cache line can be. So, for MSI, each block contained inside a cache can have one of three possible states:

- **Modified: The block has been modified in the cache. The data in the cache is then inconsistent with the backing store (e.g. memory). A cache with a block in the "M" state has the responsibility to write the block to the backing store when it is evicted.**
- **Shared: This block is unmodified and exists a read-only state in at least one cache. The cache can evict the data without writing it to the backing store.**
- **Invalid: This block is not present in the current cache, and must be fetched from memory or another cache if the block is to be stored in this cache.**

When a read request arrives at a cache for a block in the "M" or "S" states, the cache supplies the data. If the block is not in the cache (in the "I" state), it must verify that the line is not in the "M" state in any other cache. Different caching architectures handle this differently. For example, bus architectures often perform snooping, where the read request is broadcast to all of the caches. Other architectures include cache directories which have agents (directories) that know which caches last had copies of a particular cache block. If another cache has the block in the "M" state, it must write back the data to the backing store and go to the "S" or "I" states. Once any "M" line is written back, the cache obtains the block from either the backing store, or another cache with the data in the "S" state. The cache can then supply the

data to the requestor. After supplying the data, the cache block is in the "S" state.

When a write request arrives at a cache for a block in the "M" state, the cache modifies the data locally. If the block is in the "S" state, the cache must notify any other caches that might contain the block in the "S" state that they must evict the block. This notification may be via bus snooping or a directory, as described above. Then the data may be locally modified. If the block is in the "I" state, the cache must notify any other caches that might contain the block in the "S" or "M" states that they must evict the block. If the block is in another cache in the "M" state, that cache must either write the data to the backing store or supply it to the requesting cache. If at this point the cache does not yet have the block locally, the block is read from the backing store before being modified in the cache. After the data is modified, the cache block is in the "M" state.

7. (de la Dan)

- **Care sunt principalele caracteristici ale procesorului pe care l-ati folost la lab? (10.02.2015)**

- arhitectura RISC
- banda de asamblare in 5 etape
- registri pe 8 biti
- 32 de registri generali (dar se foloseau doar 16, nu?) - noi am folosit doar 16 pentru ca multe instructiuni folosesc doar 16, exista instructiuni care pot sa ii foloseasca si pe ceilalti 16
- adresare pe 8 biti

Partea 2 (1p --- cica, nu stiu daca este asa cu adevarat)

(10.02.2015)

Avem un procesor cu adrese pe 8 biţi şi o memorie cache de 8 octeţi cu lungimea liniei de 2 octeţi, datele fiind accesibile la nivel de octet. Timpul de acces în cazul unui hit este T_{hit} = 10 ns şi timpul de acces în cazul unui miss este T_{miss} = 100 ns. Cât va dura urmatoarea secvenţă de cod şi ce date se vor afla în cache dacă:

A. Adresele sunt mapate direct
B. Adresele sunt mapate set asociativ cu 2 căi şi politica de înlocuire a liniilor este LRU
C. Adresele sunt mapate full asociativ şi politica de înlocuire a liniilor este FIFO

```
        ldi R19, 0x08
loop:
        lds R20, 0x01
        lds R21, 0x03
        lds R22, 0x05
        lds R23, 0x07
        lds R24, 0x09

        dec R19
        cpi R19, 0
        brne loop
```

Menţionaţi care dintre cele 3 variante (A, B sau C) a fost cea mai eficientă din punct de vedere al timpului de execuţie şi explicaţi de ce.

Observaţie: Considerăm ca instrucţiunile ldi (load immediate), dec (decrement), cpi (compare immediate) şi brne (branch not equal) se execută în timp 0 şi nu afecteaza cache-ul.

la primul ldi nu este miss, greseala mea :(

$T_{hit} = 10\,ns$

$T_{miss} = 100\,ns$

1) ldi R_{17}, 0x02

loop:

2) lds R_{20}, 0x01
3) lds R_{21}, 0x03
4) lds R_{22}, 0x05
5) lds R_{23}, 0x07
6) lds R_{24}, 0x09

dec R_{17}
cpi R_{17}, 0
brne loop



Cât durează următoarea secvență de cod și ce date se află în cache, dacă:

A. adresă mapată direct

: 2 (adresa = adresa totală a lui)

0x08	0x09
0x02	0x03
0x04	0x05
0x06	0x07

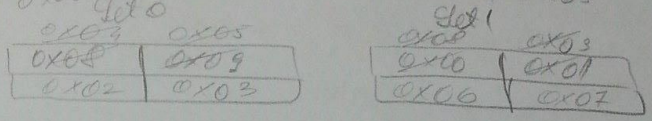
- miss (compulsory) 1)
- miss (2) miss miss
- miss (3) hit hit
- miss (4) hit hit
- miss (5) hit hit
- miss (6) miss miss

$\Rightarrow$ Timp = 6 · $T_{miss\text{-}compulsory}$ +

+ (0x08 − 1) * (3 × T_{miss} + 3 T_{hit}

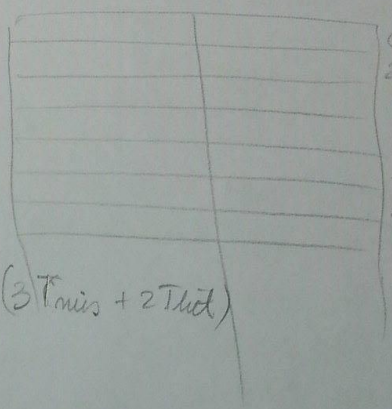
= 600 + (0x08 − 1) × 230 = ..

b. Adresele sunt mapate set asociativ cu 2 căi și politica de înlocuire = LRU.



4 accesări

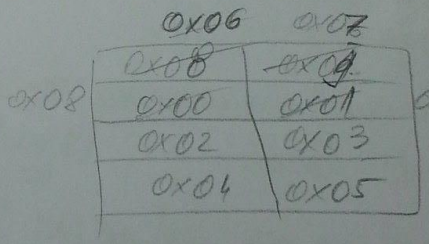
miss (1)
miss (2) miss(2) miss
miss (3) hit hit
miss (4) miss miss
miss (5) hit hit
miss (6) miss miss



$T_{total} = 6 * T_{miss} + (0x02 - 1) * (3 T_{miss} + 2 T_{hit})$

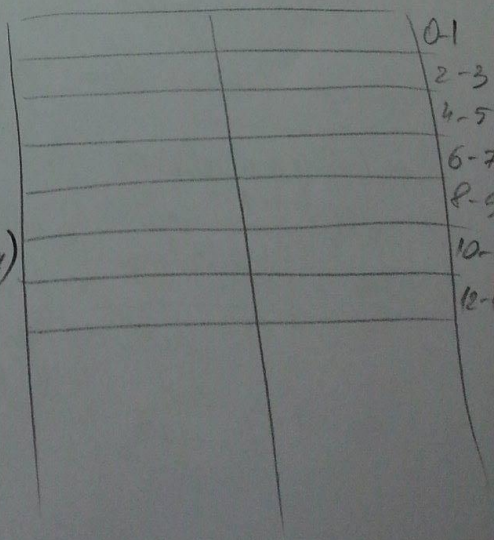
$= 600 + (0x02 - 1) * 320$

c) Adresele sunt mapate full asociativ și politica de înlocuire a liniilor = FIFO



(1) miss

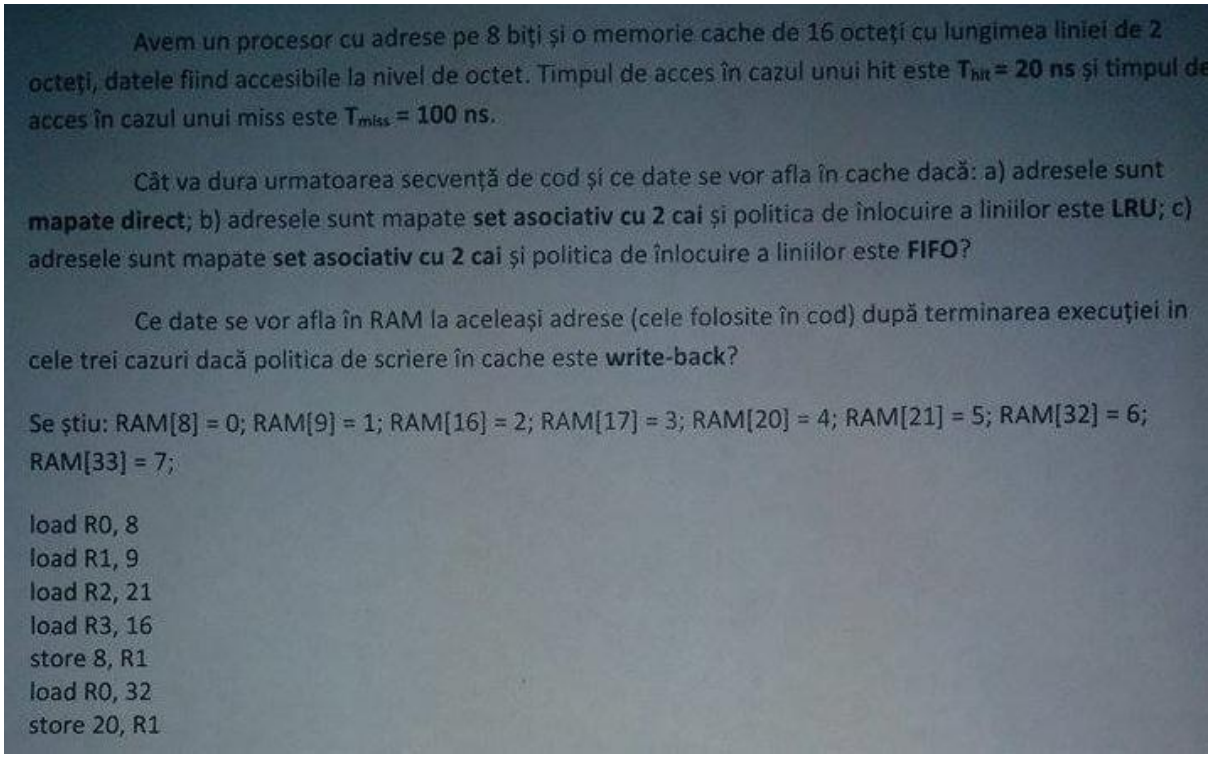
2 miss miss
3 miss miss
4 miss miss
5 miss miss
6 miss miss



$6 \cdot T_{miss} + (0x08 - 1) T_{miss}$

$= T_{miss} * (5 + 0x08) = 600 * (5 + 0x08)$

(08.09.2014)



Avem un procesor cu adrese pe 8 biți și o memorie cache de 16 octeți cu lungimea liniei de 2 octeți, datele fiind accesibile la nivel de octet. Timpul de acces în cazul unui hit este T_{hit} = **20 ns** și timpul de acces în cazul unui miss este T_{miss} = **100 ns**.

Cât va dura urmatoarea secvență de cod și ce date se vor afla în cache dacă: a) adresele sunt **mapate direct**; b) adresele sunt mapate **set asociativ cu 2 cai** și politica de înlocuire a liniilor este **LRU**; c) adresele sunt mapate **set asociativ cu 2 cai** și politica de înlocuire a liniilor este **FIFO**?

Ce date se vor afla în RAM la aceleași adrese (cele folosite în cod) după terminarea execuției în cele trei cazuri dacă politica de scriere în cache este **write-back**?

Se știu: RAM[8] = 0; RAM[9] = 1; RAM[16] = 2; RAM[17] = 3; RAM[20] = 4; RAM[21] = 5; RAM[32] = 6; RAM[33] = 7;

```
load R0, 8
load R1, 9
load R2, 21
load R3, 16
store 8, R1
load R0, 32
store 20, R1
```

(03.02.2014)

ATENTIE!!!

8-9 se mapeaza la 4

4 e tot miss, la 5 ar fi trebuit sa fie hit

RAM-ul nu cred ca e bine actualizat, pentru ca la write-back se actualizeaza doar linia invalidata, nu tot ce are flag “dirty” pana atunci.

Adrese mapate [illegible]
Memoria CPU

1. Adresa mapată direct

Cache

	32	33
0	16	17
1		
2	20	21
3	1	9
4		
5		
6		
7		

RAM

0-1		
2-3		
8-9	8 1	1
16-17	2	3
20-21	4	5
32-33	6	7

1: miss
2: hit
3: miss
4: hit
5: miss
6: miss
7: hit

T = 3 hit + 4 miss

Write-back => RAM actualizat doar la invalidare linie => doar 0x08 se actualizează

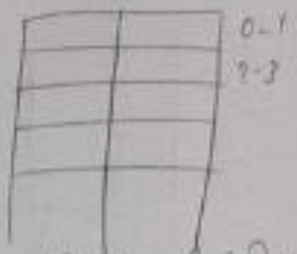
B. Adresse mapate 2-way set asociativ

Cache

16	17
20	21
8	9

32	33

RAM



1: miss
2: hit
3: miss
4: miss
5: hit
6: miss
7: hit

T = 3 hit + 4 miss
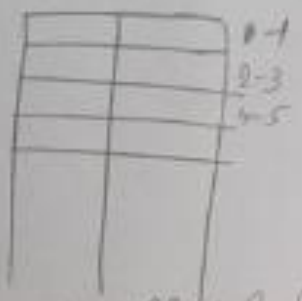
RAM-ul raspunde la fel, i.e. cu miss deoarece un a fost invalidat

C. Adresse mapate full-asociativ

Cache

8	9
20	21
16	17
32	33

RAM



1- miss
2- hit
3- miss
4- miss
5- hit
6- miss
7- hit

T = 3 hit + 4 miss RAM la fel.

1. Un procesor are doua niveluri de cache (L1 si L2) si o memorie RAM. ~~Sistemul are un CPI=1 daca~~ Programul care ruleaza pe aceasta arhitectura are 10000 de operații load și store care au 10% hit in L1, 30% hit in L2 iar restul rezida in RAM. Timpii de acces pentru diferitele memorii sunt: t_{L1} = 2cicli, t_{L2}=5 cicli si t_{RAM}=100 cicli. Care este valoarea timpului mediu de acces la memorie? (1p)

2. Un bus sincron are o perioadă de ceas de 10ns și fiecare transmisie durează un ciclu de ceas. Alt bus asincron are nevoie de 40ns pentru fiecare handshake. Fiecare bus are 32 de biți de date. Care este lățimea de bandă a fiecărui bus pentru citiri de un cuvânt dintr-o memorie cu 500ns per read? (1p)

3. Avem un procesor cu adrese pe 8 biți și o memorie cache de 16 octeți cu lungimea liniei de 2 octeți, datele fiind accesibile la nivel de octet. Timpul de acces în cazul unui hit este $\mathbf{T_{hit}}$ **= 20 ns** și timpul de acces în cazul unui miss este $\mathbf{T_{miss}}$ **= 100 ns**. Cât va dura urmatoarea secvență de cod și ce date se vor afla în cache dacă:

a) adresele sunt **mapate direct**;(1p)

b) adresele sunt mapate **set asociativ cu 2 cai** și politica de înlocuire a liniilor este **LRU**; (2p)

c) adresele sunt mapate **set asociativ cu 2 cai** și politica de înlocuire a liniilor este **FIFO**? (2p)

d) Ce date se vor afla în RAM la aceleași adrese (cele folosite în cod) după terminarea execuției in cele trei cazuri dacă politica de scriere în cache este **write-back**? (4p)

Se știu: RAM[8] = 0; RAM[9] = 1; RAM[16] = 2; RAM[17] = 3; RAM[20] = 4; RAM[21] = 5; RAM[32] = 6; RAM[33] = 7;

```
load R0, 8
load R1, 9
load R2, 21
load R3, 16
store 8, R1
load R0, 32
store 20, R1
```

1. E o combinatie intre AMAT si niveluri multiple de cache, dar nu stiu sigur cum se face

(am urmat modelul din curs)

2. Pentru bus sincron:
   - trimite adresa la memorie: 10ns
   - citeste memoria: 500ns
   - trimite date la dispozitiv: 10ns

   => in total = 500 + 10 + 10 = 520ns

   => BW = 4 octeti la 520ns => 4 / 520 = 7.69 MB/s

   Pentru bus asincron:
   - memory read: 40ns
   - data ready & hadshake: 3 * 40 + 500 = 620 ns
   - read & ack: 3 * 40 = 120 ns

   => in total = 780 ns

   => BW = 4 octeti la 780ns => 4 / 780 = 5.12 MB/s

3. e acelasi cu cel de mai sus

(28.01.2014)

Este partea de problema care consta din 3 exercitii revin cu ele cand fac rost
In principiu erau probleme cu speedup, performanta si numarat miss-uri intr-un cache.

Rezolvare of Problemă examen CN2 09.02.2015

Avem un procesor cu adrese pe 8 biți și o memorie cache de 16 octeți cu lungimea liniei de 2 octeți, datele fiind accesibile la nivel de octet. Timpul de acces în cazul unui hit este T_{hit} = 50 ns și timpul de acces în cazul unui miss este T_{miss} = 250 ns. Cât va dura urmatoarea secvență de cod și ce date se vor afla în cache dacă:

A. Adresele sunt mapate direct
B. Adresele sunt mapate set asociativ cu 2 căi și politica de înlocuire a liniilor este LRU
C. Adresele sunt mapate full asociativ și politica de înlocuire a liniilor este FIFO

Ce date se vor afla în RAM la aceleași adrese (cele folosite în cod) după terminarea execuției in cele trei cazuri dacă politica de scriere în cache este write-through?

Se știu:
RAM[0x04] = 0;
RAM[0x05] = 1;
RAM[0x0A] = 2;
RAM[0x0B] = 3;
RAM[0x1A] = 4;
RAM[0x1B] = 5;
RAM[0x40] = 6;
RAM[0x41] = 7;

1) lds R_{19}, 0x0A
2) lds R_{20}, 0x0B
3) sts 0x40, R_{20}
4) lds $R_{20,}$ 0x1A
5) lds R_{21}, 0x05
6) sts 0x05, R_{19}
7) lds R_{19}, 0x0A

Observație: Pentru fiecare subpunct scrieți ce conține memoria RAM la adresele folosite în cod, ce se află în fiecare linie de cache la finalul execuției codului și timpul total de execuție. Puteți scrie timpul necesar fiecărei instrucțiuni (se punctează parțial).

A.

lds R_{19}, 0x0A ; miss
lds R_{20}, 0x0B ; hit
sts 0x40, R_{20} ; miss
lds $R_{20,}$ 0x1A ; miss
lds R_{21}, 0x05 ; miss
sts 0x05, R_{19} ; hit
lds R_{19}, 0x0A ; miss

Cache:

0: 0x40, 0x41
1: x, x
2: 0x04, 0x05
3: x, x
4: x, x
5: ~~0x0A~~ ~~0x1A~~ 0x0A, ~~0x0B~~ ~~0x1B~~ 0x0B———
6: x, x
7: x, x

RAM:

RAM[0x40] = 3;
RAM[0x05] = 2;

$T = 5*T_{miss} + 2*T_{hit} = 1350$

B. b

lds R_{19}, 0x0A ; miss
lds R_{20}, 0x0B ; hit
sts 0x40, R_{20} ; miss
lds $R_{20,}$ 0x1A ; miss
lds R_{21}, 0x05 ; miss
sts 0x05, R_{19} ; hit
lds R_{19}, 0x0A ; hit

Cache:
0.0: 0x40, 0x41
0.1: x, x
1.0: 0x0A, 0x0B
1.1: 0x1A, 0x1B
2.0: 0x04, 0x05
2.1: x, x———
3.0: x, x
3.1: x, x

RAM:
RAM[0x40] = 3;
RAM[0x05] = 2;

$T = 4*T_{miss} + 3*T_{hit} = 1150$

C. c

lds R_{19}, 0x0A ; miss
lds R_{20}, 0x0B ; hit
sts 0x40, R_{20} ; miss
lds $R_{20,}$ 0x1A ; miss
lds R_{21}, 0x05 ; miss
sts 0x05, R_{19} ; hit
lds R_{19}, 0x0A ; hit

Cache:
0: 0x0A, 0x0B
0: 0x40, 0x41
0: 0x1A, 0x1B
0: 0x04, 0x05
0: x, x
0: x, x———
0: x, x
0: x, x

RAM:
RAM[0x40] = 3;
RAM[0x05] = 2;

$T = 4*T_{miss} + 3*T_{hit} = 1150$

:p

1) De cate chip-uri de memorie 32K x 8 aveti nevoie pt a construi o memorie 256K x 32 ?
a) 10
*b) 32
c) 4
d) 8
e) 16
memorie / memorie_chip = nr_chipuri
256K * 32 / (32K * 8) = 32

2) Un sistem de calcul cu un singur nivel de cache are o rata de hit de 85%. Timpul de acces la date in cazul unui cache hit este de 2ns, iar timpul de access in cazul unui miss este de 500ns. Care este timpul mediu de acces la memorie?
a) 2.75ns
b) 9.5ns
*c) 77ns
d) 752ns
e) nicio varianta
AMAT = hit_time + miss_rate * miss_penalty
1 = hit_rate + miss_rate
=> AMAT = 2 + (1 - 85/100) * 500 = 2 + 75 = 77

3) Un sistem cache L1 si L2 are un miss rate de 5% din L1 si 2.5% din L2. Care este miss rate-ul global?
a) 2.5%
b) 12.5%
c) 0.125%
*d) 1.25%
e) 0.0125%
x = nr fetch-uri
5% din x sunt miss-uri in L1 (deci cauta in L2 mai departe)
2.5% din 5% din x sunt miss-uri in L2
=> 2.5/100 * 5/100 = 1.25/100 => 1.25% rata miss globala

4) Citirea dintr-un fisier exploateaza:
a) localitatea temporala
*b) localitatea spatiala
c) ambele localitati
d) niciuna
e) nu se poate determina
Datele din fisier sunt in zona continua de memorie, dupa o citire, se va intoarce in acelasi loc in memorie (plus un deplasament) pt urmatoarea citire => localitate spatiala.

5) Avem o memorie cache cu linii de 4 octeti lungime organizata in 64 de seturi a cate 2 linii.

Daca adresarea se face la nivel de octet, cu adrese pe 32 de biti, care este dimensiunea in biti a tag-ului pt fiecare linie de cache?
a) 21
b) 20
*c) 23
d) 24
e) 22
o_adresa = offset + set_index + tag
line_width = 4B = 2^2 B
num_of_sets = dim_of_set / set_size
fiecare linie, deci numaram si liniile:
num_of_sets = 64 * 2 = 2^7
=> 32 - 2 - 7 = 23

6) Pt o masina de calcul pe 32 biti, divizam adresele virtuale in urmatoarele campuri:
10-bit | 8-bit | 6-bit | 8-bit
Folosim o tablea de pagini ierarhica pe 3 niveluri, astfel incat primii 10 biti sunt pt primul nivel, urmatorii 8 pt al doilea si ultimii 6 pt al treilea. Segmentul final ce contine ultimii 8 biti specifica indexul in cadrul paginii. Care este dimensiunea totala a tabelei de pagini pt un proces care ocupa 256KB de memorie, incepand cu adresa 0?
a) 1024 octeti
b) 2304 octeti
c) 4608 octeti
d) 1304 octeti
e) 256 octeti
Cum se rezolva aici, cu o explicatie clara?

7) Iulian optimizeaza programul de mai jos pt o executie mai rapida. A procedat corect?
*a) Da
b) Nu
Before:

```
for (i=0; i<n; i++) {
        for (j=0; j<n; j++) {
                sum = 0.0;
                for (k=0; k<n; k++)
                        sum += a[i][k] * b[k][j];
                c[i][j] = sum;
        }
}
```

After:

```
for (k=0; k<n; k++) {
        for (i=0; i<n; i++) {
```

```
                r = a[i][k];
                for (j=0; j<n; j++)
                        c[i][j] += r * b[k][j];
        }
}
```

Sumarea se face pe linii la a doua vairanta: c[i][j] se misca pe linie, b[k][j] e pe linie. Parcurgerile pe linii sunt mai rapide decat cele pe coloane din cauza ca la o linie ne putem folosi de cache, la o coloana nu.

8) Avem un sistem de memorie virtuala cu o tabela de pagini pe un singur nivel, stocata in memorie. Daca rata de hit pt TLB este de 80%, o cautare in TLB dureaza 15ns si dureaza 150ns pt un singur acces in memoria principala, care este timpul mediu de acces la memorie?
a) 185ns
b) 195ns
c) 205ns
d) 175ns
e) 165ns
Nu se aplica aceleasi formule ca la cache cu hit si miss? ca memoria fizica e un cache pt memoria virtuala.
AMAT = hit_time + miss_rate * miss_penalty
1 = hit_rate + miss_rate
AMAT = 15 + (1 - 80/100) * (15 + 150) 15 + 33 = 48
15 + 150 pt ca tot trebuie sa caute in tlb, si pt hit si pt miss, sa vada de care e. am presupus ca la un hit doar cauta, gaseste. iar pt un miss cauta, nu gaseste, si trebuie sa acceseze memoria.

9) Adevarat sau fals: Translatarea adreselor din memoria virtuala este utila chiar si atunci cand dimensiunea totala a memoriei virtuale este mai mica decat dimensiunea memoriei fizice.
Da, este utila intrucat inca poate oferi siguranta ca programele nu isi acceseaza unele altora memoria.
=> Adevarat.

10) Avem doua memorii cache de aceeasi dimensiune si care au acelasi timp de acces. Una este complet set-asociativa si cealalta este mapata direct. Daca codul vostru nu acceseaza aceeasi adresa de doua ori, care memorie va genera mai putine miss-uri?
a) set-asociativa
b) mapata direct
*c) sunt egale
d) niciuna
e) nu se poate determina
Daca codul nostru nu acceseaza aceeasi adresa de doua ori, inseamna ca vom avea doar compulsory misses.
=> sunt egale

Started on	Friday, 19 February 2021, 8:19 AM
State	Finished
Completed on	Friday, 19 February 2021, 8:48 AM
Time taken	29 mins 13 secs
Grade	**9.00** out of 10.00 (**90**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Bill dorește să construiască un cip de memorie de 128K x 32, dar are doar cipuri de 8K x 16. De câte cipuri are nevoie Bill?

- a. 32 ✔
- b. 64
- c. 256
- d. 16
- e. 128

(128K / 8K) * (32 / 16) = 32

The correct answer is: 32

Question **2**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 3ns iar rata de hit locală este 95%
- Latența de acces din L1 în L2 este de 9ns iar rata de hit locală este de 98%
- Latența de acces din L2 în memoria RAM este de 600ns

Care este timpul mediu de acces la memorie?

- a. 4050ns
- b. 0.405ns
- c. 40.5ns
- d. 405ns
- e. 4.05ns ✔

0.95 * 3ns + 0.05 * (0.98 * 12ns + 0.02 * 612ns) = 4.05ns

The correct answer is: 4.05ns

Question **3**

Correct

Mark 1.00 out of 1.00

Un sistem cu cache L1 si L2 are un hit rate de 95% din L1 și de 98% din L2. Care este miss rate-ul global?

- a. 0.1% ✔
- b. 1%
- c. 0.01%
- d. 0.99%
- e. 99%

1 - 0.95 - 0.05 * 0.98 = 0.001

The correct answer is: 0.1%

Question **4**

Correct

Mark 1.00 out of 1.00

Parcurgerea unei liste înlănțuite alocate contiguu în memorie exploatează:

- a. Localitatea temporală
- b. Nu se poate determina
- c. Localitatea spațială ✔
- d. Ambele localități
- e. Niciuna

Lista este alocată contiguu în memorie, iar parcurgere rezultă în acces secvențial. Asta înseamnă că, dacă am accesat o adresă din listă, este probabil să vrem să accesăm și adresele următoare sau precedente.

The correct answer is: Localitatea spațială

Question **5**

Correct

Mark 1.00 out of 1.00

O memorie cache cu linii de 32 octeți lungime este organizată în 32 de seturi a câte 32 linii. Dacă adresarea se face la nivel de octet, cu adrese pe 32 de biți, care este dimensiunea în biți a tag-ului pentru fiecare linie de cache?

- a. 20 ✔
- b. 18
- c. 19
- d. 21
- e. 17

~~32 - log2(32 * 32 * 32) = 17~~

32 - log2(32 * 32) = 22. Cum nu ati primit aceasta varianta, orice raspuns e corect!

The correct answers are: 20, 21, 19, 18, 17

Question **6**

Correct

Mark 1.00 out of 1.00

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before

```
for (i = 0; i < N; i++)
  a[i] = b[i] + 100;
for (i = 0; i < N; i++)
  b[i] = c[i] * 3.14;
for (i = 0; i < N; i++)
  c[i] = d[i] + e[i];
```

After

```
for (i = 0; i < N; i++){
  a[i] = b[i] + 100;
  b[i] = c[i] * 3.14;
  c[i] = d[i] + e[i];
}
```

- a. Da ✔
- b. Nu

Your answer is correct.

A doua variantă este mai rapidă fiindcă are mai puține bucle.

The correct answer is:
Da

Question **7**

Correct

Mark 1.00 out of 1.00

Presupuneți că acum 20 de ani ați scris un cod pentru o arhitectura x86 care petrecea jumătate din timp cu accese la memorie și cealaltă jumătate făcând calcule. Dacă am rula același cod astăzi pe o arhitectură x86-compatible, cel mai probabil el ar:

- a. Exporta calculele in Cloud
- b. Nu se poate determina
- c. Petrece mai mult timp cu accesul memoriei ✔
- d. Menține aceeași proporție 50-50% a timpilor de rulare
- e. Petrece mai mult timp făcând calcule

Viteza procesoarelor a crescut mai mult decât viteza memoriilor.

The correct answer is: Petrece mai mult timp cu accesul memoriei

Question **8**

Incorrect

Mark 0.00 out of 1.00

Dacă înjumătățim dimensiunea liniilor unui cache dar păstrăm numărul de seturi și asociativitatea neschimbate, cum se modifică miss penalty?

- a. Rămâne neschimbată
- b. Se dublează
- c. Se înjumătățește
- d. Se triplează
- e. Imposibil de determinat ✖

Dimensiunea liniilor, numărul de seturi și asociativitatea nu influențează miss penalty. Ele vor influența miss rate, dar timpii ce acces rămân aceiași.

The correct answer is: Rămâne neschimbată

Question **9**

Correct

Mark 1.00 out of 1.00

Magistrala de comunicație SPI:

- a. Permite comunicația la distanțe mari
- b. Are o schemă de adresare pe 8 biți
- c. Are o topologie multi-master, multi-slave
- d. Are o viteză de transmisie de maxim 9600 biți pe secundă
- e. Este o magistrală sincronă de transfer a datelor ✔

SPI este un protocol de comunicare sincron (i.e. folosește un semnal de ceas).

The correct answer is: Este o magistrală sincronă de transfer a datelor

Question **10**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul ce are doar un L1 cache de 128kB este **întotdeauna** mai rapid decât un sistem ce are L1 și L2 cache cu dimensiunea totală de 128kB.

- a. Adevărat
- b. Fals ✔

Nu putem zice că un sistem cu un nivel de cache este **întotdeauna** mai rapid ca un sistem cu două niveluri de cache. Chiar dacă primul sistem are mai mult cache rapid (L1), există cazuri în care asta nu îl ajută să atingă o performanță mai mare ca al doilea sistem. Spre exemplu, un program poate să nu acceseze memoria deloc, caz în care cele două sisteme sunt la fel de rapide.

The correct answer is: Fals

◄ Colocviu

Jump to...

Examen - Partea II ►

Started on	Thursday, 28 January 2021, 8:19 AM
State	Finished
Completed on	Thursday, 28 January 2021, 8:58 AM
Time taken	39 mins 34 secs
Grade	**8.85** out of 10.00 (**89**%)

Question **1**

Complete

Mark 2.50 out of 2.50

Ce este Program Counter? Care este utilitatea lui? Dați exemplu de 2 instrucţiuni care îi alterează conţinutul.

Program Counter este un registru folosit pentru determinarea urmatorii instructiuni de executat in cadrul unui program. Acesta este utilizat in etape de Instruction Fetch, in care este adusa instructiunea din zona de memoria catre care pointeaza continutul Program Counter. In urma executarii unei instructiuni, Program Counter este incrementat in vederea trecerii la urmatoarea instructiune (de obicei), insa continutul acestuia poate fi manipulat si prin tipuri specifice de instructiuni pentru modificarea flow ului obisnuit al programului. Aceste instructiuni (de tip salt) pot fi: BREQ, BRBS, RJMP

Comment:

Question **2**

Complete

Mark 2.50 out of 2.50

Este vreo greşeală în următoarea secvență de cod AVRASM? Dacă da, corectați problema. Justificați.

```
    rjmp main
foo:
    ldi r16, 256
    mov r7, 8
    reti
main:
    rcall foo
```

Greseli: r16 este pe 8 biti, nu se pot incarca valori mai mari de 266 -> ldi r16, 256

folosind MOV nu se poate schimba valoare dintr-un registru cu o valoare imediat, ci cu valoarea din alt registru -> mov r7, 8

Folosirea reti in loc de ret in cadrul unei subrutine.

Corectare:

```
rjmp main
foo:
ldi r16, 220
mov r7, r0
ret
main:
rcall foo
```

Comment:

Question **3**

Complete

Mark 2.25 out of 2.50

Folosind setul de instrucțiuni AVR, scrieți o funcție care salvează în R20 cea mai mare dintre valorile aflate în registrele R16, R17, R8 și R19. Scrieți un program care apelează funcția cu R16 = 1, R17 = 20, R8 = 12 și R19 = 70.

```
foo:
sub r17, r16
brge r17_greater
sub r16, r8
brge r16_greater_2
r8_greater_2:
sub r8, r19
brge r8_greater_3
r19_greater_3:
mov r20, r19
ret
r17_greater:
```

```
sub r17, r8
brge r17_greater_2
jmp r8_greater_2
r17_greater_2:
sub r17, r19
jmp r19_greater_3
brge r17_greater_3
r17_greater_3:
mov r20, r17
ret
r16_greater_2
sub r16, r19
brge r16_greater_3
jmp r19_greater_3
r16_greater_3:
mov r20, r16
ret
r8_greater_3:
mov r20, r8
ret
main:
ldi r16, 1
ldi r17, 20
ldi r8, 12
ldi r19, 70
call foo
```

Comment:

Question **4**

Complete

Mark 1.60 out of 2.50

Folosind setul de instrucțiuni AVR, scrieți o secvență de cod care:
- Configurează portul A al unui microcontroller ATtiny20 astfel încât toți pinii să fie de ieșire
- Configurează portul B al aceluiași microcontroller astfel încât toți pinii să fie de intrare
- Citește valoarea pinilor de intrare și o scrie pe pinii de ieșire

Configurarea portA pentru ca toti pinii sa fie de iesire:

ldi r16, 255

sts 0x02, r16

Configurare portB pentru ca toti pinii sa fie de intrare:

ldi r15, 0

sts 0x06, r15

Comment:
Ultimul subpunct nu l-ai facut.

◄ Cursul 2

Jump to...

Examen - Partea I ►

Started on	Friday, 19 February 2021, 8:19 AM
State	Finished
Completed on	Friday, 19 February 2021, 8:52 AM
Time taken	33 mins 16 secs
Grade	**9.00** out of 10.00 (**90**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Bill dorește să construiască un cip de memorie de 512K x 64, dar are doar cipuri de 16K x 8. De câte cipuri are nevoie Bill?

- a. 16
- b. 8
- c. 128
- d. 256 ✔
- e. 64

(512K / 16K) * (64 / 8) = 256

The correct answer is: 256

Question **2**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 4ns iar rata de hit locală este 90%
- Latența de acces din L1 în L2 este de 10ns iar rata de hit locală este de 95%
- Latența de acces din L2 în memoria RAM este de 500ns

Care este timpul mediu de acces la memorie?

- a. 75ns
- b. 0.075ns
- c. 0.75ns
- d. 750ns
- e. 7.5ns ✔

0.9 * 4ns + 0.1 * (0.95 * 14ns + 0.05 * 514ns) = 7.5ns

The correct answer is: 7.5ns

Question **3**

Correct

Mark 1.00 out of 1.00

Un sistem cu cache L1 si L2 are un hit rate de 80% din L1 și de 90% din L2. Care este miss rate-ul global?

- a. 20%
- b. 9.8%
- c. 0.02%
- d. 2% ✔
- e. 98%

1 - 0.8 - 0.2 * 0.9 = 0.02

The correct answer is: 2%

Question **4**

Incorrect

Mark 0.00 out of 1.00

Parcurgerea unui vector exploatează:

- a. Localitatea temporală
- b. Niciuna
- c. Nu se poate determina
- d. Ambele localități ✖
- e. Localitatea spațială

Vectorii sunt ținuți contiguu în memorie iar parcugerea unui vector rezultă în acces secvențial. Asta înseamnă că, dacă am accesat o adresă din vector, este probabil să vrem să accesăm și adresele următoare sau precedente.

The correct answer is: Localitatea spațială

Question **5**

Correct

Mark 1.00 out of 1.00

O memorie cache cu linii de 32 octeți lungime este organizată în 32 de seturi a câte 32 linii. Dacă adresarea se face la nivel de octet, cu adrese pe 32 de biți, care este dimensiunea în biți a tag-ului pentru fiecare linie de cache?

- a. 18
- b. 17
- c. 20 ✔
- d. 19
- e. 21

~~32 - log2(32 * 32 * 32) = 17~~

32 - log2(32 * 32) = 22. Cum nu ati primit aceasta varianta, orice raspuns e corect!

The correct answers are: 20, 21, 19, 18, 17

Question **6**

Correct

Mark 1.00 out of 1.00

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before

```
for (i = 0; i < N; i++)
 a[i] = b[i] * c[i];
for (i = 0; i < N; i++)
 d[i] = a[i] * c[i];
```

After

```
for (i = 0; i < N; i++) {
 a[i] = b[i] * c[i];
 d[i] = a[i] * c[i];
}
```

- a. Nu
- b. Da ✔

Your answer is correct.

A doua variantă este mai rapidă fiindcă are mai puține bucle.

The correct answer is:
Da

Question **7**

Correct

Mark 1.00 out of 1.00

Problema cu un cache ce folosește o politică Write-Through este:

- a. Riscă să lase memoria principală invalidă pentru o perioada lungă de timp
- b. Nici o variantă
- c. Necesită un overhead suplimentar în cache pentru a contabiliza blocurile care au fost modificate
- d. Nu poate fi folosită în sisteme multi-procesor, chiar dacă CPU-urile fac snooping pentru un update
- e. Scade performanța cache-ului ✔

Faptul că la fiecare scriere cache-ul trebuie să facă o scriere și în RAM înseamnă că nu poate fi la fel de rapid ca un cache care nu trebuie să facă asta.

The correct answer is: Scade performanța cache-ului

Question **8**

Correct

Mark 1.00 out of 1.00

Avem un sistem de calcul cu un singur nivel de cache (L1). Dacă adăugam un al doilea nivel de cache (L2), cum se vor modifica următorii parametri de funcționare: L1 hit time, L1 miss rate, L1 miss penalty?

- a. L1 hit time scade, L1 miss rate neschimbat, L1 miss penalty scade
- b. L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty scade ✔
- c. L1 hit time scade, L1 miss rate crește, L1 miss penalty scade
- d. L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty crește
- e. L1 hit time crește, L1 miss rate scade, L1 miss penalty scade

Viteza de acces în primul nivel de cache rămâne neschimbată, deci L1 hit time rămâne neschimbat. Dimensiunea și funcționarea primului nivel de cache rămân neschimbate, deci miss rate rămâne neschimbat. Cum accesul la al doilea nivel de cache este mai rapid decât accesul la RAM, miss penalty scade.

The correct answer is: L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty scade

Question **9**

Correct

Mark 1.00 out of 1.00

Magistrala de comunicație serială RS232:

- a. Folosește baud rate pentru a calcula distanța optimă de transmisie
- b. Stă la baza tuturor rețelelor de calculatoare
- c. Permite conectarea punct-la-punct a două dispozitive de calcul ✔
- d. Implementează un mecanism CSMA-CD
- e. Are o schemă de adresare pe 8 biți

RS232 este un protocol de comunicare punct-la-punct.

The correct answer is: Permite conectarea punct-la-punct a două dispozitive de calcul

Question **10**

Correct

Mark 1.00 out of 1.00

Translatarea adreselor din memoria virtuală este utilă chiar și atunci când dimensiunea totală a memoriei virtuale este mai mică decât dimensiunea memoriei fizice

- a. Fals
- b. Adevărat ✔

Memoria virtuală este utilă chiar și atunci când spațiul de adresă virtual este mai mic decât spațiul de adresă fizic, spre exemplu, pentru separația spațiului de adresă între procese.

The correct answer is: Adevărat

◄ Colocviu

Jump to...

Examen - Partea II ►

Started on	Friday, 19 February 2021, 8:19 AM
State	Finished
Completed on	Friday, 19 February 2021, 8:43 AM
Time taken	23 mins 27 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Bill dorește să construiască un cip de memorie de 256K x 32, dar are doar cipuri de 32K x 8. De câte cipuri are nevoie Bill?

- a. 8
- b. 32 ✔
- c. 128
- d. 16
- e. 64

(256K / 32K) * (32 / 8) = 32

The correct answer is: 32

Question **2**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 4ns iar rata de hit locală este 90%
- Latența de acces din L1 în L2 este de 10ns iar rata de hit locală este de 95%
- Latența de acces din L2 în memoria RAM este de 500ns

Care este timpul mediu de acces la memorie?

- a. 750ns
- b. 0.075ns
- c. 7.5ns ✔
- d. 0.75ns
- e. 75ns

0.9 * 4ns + 0.1 * (0.95 * 14ns + 0.05 * 514ns) = 7.5ns

The correct answer is: 7.5ns

Question **3**

Correct

Mark 1.00 out of 1.00

Un sistem cu cache L1 si L2 are un hit rate de 98% din L1 și de 99% din L2. Care este miss rate-ul global?

- a. 98%
- b. 9.8%
- c. 2%
- d. 20%
- e. 0.02% ✔

1 - 0.98 - 0.02 * 0.99 = 0.0002

The correct answer is: 0.02%

Question **4**

Correct

Mark 1.00 out of 1.00

Incrementarea unei variabile exploatează:

- a. Niciuna
- b. Ambele localități
- c. Localitatea temporală ✔
- d. Nu se poate determina
- e. Localitatea spațială

Incrementarea unei variabile presupune scrieri repetate la aceeași adresă. Deci, dacă am accesat acea adresă este probabil că vom dori să o accesăm din nou în curând.

The correct answer is: Localitatea temporală

Question **5**

Correct

Mark 1.00 out of 1.00

O memorie cache cu linii de 32 octeți lungime este organizată în 32 de seturi a câte 32 linii. Dacă adresarea se face la nivel de octet, cu adrese pe 32 de biți, care este dimensiunea în biți a tag-ului pentru fiecare linie de cache?

- a. 17
- b. 21
- c. 18
- d. 19
- e. 20 ✔

~~32 - log2(32 * 32 * 32) = 17~~

32 - log2(32 * 32) = 22. Cum nu ati primit aceasta varianta, orice raspuns e corect!

The correct answers are: 20, 21, 19, 18, 17

Question **6**

Correct

Mark 1.00 out of 1.00

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before:

```
static long factorial_table[] = {1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880};
```

```
long long factorial(int i) {
    if(i < 10) return factorial_table[i];
    else return i * factorial(i - 1);
}
```

After:

```
long long factorial(int i){
    if (i == 0)
        return 1;
    else
        return i * factorial(i - 1);
}
```

- a. Nu ✔
- b. Da

Your answer is correct.

Prima variantă este mai rapidă fiindcă are primii 10 termeni ai șirului factorial pre-calculați.

The correct answer is:
Nu

Question **7**

Correct

Mark 1.00 out of 1.00

Problema cu un cache ce folosește o politică Write-Through este:

- a. Scade performanța cache-ului ✔
- b. Riscă să lase memoria principală invalidă pentru o perioada lungă de timp
- c. Nu poate fi folosită în sisteme multi-procesor, chiar dacă CPU-urile fac snooping pentru un update
- d. Nici o variantă
- e. Necesită un overhead suplimentar în cache pentru a contabiliza blocurile care au fost modificate

Faptul că la fiecare scriere cache-ul trebuie să facă o scriere și în RAM înseamnă că nu poate fi la fel de rapid ca un cache care nu trebuie să facă asta.

The correct answer is: Scade performanța cache-ului

Question **8**

Correct

Mark 1.00 out of 1.00

Care funcție de mapare cache are cele mai puține șanse să producă thrashing (adică oferă șanse minime ca două blocuri să concureze la stocarea în aceeași linie)?

- a. Directă
- b. Complet-asociativă ✔
- c. Toate trei
- d. Nu se poate determina
- e. Set-asociativă

Cum orice linie de memorie poate intra în orice linie de cache, un cache complet-asociativ oferă cea mai bună șansă de a evita thrashing.

The correct answer is: Complet-asociativă

Question **9**

Correct

Mark 1.00 out of 1.00

O magistrala de comunicație serială:

- a. Poate transfera date doar asincron
- b. Permite doar comunicarea punct la punct
- c. Necesita un control mai complex decât o magistrală paralelă ✔
- d. Necesită în general un număr mare de linii de comunicație
- e. Are limitări puternice de lățime de bandă

Un protocol de comunicare serial necesită un control mai complex pentru, spre exemplu, determinarea biților transmiși și a numărului acestora.

The correct answer is: Necesita un control mai complex decât o magistrală paralelă

Question **10**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul ce are doar un L1 cache de 128kB este **întotdeauna** mai rapid decât un sistem ce are L1 și L2 cache cu dimensiunea totală de 128kB.

- a. Fals ✔
- b. Adevărat

Nu putem zice că un sistem cu un nivel de cache este **întotdeauna** mai rapid ca un sistem cu două niveluri de cache. Chiar dacă primul sistem are mai mult cache rapid (L1), există cazuri în care asta nu îl ajută să atingă o performanță mai mare ca al doilea sistem. Spre exemplu, un program poate să nu acceseze memoria deloc, caz în care cele două sisteme sunt la fel de rapide.

The correct answer is: Fals

◄ Colocviu

Jump to...

Examen - Partea II ►

1) De cate chip-uri de memorie 32K x 8 aveti nevoie pt a construi o memorie 256K x 32 ?
a) 10
*b) 32
c) 4
d) 8
e) 16
memorie / memorie_chip = nr_chipuri
256K * 32 / (32K * 8) = 32

2) Un sistem de calcul cu un singur nivel de cache are o rata de hit de 85%. Timpul de acces la date in cazul unui cache hit este de 2ns, iar timpul de access in cazul unui miss este de 500ns. Care este timpul mediu de acces la memorie?
a) 2.75ns
b) 9.5ns
*c) 77ns
d) 752ns
e) nicio varianta
AMAT = hit_time + miss_rate * miss_penalty
1 = hit_rate + miss_rate
=> AMAT = 2 + (1 - 85/100) * 500 = 2 + 75 = 77

3) Un sistem cache L1 si L2 are un miss rate de 5% din L1 si 2.5% din L2. Care este miss rate-ul global?
a) 2.5%
b) 12.5%
c) 0.125%
*d) 1.25%
e) 0.0125%
x = nr fetch-uri
5% din x sunt miss-uri in L1 (deci cauta in L2 mai departe)
2.5% din 5% din x sunt miss-uri in L2
=> 2.5/100 * 5/100 = 1.25/100 => 1.25% rata miss globala

4) Citirea dintr-un fisier exploateaza:
a) localitatea temporala
*b) localitatea spatiala
c) ambele localitati
d) niciuna
e) nu se poate determina
Datele din fisier sunt in zona continua de memorie, dupa o citire, se va intoarce in acelasi loc in memorie (plus un deplasament) pt urmatoarea citire => localitate spatiala.

5) Avem o memorie cache cu linii de 4 octeti lungime organizata in 64 de seturi a cate 2 linii.

Daca adresarea se face la nivel de octet, cu adrese pe 32 de biti, care este dimensiunea in biti a tag-ului pt fiecare linie de cache?
a) 21
b) 20
*c) 23
d) 24
e) 22
o_adresa = offset + set_index + tag
line_width = 4B = 2^2 B
num_of_sets = dim_of_set / set_size
fiecare linie, deci numaram si liniile:
num_of_sets = 64 * 2 = 2^7
=> 32 - 2 - 7 = 23

6) Pt o masina de calcul pe 32 biti, divizam adresele virtuale in urmatoarele campuri:
10-bit | 8-bit | 6-bit | 8-bit
Folosim o tablea de pagini ierarhica pe 3 niveluri, astfel incat primii 10 biti sunt pt primul nivel, urmatorii 8 pt al doilea si ultimii 6 pt al treilea. Segmentul final ce contine ultimii 8 biti specifica indexul in cadrul paginii. Care este dimensiunea totala a tabelei de pagini pt un proces care ocupa 256KB de memorie, incepand cu adresa 0?
a) 1024 octeti
b) 2304 octeti
c) 4608 octeti
d) 1304 octeti
e) 256 octeti
Cum se rezolva aici, cu o explicatie clara?

7) Iulian optimizeaza programul de mai jos pt o executie mai rapida. A procedat corect?
*a) Da
b) Nu
Before:

```
for (i=0; i<n; i++) {
        for (j=0; j<n; j++) {
                sum = 0.0;
                for (k=0; k<n; k++)
                        sum += a[i][k] * b[k][j];
                c[i][j] = sum;
        }
}
```

After:

```
for (k=0; k<n; k++) {
        for (i=0; i<n; i++) {
```

```
                r = a[i][k];
                for (j=0; j<n; j++)
                        c[i][j] += r * b[k][j];
        }
}
```

Sumarea se face pe linii la a doua vairanta: c[i][j] se misca pe linie, b[k][j] e pe linie. Parcurgerile pe linii sunt mai rapide decat cele pe coloane din cauza ca la o linie ne putem folosi de cache, la o coloana nu.

8) Avem un sistem de memorie virtuala cu o tabela de pagini pe un singur nivel, stocata in memorie. Daca rata de hit pt TLB este de 80%, o cautare in TLB dureaza 15ns si dureaza 150ns pt un singur acces in memoria principala, care este timpul mediu de acces la memorie?
a) 185ns
b) 195ns
c) 205ns
d) 175ns
e) 165ns
Nu se aplica aceleasi formule ca la cache cu hit si miss? ca memoria fizica e un cache pt memoria virtuala.
AMAT = hit_time + miss_rate * miss_penalty
1 = hit_rate + miss_rate
AMAT = 15 + (1 - 80/100) * (15 + 150) 15 + 33 = 48
15 + 150 pt ca tot trebuie sa caute in tlb, si pt hit si pt miss, sa vada de care e. am presupus ca la un hit doar cauta, gaseste. iar pt un miss cauta, nu gaseste, si trebuie sa acceseze memoria.

9) Adevarat sau fals: Translatarea adreselor din memoria virtuala este utila chiar si atunci cand dimensiunea totala a memoriei virtuale este mai mica decat dimensiunea memoriei fizice.
Da, este utila intrucat inca poate oferi siguranta ca programele nu isi acceseaza unele altora memoria.
=> Adevarat.

10) Avem doua memorii cache de aceeasi dimensiune si care au acelasi timp de acces. Una este complet set-asociativa si cealalta este mapata direct. Daca codul vostru nu acceseaza aceeasi adresa de doua ori, care memorie va genera mai putine miss-uri?
a) set-asociativa
b) mapata direct
*c) sunt egale
d) niciuna
e) nu se poate determina
Daca codul nostru nu acceseaza aceeasi adresa de doua ori, inseamna ca vom avea doar compulsory misses.
=> sunt egale

Started on	Thursday, 28 January 2021, 8:19 AM
State	Finished
Completed on	Thursday, 28 January 2021, 8:59 AM
Time taken	40 mins
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Complete

Mark 2.50 out of 2.50

Ce este Stack Pointer? Care este utilitatea lui? Dați exemplu de 2 instrucţiuni care îi alterează conţinutul.

Stack Pointer este un registru care stocheaza adresa varfului stivei (acesta se afla mereu pe prima pozitie liberea apropiata de baza stivei). Acesta se muta catre varf (se decrementeaza) cand se pune un octet pe stiva. Daca se ia un octet de pe stiva, stack pointerul se muta catre baza. Instructiuni care ii altereaza continutul: push, pop, rcall.

Comment:

Question **2**

Complete

Mark 2.50 out of 2.50

Care sunt cele trei condiţii care trebuie să fie îndeplinite pentru a fi generată o întrerupere către procesor?

1. sa fie activate la nivel global (bitul I din SREG sa fie 1)
2. intreruperea sa nu fie mascata (bitul corespunzator din registrul de control este 1, deci masca va fi 1)
3. intreruperea respectiva este activa (bitul dcorepsunzator din registrul ei de staree e 1, deci flagul va fi 1)

Comment:

Question **3**

Complete

Mark 2.50 out of 2.50

Folosind setul de instrucțiuni AVR, scrieți o funcție care salvează în R20 cea mai mare dintre valorile aflate în registrele R16, R17, R8 și R19. Scrieți un program care apelează funcția cu R16 = 1, R17 = 20, R8 = 12 și R19 = 70.

```
maxim:
  mov r25, r16
  sub r16, r17
  brsh r16maimare
  mov r20, r17

r16maimare:
  mov r20, r25
  mov r16, r20
  sub r16, r8
  brsh r20maimare1
  mov r20, r8

r20maimare1:
  mov r16, r20
  sub r16, r19
  brsh r20maimare2
  mov r20, r19

r20maimare2:
  ret

main:
  ldi r16,1
  ldi r17,20
  ldi r8, 12
  ldi r19, 70
  rcall maxim
  nop
```

Comment:

Question **4**

Complete

Mark 2.50 out of 2.50

Folosind setul de instrucțiuni AVR, scrieți o secvență de cod care:
- Configurează portul A al unui microcontroller ATtiny20 astfel: PA0, PA2, PA4, PA6 ca pini de ieşire
- Setează valoarea pinilor de ieșire să fie 0 (Low)
- Într-o buclă, schimbă starea pinilor de ieșire (toggle)

```
ldi r16, 0b01010101
out DDRA, r16

ldi r16, 0
out PORTA, r16

loop:
eor r16, r16
out PORTA, R16
rjmp loop
```

Comment:

◄ Cursul 2

Jump to...

Examen - Partea I ►

Started on	Thursday, 28 January 2021, 8:19 AM
State	Finished
Completed on	Thursday, 28 January 2021, 8:59 AM
Time taken	40 mins
Grade	**7.80** out of 10.00 (**78**%)

Question **1**

Complete

Mark 2.50 out of 2.50

Ce efect are instrucțiunea BRBC? Care sunt diferențele dintre instrucțiunile BRBC și RJMP?

BRBC (Branch If Bit SREG is Cleared) este o intructiune de control care va face salt doar daca bit-ul selectat este 0.

Diferente intre BRBC si RJMP:

- BRBC este o instructiune conditionata de valoarea din SREG, in timp ce RJMP este o instructiune de salt neconditionat

- RJMP este o instructiune cu un singur operand ce specifica numarul de instructiuni peste care trebuie sa sara counter-ul, in timp ce BRBC este o instructiune cu 2 operanzi, in care operandul suplimentar este bit-ul din SREG specificand flag-ul pe care dorim sa il verificam

- RJMP poate executa salt doar la +/- 2k instructiuni relativ de cea curenta, in timp ce BRBC se poate ramifica in functie de valoarea din PC

Comment:

Question **2**

Complete

Mark 2.50 out of 2.50

Într-un microcontroller ATtiny20, cum se face comunicarea cu perifericele? Exemplificați pentru perifericul Timer/Counter0.

Comunicarea cu perifericele intr-un microcontroller ATtiny20 se realizeaza printr-o magistrala in care exista un singur set de semnale (de control, date si adrese) la care au acces modulele conectate.

Timerele de pe ATtiny20 conțin 3 registre importante: TCNT0, OCR0A/OCR0B, TCCR0A, TCCR0B

In modul normal de functionare, Timer-ul numără crescător de la 0 la 255, apoi se resetează la 0

Comment:

Question **3**

Complete

Mark 1.30 out of 2.50

Folosind setul de instrucțiuni AVR, scrieți o funcție care salvează în R29 suma numerelor naturale mai mici ca n, unde n este valoarea din R18. Scrieți un program care apelează funcția pentru n = 10.

```
ldi r18, 10
ldi, r29, 0
loop:
add r29, r18
dec r18
brne loop
```

Comment:

Question **4**

Complete

Mark 1.50 out of 2.50

Configurați Timer/Counter0 al unui microcontroller ATtiny20 astfel încât:

- Să funcționeze în modul Normal
- Să schimbe starea pinului OC0B astfel: Set OC0B on Compare Match
- Să folosească un prescaler de 256
- Să genereze o întrerupere de overflow

```
ldi r16, 0b00000000
out TCCR0A, R16
ldi r16, 0b000100
out TCCR0B, R16
ldi r16,
out OCR0A, R16
```

Comment:

◄ Cursul 2

Jump to...

Examen - Partea I ►

Un sistem de calcul are două niveluri de cache cu următoarele specificații: Latența de acces la L1 este de 4ns iar rata de hit locală este 90% Latența de acces din L1 în L2 este de 10ns iar rata de hit locală este de 95% Latența de acces din L2 în memoria RAM este de 500ns Care este timpul mediu de acces la memorie?

a. 75ns
b. 0.075ns
c. 7.5ns +1. e bine + 1
d. 0.75ns
e. 750ns
AMAT = hit_time + miss_rate * miss_penalty
1 = hit_rate + miss_rate

Bill dorește să construiască un cip de memorie de 512K x 64, dar are doar cipuri de 16K x 8. De câte cipuri are nevoie Bill?

a.128

b.256 (+ 1 e okay)

c.8

d.16

e.64

memorie / memorie_chip = nr_chipuri

Un sistem de calcul are două niveluri de cache cu următoarele specificații:
Latența de acces la L1 este de 3ns iar rata de hit locală este 95%
Latența de acces din L1 în L2 este de 9ns iar rata de hit locală este de 98%
Latența de acces din L2 în memoria RAM este de 600ns

Timp = 3 + 0.45 + 0.6 = 4.05 ns
Cum ai calculat asta? +1 cineva??
Care este timpul mediu de acces la memorie?

a.4050ns

B.4.05ns +2

Cum ati calc?
Aduni: timp de hit + (100-95)% timp L1-L2 + (2% din 5)% L2-RAm
c.40.5ns

d.0.405ns

e.405ns

Un sistem cu cache L1 si L2 are un hit rate de 95% din L1 și de 98% din L2. Care este miss rate-ul global?
a. 0.1%
b. 99%
c. 1%
d. 0.01%
e. 0.99%
Formula = missrate1 * missrate2

Accesul la o stivă exploatează:
a. Ambele localități -> Nu e si asta? NU! Am dat eu submit
b. Localitatea temporală
c. Niciuna
d. Localitatea spațială
e. Nu se poate determina

Nu sunt ambele?

O memorie cache cu linii de 32 octeți lungime este organizată în 32 de seturi a câte 32 linii. Dacă adresarea se face la nivel de octet, cu adrese pe 32 de biți, care este dimensiunea în biți a tag-ului pentru fiecare linie de cache?
log2(32)= 5
log2(32)= 5
32 - 5 - 5= 22

a.20
b.18
c.21
d.19
E.17
Orice pui aici iti da punctaj, ca nu exista raspunsul corect (22)

O memorie cache cu linii de 8 octeți lungime este organizată în 128 de seturi a câte 4 linii. Dacă adresarea se face la nivel de octet, cu adrese pe 32 de biți, care este dimensiunea în biți a tag-ului pentru fiecare linie de cache?

a. 20
b. 21
c. 23
d. 19
e. 22

E?

Eu as zice 22. Sa ma contrazica cineva daca nu e ok.
Ai 3 biti pentru coloane si 7 biti pentru seturi. ????? deci?
Eu tot 22 zic

Parcurgerea unui vector exploatează:

a.Nu se poate determina
b.Localitatea spațială
c.Niciuna
d.Localitatea temporală
e.Ambele localități

Procesorul poate să citească din RAM în timp ce RAM-ul este reîmprospătat.sigur? Da ca elementele sunt grupate in memorie si procesorul se asteapta sa le accesezi unele dupa altele
Cred ca e FALS.

Decrementarea unui pointer și accesarea memoriei către care acesta pointează exploatează:

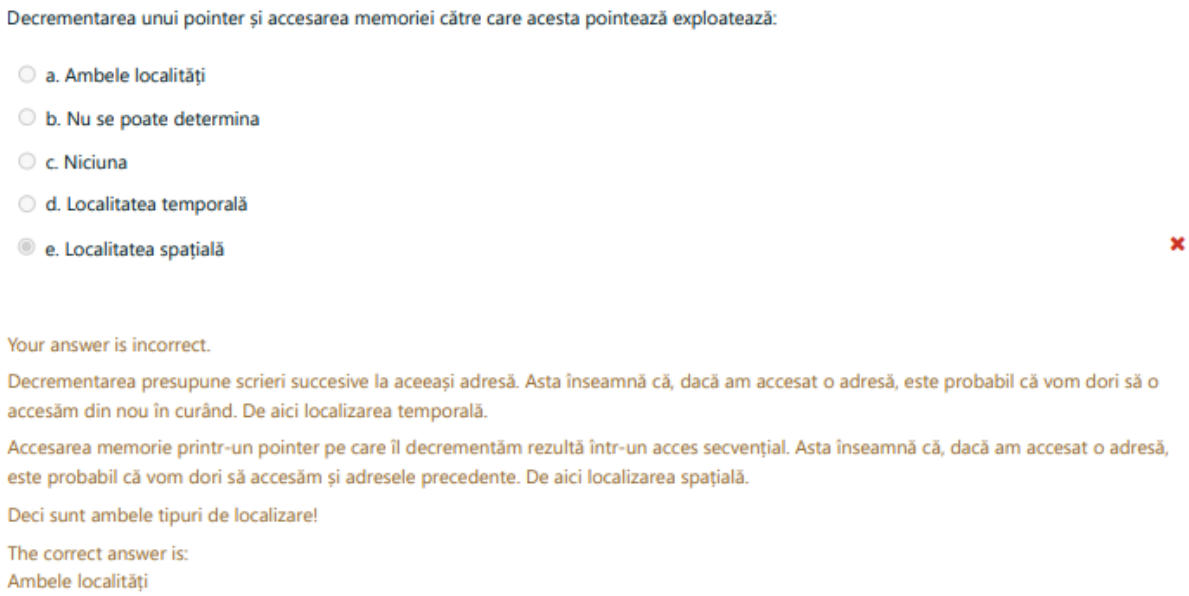
a.Ambele localități

b.Nu se poate determina

c.Niciuna

d.Localitatea temporală

e.Localitatea spațială



Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 2ns iar rata de hit locală este 93%
- Latența de acces din L1 în L2 este de 8ns iar rata de hit locală este de 94%
- Latența de acces din L2 în memoria RAM este de 300ns

Care este timpul mediu de acces la memorie?
a.38.2ns
b.3820ns
c.382ns
d.3.82ns
e.0.382ns

Un cache mapat direct este organizat intern pe 16384 linii cu lățime de 64 octeți. Adresele cu care lucrează memoria cache sunt pe 32 de biți. Care este lungimea în biți a tag-ului?

a.12

b.11

c.14

d.15

e.13

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect? Before:

```
for (i = 0; i < n; i++) {
  for (j = 0; j < n; j++) {
          sum = 0.0;
          for (k = 0; k < n; k++)
          sum += a[i][k] * b[k][j];
          c[i][j] = sum;
  }
}
```

After:

```
for (k = 0; k < n; k++) {
  for (i = 0; i < n; i++) {
          r = a[i][k];
          for (j = 0; j < n; j++) c[i][j] += r * b[k][j];
  }
}
 c[i][j] += r * b[k][j]; } }
```

a. Nu
b. Da
Cred ca e a

Presupuneți că acum 20 de ani ați scris un cod pentru o arhitectura x86 care petrecea jumătate din timp cu accese la memorie și cealaltă jumătate făcând calcule. Dacă am rula același cod astăzi pe o arhitectură x86-compatible, cel mai probabil el ar:

a.Menține aceeași proporție 50-50% a timpilor de rulare

b.Exporta calculele in Cloud

c.Petrece mai mult timp cu accesul memoriei

d.Nu se poate determina

e.Petrece mai mult timp făcând calcule

Cred ca e C sau D, vreo idee? e C

Un sistem cu cache L1 si L2 are un hit rate de 90% din L1 și de 95% din L2. Care este miss rate-ul global
a.0.05%
b.9.5%
c.**0.5%+1**
d.5%
e.95%

Cum se calculeaza?
miss_rate1 * miss_rate2 / 100 ?? Confirmare?

Un sistem cu cache L1 si L2 are un hit rate de 80% din L1 și de 90% din L2. Care este miss rate-ul global?

a.20%

b.0.02%

c.9.8%

d.2% -> cred ?

e.98%

Un sistem cu cache L1 si L2 are un hit rate de 98% din L1 și de 99% din L2. Care este miss rate-ul global?

a.

9.8%

b.

0.02%

c.

2%

d.

20%

e.

98%

Magistrala de comunicație serială RS232:

a.Are o schemă de adresare pe 8 biți

b.Stă la baza tuturor rețelelor de calculatoare

c.Permite conectarea punct-la-punct a două dispozitive de calcul asta cred

d.Folosește baud rate pentru a calcula distanța optimă de transmisie

e.Implementează un mecanism CSMA-CD

Cred ca conecteaza un calculator la periferice, eu cred ca e d
Cineva la asta? Aveti idee??

Care dintre următoarele este adevărată?
a. Un cache set-asociativ cu două seturi este un cache complet-asociativ
b. Un cache set-asociativ cu o singură linie per set este un cache mapat direct
c. Un cache set-asociativ cu un singur set este un cache mapat direct
d. Un cache set-asociativ cu o singură linie per set este un cache complet-asociativ
e. Toate variantele sunt adevărate
Pls si aici

e B +1

Incrementarea unei variabile exploatează:
a.Niciuna
b.Ambele localități
c.Nu se poate determina
d.Localitatea spațială
e.Localitatea temporală
Eu cred
ca E +2
Sigur?

Bill dorește să construiască un cip de memorie de 256K x 64, dar are doar cipuri de 16K x 8. De câte cipuri are nevoie Bill?

a.32

b.128

c.64

d.8

e.16

Parcurgerea unei liste înlănțuite alocate contiguu în memorie exploatează:

a.
Niciuna
b.
Localitatea temporală
c.
Ambele localități
d.
Nu se poate determina
e.
Localitatea spațială

E doar spatiala? ++ :(plss

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before:

```
 static long factorial_table[] = {1, 1, 2, 6, 24, 120, 720, 5040, 40320,
362880};

  long long factorial(int i) {

      if(i < 10) return factorial_table[i];

      else return i * factorial(i - 1);

  }
```

After:

```
 long long factorial(int i){

      if (i == 0)

          return 1;

      else

          return i * factorial(i - 1);

  }
```

a.

Nu

b.

Da

AICI? +1???

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before

```
for (i = 0; i < N; i++)
  a[i] = b[i] + 100;
for (i = 0; i < N; i++)
  b[i] = c[i] * 3.14;
for (i = 0; i < N; i++)
  c[i] = d[i] + e[i];
```

After

```
for (i = 0; i < N; i++){
  a[i] = b[i] + 100;
  b[i] = c[i] * 3.14;
  c[i] = d[i] + e[i];
}
```

a.Nu

b.Da + 3, si eu zic ca e optimizat ca nu mai faci un for de 3 ori, il faci o dataa

---

Bill dorește să construiască un cip de memorie de 128K x 32, dar are doar cipuri de 8K x 16. De câte cipuri are nevoie Bill?

a.

256

b.

128

c.

16

d.

32 e ok

e.

64

Bill dorește să construiască un cip de memorie de 1M x 64, dar are doar cipuri de 128K x 32. De câte cipuri are nevoie Bill?

a.32

b.8

c.64

d.128

E.16 asta ? da

Un sistem cu cache L1 si L2 are un hit rate de 80% din L1 și de 90% din L2. Care este miss rate-ul global?

a. 9.8%
b. 0.02%
c. 20%
d. 2%
e. 98%

---

un cache m

Bill dorește să construiască un cip de memorie de 128K x 32, dar are doar cipuri de 32K x 16. De câte cipuri are nevoie Bill?

a.

16

b.

128

c.

64

d.

32

e.

8

Un cache mapat direct este organizat intern pe 4096 linii cu lățime de 32 octeți. Adresele cu care lucrează memoria cache sunt pe 32 de biți. Care este lungimea în biți a tag-ului?
a.14
b.18
c.17
d.16
E.15 -> e asta?da, +1 am facut si eu acum

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 5ns iar rata de hit locală este 80%
- Latența de acces din L1 în L2 este de 12ns iar rata de hit locală este de 90%
- Latența de acces din L2 în memoria RAM este de 400ns

Care este timpul mediu de acces la memorie? (+1??) +1? +1?????

a.1540ns

b.154ns

c.1.54ns

d.15.4ns

E.0.154ns

Stiu ca s-a mai facut dar nu imi dau calculele deloc

timp de hit + (100-80)% timp L1-L2 + (10% din 20)% L2-RAm
5 + 20% din 12 + ??? din 400

Avem un sistem de calcul cu un singur nivel de cache (L1). Dacă adăugam un al doilea nivel de cache (L2), cum se vor modifica următorii parametri de funcționare: L1 hit time, L1 miss rate, L1 miss penalty?
a.L1 hit time scade, L1 miss rate neschimbat, L1 miss penalty scade
b.L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty crește
c.L1 hit time crește, L1 miss rate scade, L1 miss penalty scade
d.L1 hit time scade, L1 miss rate crește, L1 miss penalty scade
e.L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty scade
Cica ar fi b, nu stiu sigur
Si eu zic la fel, ca ai acc L1, dar ai mai mult penalty ca nu accesezi RAM-ul
B sau e?

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before:

```
for (i = 0; i < N; i++)
 c[i] = a[i] + b[i];
```

After:

```
int P = 1;
for (i = 0; i < N; i++) {
      prefetch(&a[i+P]);
      prefetch(&b[i+P]);
      prefetch(&c[i+P]);
      c[i] = a[i] + b[i];
}
```

DA
NU

NICIO IDEE

E

Citirea secvențială dintr-un fișier exploatează: a. Nu se poate determina b. Localitatea temporală c. Localitatea spațială d. Niciuna e. Ambele localități
Aveti idee va rog la asta? ++

Spatiala parca, dar nu mai stiu sigur, ca se intoarce in acc zona de mem

---

Dacă înjumătățim dimensiunea liniilor unui cache dar păstrăm numărul de seturi și asociativitatea neschimbate, cum se modifică miss penalty? cineva????????

a.Imposibil de determinat

b.Se înjumătățește

c.Rămâne neschimbată

d.Se dublează

e.Se triplează

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before:

```
 static long factorial_table[] = {1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880};
```

```
long long factorial(int i) {
    if(i < 10) return factorial_table[i];
    else return i * factorial(i - 1);
}
```

After:

```
long long factorial(int i){
    if (i == 0)
        return 1;
    else
        return i * factorial(i - 1);
}
```

a.
Da

b.
Nu

Care dintre următoarele rămâne neschimbat la translatarea unei adrese virtuale într-o adresă fizică?

a.

Page offset + 1

b.

Cache tag

c.

Toate variantele

d.

TLB tag

e.

Page number

Care dintre următoarele este un motiv valid pentru folosirea adreselor virtuale?

a.
Este mai dificil ca un program ce funcționează prost să afecteze alte programe concurente (cred, +1 si eu zic asta ca are rolul sa le izoleze intre ele)

b.Permite lucrul mai facil cu stiva

c.Mărește rata de transfer a dispozitivelor I/O

d.Permite upgrade-ul gratuit de la Windows 7 la Windows 10 (asta e funny)

e.Permite unui proces să acceseze întreg spațiul de memorie

O locație de memorie și una mapată într-un periferic pot fi accesate simultan de către un procesor.

a.Fals

b.Adevărat

Problema cu un cache ce folosește o politică Write-Through este:

a.

Scade performanța cache-ului asta zic eu; Da, asa scrie si in curs: “Probleme la scriere: Write-through încetinește memoria cache pentru a permite memoriei principale să scrie datele”

b.

Necesită un overhead suplimentar în cache pentru a contabiliza blocurile care au fost modificate

c.

Nu poate fi folosită în sisteme multi-procesor, chiar dacă CPU-urile fac snooping pentru un update

d.

Riscă să lase memoria principală invalidă pentru o perioada lungă de timp

e.

Nici o variantă

Care funcție de mapare cache are cele mai puține șanse să producă thrashing (adică oferă șanse minime ca două blocuri să concureze la stocarea în aceeași linie)?

a.Nu se poate determina

b.Toate trei

c.Directă

d.Set-asociativă

e.Complet-asociativă eu as fi zis asta da nu sunt sigur, ba da asta e

Concluzia: random intre d si e, yolo
:))))) cine a gresit da o bere dupa :))) ok deal -> astept berea :)
What is cache thrashing? - Quora posibil sa ajute pt asta de sus

---

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before

```
int i = 0;
while (i < 4*num) {
 o_functie(i);
 i++;
}
```

After

```
int i = 0;
while (i < 4*num) {
 o_functie(i);
 o_functie(i+1);
 o_functie(i+2);
 o_functie(i+3);
 i += 4;

}
```

a. **Tin minte de la iocla ca este o optimizare asta, da**

Da

b.

Nu

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before

```
for (i = 0; i < N; i++)
 a[i] = b[i] * c[i];
for (i = 0; i < N; i++)
 d[i] = a[i] * c[i];
```

After

```
for (i = 0; i < N; i++) {
 a[i] = b[i] * c[i];
 d[i] = a[i] * c[i];
}
```

Da ? Cineva?

Problema cu un cache ce folosește o politică Write-Back este:

a.

Nu menține consistența datelor din memoriile unui sistem multi-procesor

b.

Scade performanța unui cache

c.

Riscă să ocupe toată lățimea de bandă a magistralei procesor-memorie

d.

Nicio opțiune

e.

Necesită un TLB separat pentru a asigura transferul de date

Pentru a maximiza performanța, TLB-urile sunt organizate de obicei sub forma unor cache-uri mapate direct.

a.Adevărat

b.Fals

E fals, am dat submit

TLB–Mic:accesatîn<1ciclu–De obicei16-512intrări–Complet asociativ

Din curs, deci cred ca f

O magistrala de comunicație serială:

a.Permite doar comunicarea punct la punct

b.Necesită în general un număr mare de linii de comunicație

c.Poate transfera date doar asincron

d.Necesita un control mai complex decât o magistrală paralelă

e.Are limitări puternice de lățime de bandă
???
Nici eu nu stiu, cred ca e d
Eu as fi zis e

Avem un sistem de calcul cu două niveluri de cache (L1 și L2). Dacă înlăturăm nivelul L2 de cache, cum se va modifica L1 hit time?

a.Scade
b.Nu se modifică
c.Depinde de L2 hit time
d.Crește
e.Nu se poate calcula

As zice d, dar nu sunt sigur, e invers ca in poza cred

7. Avem un sistem de calcul cu un singur nivel de cache (L1) Daca adaugam un al doilea nivel de cache (L2), cum se vor modifica urmatorii parametri de functionare: L1 Hit Time, L1 Miss Rate, L1 Miss Penalty?

15.
L1 hit time scade, l1 miss penalty scade, l1 miss rate... (ramane la fel!?) // miss rate creste, deoarece L1 este un cache mai mic

Un TLB:

a.

Ajută la reducerea timpului de acces la o locație din memoria principală

b.

Poate fi folosit fără probleme și pentru accesul la locații de I/O

c.

Are un număr limitat de acționări pe ciclu de memorie

d.

Nu poate fi folosit în sisteme multi-procesor, pentru că produce inconsistențe la accesul memoriei partajate

e.

Nicio opțiune

AICII?????

Magistrala de comunicație SPI:
a.Are o topologie multi-master, multi-slave
b.Este o magistrală sincronă de transfer a datelor e asta?, asa zic si eu, am gasit pe net ca e sincrona, asa e
c.Are o viteză de transmisie de maxim 9600 biți pe secundă
d.Are o schemă de adresare pe 8 biți
e.Permite comunicația la distanțe mari

Care funcție de mapare cache are cele mai puține șanse să producă thrashing (adică oferă șanse minime ca două blocuri să concureze la stocarea în aceeași linie)?

a.

Complet-asociativă --- cred ca asta

b.

Toate trei

c.

Set-asociativă

d.

Directă

e.

Nu se poate determina

Cum o să fie afectați parametrii hit time, miss rate și miss penalty dacă adăugăm capabilitatea de prefetching unui cache?

a.Hit time neschimbat, miss rate crește, miss penalty crește

b.Hit time scade, miss rate crește, miss penalty neschimbat

c.Hit time neschimbat, miss rate scade, miss penalty neschimbat

d.Hit time scade, miss rate scade, miss penalty crește

e.Hit time creste, miss rate crește, miss penalty crește

R: ?

În toate sistemele multiprocesor, timpul de acces la memorie este identic pentru fiecare procesor.

a.Fals

b.Adevărat

Magistrala de comunicație serială sincronă SPI:

a.

Alocă o adresă de 16 biți pentru orice dispozitiv slave

b.

Este folosită in retelele 5G

c.

Permite interconectarea mai multor dispozitive master

d.

Permite selectarea dispozitivului slave printr-un semnal de chip-select

e.

Are un baud rate de maxim 115200kbps

cineva??????????

Translatarea adreselor din memoria virtuală este utilă chiar și atunci când dimensiunea totală a memoriei virtuale este mai mică decât dimensiunea memoriei fizice

a.Adevărat

b.Fals

Adevarat

PUNETI SI PROBLEME SA NE VERIFICAM/AJUTAM

Fie un procesor cu adrese pe 8 biți și o memorie cache de 8 octeți cu lungimea liniei de 2 octeți, datele fiind accesibile la nivel de octet. Timpul de acces la memorie în cazul unui cache hit este T_{hit} = 10 ns și timpul de acces în cazul unui cache miss este T_{miss} = 80 ns. Politica de scriere în cache este **write-back cu write-allocate**. Se știe că inițial RAM[i] = i, oricare i în [0x00, 0xFF].

Cerințe:

- Ce adrese și ce valori se vor afla în cache la finalul secvenței de cod?
- Ce date se vor afla în RAM la final, la adresele folosite în cod?
- Cât timp va dura secvența de cod?

Rezolvați cerințele **în fiecare dintre situațiile următoare**:

A. Adresele sunt mapate direct.
B. Adresele sunt mapate full asociativ și politica de înlocuire a liniilor este LRU.
C. Adresele sunt mapate set asociativ cu 2 căi și politica de înlocuire a liniilor este FIFO.

Note: Pentru rezolvarea acestei probleme vom presupune că instrucțiunile care nu accesează memoria (ldi, inc, dec, cpi, brne) sunt executate în timp 0. Pentru a facilita corectarea, vă rugăm să folosiți următoarea notație pentru indicarea răspunsurilor:

- Pentru adresele și valorile din cache: Cache[i] = Addr0 (Val0), ..., AddrN (ValN)
  - i este indexul liniei de cache
  - Addr0, ..., AddrN sunt adresele de memorie stocate în acea linie de cache
  - Val0, ..., ValN sunt datele de stocate în acea linie de cache
- Pentru datele din RAM: RAM[Addr] = Val
  - Addr este adresa din RAM
  - Val este valoarea de la acea adresă
- Pentru durata secvenței de cod: T = Xns
  - X este durata secvenței

```
1. ldi R16, 2
2. ldi R17, 11
3. ldi R18, 13
4. loop:
5.    lds R12, 0x09
6.    lds R19, 0x04
7.    sts 0x02, R17
8.    lds R20, 0x05
9.    lds R13, 0x03
10.   lds R15, 0x01
11.   sts 0x04, R18
12.   lds R14, 0x02
13.   lds r21, 0x10
14.   inc R17
15.   dec R18
16.   dec R16
17.   cpi R16, 0
18.   brne loop
```

Cum e in RAM la A? ++

Fie un procesor cu adrese pe 8 biți și o memorie cache de 8 octeți cu lungimea liniei de 2 octeți, datele fiind accesibile la nivel de octet. Timpul de acces la memorie în cazul unui cache hit este T_{hit} = 25 ns și timpul de acces în cazul unui cache miss este T_{miss} = 150 ns. Politica de scriere în cache este **write-through fără write-allocate**. Se știe că inițial RAM[i] = i, oricare i în [0x00, 0xFF].

Cerințe:

- Ce adrese și ce valori se vor afla în cache la finalul secvenței de cod?
- Ce date se vor afla în RAM la final, la adresele folosite în cod?
- Cât timp va dura secvența de cod?

Rezolvați cerințele **în fiecare dintre situațiile următoare**:

A. Adresele sunt mapate direct.
B. Adresele sunt mapate full asociativ și politica de înlocuire a liniilor este FIFO.
C. Adresele sunt mapate set asociativ cu 2 căi și politica de înlocuire a liniilor este LRU.

Note: Pentru rezolvarea acestei probleme vom presupune că instrucțiunile care nu accesează memoria (TODO) sunt executate în timp 0. Pentru a facilita corectarea, vă rugăm să folosiți următoarea notație pentru indicarea răspunsurilor:

- Pentru adresele și valorile din cache: Cache[i] = Addr0 (Val0), …, AddrN (ValN)
  - i este indexul liniei de cache
  - Addr0, …, AddrN sunt adresele de memorie stocate în acea linie de cache
  - Val0, …, ValN sunt datele de stocate în acea linie de cache
- Pentru datele din RAM: RAM[Addr] = Val
  - Addr este adresa din RAM
  - Val este valoarea de la acea adresă
- Pentru durata secvenței de cod: T = Xns
  - X este durata secvenței

```
1.  ldi R16, 2
2.  ldi R17, 17
3.  ldi R18, 18
4.  loop:
5.    lds R12, 0x00
6.    lds R19, 0x03
7.    sts 0x02, R17
8.    lds R20, 0x05
9.    lds R13, 0x04
10.   lds R15, 0x07
11.   sts 0x09, R18
12.   lds R21, 0x10
13.   lds R14, 0x02
14.   inc R17
15.   dec R18
16.   dec R16
17.   cpi r16, 0
18.   brne loop
```

Fie un procesor cu adrese pe 8 biți și o memorie cache de 8 octeți cu lungimea liniei de 2 octeți, datele fiind accesibile la nivel de octet. Timpul de acces la memorie în cazul unui cache hit este Thit = 25 ns și timpul de acces în cazul unui cache miss este Tmiss = 150 ns. Politica de scriere în cache este write-through fără write-allocate. Se știe că inițial RAM[i] = i, oricare i în [0x00, 0xFF].

Cerințe:

- Ce adrese și ce valori se vor afla în cache la finalul secvenței de cod?
- Ce date se vor afla în RAM la final, la adresele folosite în cod?
- Cât timp va dura secvența de cod?

Rezolvați cerințele în fiecare dintre situațiile următoare:

1. Adresele sunt mapate direct.
2. Adresele sunt mapate full asociativ și politica de înlocuire a liniilor este FIFO.
3. Adresele sunt mapate set asociativ cu 2 căi și politica de înlocuire a liniilor este LRU.

Note: Pentru rezolvarea acestei probleme vom presupune că instrucțiunile care nu accesează memoria (TODO) sunt executate în timp 0. Pentru a facilita corectarea, vă rugăm să folosiți următoarea notație pentru indicarea răspunsurilor:

- Pentru adresele și valorile din cache: Cache[i] = Addr0 (Val0), …, AddrN (ValN)

    - i este indexul liniei de cache
    - Addr0, ..., AddrN sunt adresele de memorie stocate în acea linie de cache
    - Val0, ..., ValN sunt datele de stocate în acea linie de cache
- Pentru datele din RAM: RAM[Addr] = Val
    - Addr este adresa din RAM
    - Val este valoarea de la acea adresă
- Pentru durata secvenței de cod: T = Xns
    - X este durata secvenței

```
1.  ldi R16, 2
2.  ldi R17, 17
3.  ldi R18, 18
4.  loop:
5.    lds R12, 0x00
6.    lds R19, 0x03
7.    sts 0x02, R17
8.    lds R20, 0x05
9.    lds R13, 0x04
10.   lds R15, 0x07
11.   sts 0x09, R18
12.   lds R21, 0x10
13.   lds R14, 0x02
14.   inc R17
15.   dec R18
16.   dec R16
17.   cpi r16, 0
18.   brne loop
```

Started on	Thursday, 28 January 2021, 8:19 AM
State	Finished
Completed on	Thursday, 28 January 2021, 8:59 AM
Time taken	40 mins 1 sec
Grade	**9.25** out of 10.00 (**93**%)

Question **1**

Complete

Mark 2.50 out of 2.50

Explicați diferența dintre instrucțiunile de salt condiționat și necondiționat. Dați exemplu de câte 2 instrucțiuni din fiecare categorie.

In cazul salturilor neconditionate continuarea rularii unui program are loc de la o anumita instructiune specificata, iar in cazul salturilor conditionate continuarea rularii unui program are loc de la o anumita instructiune specificata doar daca se indeplineste o anumita conditie.

Salt conditionat: breq, brbs

Salt neconditionat: rcall, rjmp

Comment:

Question **2**

Complete

Mark 2.50 out of 2.50

Ce se întamplă dacă, în timp ce procesorul se află într-o rutină de tratare a unei întreruperi (ISR), este generată o nouă întrerupere.

La inceputul tratarii oricarei intreruperi, intreruperile globale vor fi dezactivate cu sei, prin urmare nu va avea niciun efect aparitia unei noi intreruperi (aceasta va fi ignorata). Acestea vor fi activate apoi la finalul rutinei cu cli.

Comment:

Question **3**

Complete

Mark 2.50 out of 2.50

Folosind setul de instrucțiuni AVR, scrieți o funcție care salvează în R31 cea mai mică dintre valorile din registrele R27, R8, R29 și R30. Scrieți un program care apelează funcția cu R27 = 12, R8 = 13, R29 = 24 și R30 = 99.

```
rjmp main

minimum:
pop r16 ; adresa de retur
pop r17; adresa de retur
pop r30
pop r29
pop r18
mov r8, r18
pop r27
push r16; restauram adresa de retur
push r17; restauram adresa de retur

cpi r30, r29
brge r29_smaller
mov r20, r30

cpi r8, r27
brge r27_smaller
mov r21, r8

cpi r20, r21
brge r21_smaller
mov r30, r20
ret

r21_smaller:
mov r31, r21
ret

r29_smaller:
mov r20, r29

cpi r8, r27
brge r27_smaller2
mov r21, r8

r27_smaller2:
mov r21, r27

cpi r20, r21
brge r21_smaller2
```

```
mov r30, r20
ret

r21_smaller2:
mov r31, r21
ret

main:
ldi r27, 12
push r27
ldi r16, 13
mov r8, r16
push r8
ldi r29, 24
push r29
ldi r30, 99
push r30
rcall minimum
```

Comment:

Question **4**

Complete

Mark 1.75 out of 2.50

Configurați Timer/Counter0 al unui microcontroller ATtiny20 astfel încât:
- Să funcționeze în modul FastPWM cu top OCR0A
- Să schimbe starea pinului OC0A astfel: Clear OC0A on Compare Match, set OC0A at BOTTOM (non-inverting mode)
- Să folosească un prescaler de 1024
- Să genereze o întrerupere de overflow

-Să funcționeze în modul FastPWM cu top OCR0A => WGM = 111

- Să schimbe starea pinului OC0A astfel: Clear OC0A on Compare Match, set OC0A at BOTTOM (non-inverting mode) => COM0A = 10

- Să folosească un prescaler de 1024 => CS0 = 101

```
TCCR0A equ 0x19
TCCR0B equ 0x18
OCR0A equ 0x16
OCR0B equ 0x15
ldi r16, 0b10000011
out TCCR0A, r16

ldi r16, 0b00001101
out TCCR0B, r16
tim0_ovf
```

Comment: Nu ai configurat deloc întreruperile.

◄ Cursul 2

Jump to...

Examen - Partea I ►

Started on	Thursday, 28 January 2021, 8:19 AM
State	Finished
Completed on	Thursday, 28 January 2021, 8:59 AM
Time taken	40 mins
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Complete

Mark 2.50 out of 2.50

Ce este Stack Pointer? Care este utilitatea lui? Dați exemplu de 2 instrucţiuni care îi alterează conţinutul.

Stack Pointer este un registru care stocheaza adresa varfului stivei (acesta se afla mereu pe prima pozitie liberea apropiata de baza stivei). Acesta se muta catre varf (se decrementeaza) cand se pune un octet pe stiva. Daca se ia un octet de pe stiva, stack pointerul se muta catre baza. Instructiuni care ii altereaza continutul: push, pop, rcall.

Comment:

Question **2**

Complete

Mark 2.50 out of 2.50

Care sunt cele trei condiţii care trebuie să fie îndeplinite pentru a fi generată o întrerupere către procesor?

1. sa fie activate la nivel global (bitul I din SREG sa fie 1)
2. intreruperea sa nu fie mascata (bitul corespunzator din registrul de control este 1, deci masca va fi 1)
3. intreruperea respectiva este activa (bitul dcorepsunzator din registrul ei de staree e 1, deci flagul va fi 1)

Comment:

Question **3**

Complete

Mark 2.50 out of 2.50

Folosind setul de instrucțiuni AVR, scrieți o funcție care salvează în R20 cea mai mare dintre valorile aflate în registrele R16, R17, R8 și R19. Scrieți un program care apelează funcția cu R16 = 1, R17 = 20, R8 = 12 și R19 = 70.

```
maxim:
  mov r25, r16
  sub r16, r17
  brsh r16maimare
  mov r20, r17

r16maimare:
  mov r20, r25
  mov r16, r20
  sub r16, r8
  brsh r20maimare1
  mov r20, r8

r20maimare1:
  mov r16, r20
  sub r16, r19
  brsh r20maimare2
  mov r20, r19

r20maimare2:
  ret

main:
  ldi r16,1
  ldi r17,20
  ldi r8, 12
  ldi r19, 70
  rcall maxim
  nop
```

Comment:

Question **4**

Complete

Mark 2.50 out of 2.50

Folosind setul de instrucțiuni AVR, scrieți o secvență de cod care:
- Configurează portul A al unui microcontroller ATtiny20 astfel: PA0, PA2, PA4, PA6 ca pini de ieşire
- Setează valoarea pinilor de ieșire să fie 0 (Low)
- Într-o buclă, schimbă starea pinilor de ieșire (toggle)

```
ldi r16, 0b01010101
out DDRA, r16

ldi r16, 0
out PORTA, r16

loop:
eor r16, r16
out PORTA, R16
rjmp loop
```

Comment:

◄ Cursul 2

Jump to...

Examen - Partea I ►

Started on	Thursday, 28 January 2021, 8:13 AM
State	Finished
Completed on	Thursday, 28 January 2021, 8:52 AM
Time taken	39 mins 38 secs
Grade	**6.05** out of 10.00 (**61**%)

Question **1**

Complete

Mark 1.60 out of 2.50

Ce este Data Space? Ce se regăsește în interiorul său? Dați exemplu de două instrucțiuni care îl pot accesa.

Data Space reprezinta spatiul de date (memoria de date) si este un SRAM de tip pipeline (sincron, 128 bytes). Memoria de date este mapata inttre 0x0040 si 0x00BF. Adresele sunt pe 7 biti, iar datele au 8 biti (aceasta fiind dimensiunea unui registru).
Procesorul lucreaza cu 16 registre de 8 biti (exista totusi 32 registre), folosindu-le doar pe cele din R16-R31.

Exemple de instructiuni: ADD, MOV

Comment:
Instructiunile sunt de load sau de store, add si mov nu au treaba. De ce ai mentionat si registrele?

Question **2**

Complete

Mark 1.50 out of 2.50

Să presupunem că pe un microcontroller ATtiny20 a fost activată întreruperea de output compare match B a Timer/Counter0 și întreruperile globale sunt active. Cum putem face astfel încât la întâlnirea unei astfel de întreruperi să fie executată o funcție numită “handle_irq”?

Dupa executia unei instructiuni, procesorul va verifica IRQ, adica linia de cereri de intreruperi, vazand daca este activa. Daca da, inseamna ca exista o intrerupere care trebuie tratata. Apoi, verifica numarul intreruperii, adica IRQNO, ca sa determine modul de tratare a intreruperii. Dupa ce afla numarul, in tabela vectorilor de intrerupere, la indexul respectiv, va gasi rutina de tratatre a intreruperii respective, apoi o executa (executa astfel si functia "handle_irq").

Comment:
Nu ai zis concret cum se executa functia (rjmp handle_irq) si nici nu ai zis care e indexul vectorului de intrerupere.

Question **3**

Complete

Mark 0.50 out of 2.50

Folosind setul de instrucțiuni AVR, scrieți o funcție care salvează în R31 cea mai mică dintre valorile din registrele R27, R8, R29 și R30. Scrieți un program care apelează funcția cu R27 = 12, R8 = 13, R29 = 24 și R30 = 99.

```
funct_smth:
  ldi R27, 12
  ldi R8, 13
  ldi R29, 24
  ldi R30, 99

  mov R31, R27
  breq R31
```

Comment:
Nu ai implementat functia, nu ai terminat de apelat functia. In R8 nu poti incarca un imediat cu ldi, trebuie sa il incarci mai intai intr-un registru R16-R31, apoi sa il muti din el in R8.

Question **4**

Complete

Mark 2.45 out of 2.50

Folosind setul de instrucțiuni AVR, scrieți o secvență de cod care:
- Configurează portul A al unui microcontroller ATtiny20 astfel: PA0, PA1, PA2, PA3, PA4 și PA5 ca pini de ieșire, PA6 și PA7 ca pini de intrare
- Setează valoarea pinilor PA0 și PA4 să fie 1 (High)
- Setează valoarea celorlalți pini să fie 0 (Low)

Ciorna:

// ddra -> 0b00111111

// porta -> 0b00010001

Secventa cod:

```
DDRA equ 0x01

PORTA equ 0x02


ldi R16, 0b00111111

out DDRA, R16


ldi R16, 0b00010001

out PORTA, R16

breq some_tag


sbi PORTA, 0

some_tag:
```

Comment:
Ce ai vrut sa faci incepand cu breq? Subiectul era rezolvat deja deasupra lui.

◄ Cursul 2

Jump to...

Examen - Partea I ►

CN2- CA

verde = sigur bine

galben = la bulan / nu sunt sigur

rosu = gresit

bag pla

Intrebare:Bill dorește să construiască un cip de memorie de 256K x 64, dar are doar cipuri de 16K x 8. De câte cipuri are nevoie Bill?
Raspuns:
128==========

Intrebare:Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- **Latența de acces la L1 este de 2ns iar rata de hit locală este 93%**
- **Latența de acces din L1 în L2 este de 8ns iar rata de hit locală este de 94%**
- **Latența de acces din L2 în memoria RAM este de 300ns**

AMAT_L1 = hit_l1 + miss_l1 * AMAT_L2
AMAT_L2 = hit_l2 + miss_l2 * miss_penalty
Raspuns: 3.82ns
==========

Intrebare:Un sistem cu cache L1 si L2 are un hit rate de 98% din L1 și de 99% din L2. Care este miss rate-ul global?
Raspuns: 0.02%
==========

Intrebare:Parcurgerea unui vector exploatează
Raspuns: Localitatea spațială
==========

Intrebare:Un cache mapat direct este organizat intern pe 16384 linii cu lățime de 64 octeți. Adresele cu care lucrează memoria cache sunt pe 32 de biți. Care este lungimea în biți a tag-ului?
Raspuns: 12
==========

Intrebare:
Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before

```
int i = 0;
while (i < 4*num) {
 o_functie(i);
 i++;
}
```

After

```
int i = 0;
while (i < 4*num) {
 o_functie(i);
 o_functie(i+1);
 o_functie(i+2);
 o_functie(i+3);
 i += 4;
}
```

Raspuns: NU ****
==========

Intrebare:Care dintre următoarele este un motiv valid pentru folosirea adreselor virtuale?
Raspuns: Este mai dificil ca un program ce funcționează prost să afecteze alte programe concurente
==========

Intrebare:Care funcție de mapare cache are cele mai puține șanse să producă thrashing (adică oferă șanse minime ca două blocuri să concureze la stocarea în aceeași linie)?
Raspuns: Complet-asociativă
==========

Intrebare:O magistrala de comunicație serială:
Raspuns: Poate transfera date doar asincron
Raspuns bun: Necesita un control mai complex decât o magistrală paralelă
==========

Intrebare:Procesorul poate să citească din RAM în timp ce RAM-ul este reîmprospătat.
Raspuns: Fals

==========

Intrebare:
Raspuns:
==========

Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========

Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========

Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========

Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========
Intrebare:
Raspuns:
==========

Started on	Friday, 19 February 2021, 8:00 AM
State	Finished
Completed on	Friday, 19 February 2021, 8:31 AM
Time taken	30 mins 55 secs
Grade	**9.00** out of 10.00 (**90**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Bill dorește să construiască un cip de memorie de 512K x 64, dar are doar cipuri de 16K x 8. De câte cipuri are nevoie Bill?

- a. 128
- b. 8
- c. 256 ✔
- d. 16
- e. 64

(512K / 16K) * (64 / 8) = 256

The correct answer is: 256

Question **2**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 4ns iar rata de hit locală este 90%
- Latența de acces din L1 în L2 este de 10ns iar rata de hit locală este de 95%
- Latența de acces din L2 în memoria RAM este de 500ns

Care este timpul mediu de acces la memorie?

- a. 750ns
- b. 0.075ns
- c. 75ns
- d. 7.5ns ✔
- e. 0.75ns

0.9 * 4ns + 0.1 * (0.95 * 14ns + 0.05 * 514ns) = 7.5ns

The correct answer is: 7.5ns

Question **3**

Correct

Mark 1.00 out of 1.00

Un sistem cu cache L1 si L2 are un hit rate de 95% din L1 și de 98% din L2. Care este miss rate-ul global?

- a. 0.01%
- b. 1%
- c. 0.1% ✔
- d. 0.99%
- e. 99%

1 - 0.95 - 0.05 * 0.98 = 0.001

The correct answer is: 0.1%

Question **4**

Correct

Mark 1.00 out of 1.00

Parcurgerea unei liste înlănțuite alocate contiguu în memorie exploatează:

- a. Ambele localități
- b. Nu se poate determina
- c. Localitatea temporală
- d. Localitatea spațială ✔
- e. Niciuna

Lista este alocată contiguu în memorie, iar parcurgere rezultă în acces secvențial. Asta înseamnă că, dacă am accesat o adresă din listă, este probabil să vrem să accesăm și adresele următoare sau precedente.

The correct answer is: Localitatea spațială

Question **5**

Correct

Mark 1.00 out of 1.00

Un cache mapat direct este organizat intern pe 16384 linii cu lățime de 64 octeți. Adresele cu care lucrează memoria cache sunt pe 32 de biți. Care este lungimea în biți a tag-ului?

- a. 12 ✔
- b. 15
- c. 13
- d. 14
- e. 11

32 - log2(16384 * 64) = 12

The correct answer is: 12

Question **6**

Correct

Mark 1.00 out of 1.00

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before

```
for (i = 0; i < M; i++) {
 for (j = 0; j < N; j++) {
        x[i*N+j] = 2 * x[i*N+j];
 }
}
```

After

```
for (j = 0; j < N; j++) {
 for (i = 0; i < M; i++) {
        x[i*N+j] = 2 * x[i*N+j];
 }
}
```

- a. Nu ✔
- b. Da

Your answer is correct.

Prima variantă este mai rapidă fiindcă accesează matricea x în ordinea în care este stocată în memorie.

The correct answer is:
Nu

Question **7**

Correct

Mark 1.00 out of 1.00

Care dintre următoarele este un motiv valid pentru folosirea adreselor virtuale?

- a. Permite upgrade-ul gratuit de la Windows 7 la Windows 10
- b. Este mai dificil ca un program ce funcționează prost să afecteze alte programe concurente ✔
- c. Permite unui proces să acceseze întreg spațiul de memorie
- d. Mărește rata de transfer a dispozitivelor I/O
- e. Permite lucrul mai facil cu stiva

Memoria virtuală permite separarea spațiilor de adresă între programe (procese).

The correct answer is: Este mai dificil ca un program ce funcționează prost să afecteze alte programe concurente

Question **8**

Correct

Mark 1.00 out of 1.00

Care funcție de mapare cache are cele mai puține șanse să producă thrashing (adică oferă șanse minime ca două blocuri să concureze la stocarea în aceeași linie)?

- a. Set-asociativă
- b. Complet-asociativă ✔
- c. Directă
- d. Nu se poate determina
- e. Toate trei

Cum orice linie de memorie poate intra în orice linie de cache, un cache complet-asociativ oferă cea mai bună șansă de a evita thrashing.

The correct answer is: Complet-asociativă

Question **9**

Incorrect

Mark 0.00 out of 1.00

O magistrala de comunicație serială:

- a. Permite doar comunicarea punct la punct ✖
- b. Poate transfera date doar asincron
- c. Necesită în general un număr mare de linii de comunicație
- d. Necesita un control mai complex decât o magistrală paralelă
- e. Are limitări puternice de lățime de bandă

Un protocol de comunicare serial necesită un control mai complex pentru, spre exemplu, determinarea biților transmiși și a numărului acestora.

The correct answer is: Necesita un control mai complex decât o magistrală paralelă

Question **10**

Correct

Mark 1.00 out of 1.00

Procesorul poate să citească din RAM în timp ce RAM-ul este reîmprospătat.

- a. Adevărat
- b. Fals ✔

RAM-ul este inaccesibil în timpul unui refresh.

The correct answer is: Fals

◄ Colocviu

Jump to...

Examen - Partea II ►

Started on	Friday, 19 February 2021, 8:34 AM
State	Finished
Completed on	Friday, 19 February 2021, 8:43 AM
Time taken	9 mins 21 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Bill dorește să construiască un cip de memorie de 256K x 32, dar are doar cipuri de 32K x 8. De câte cipuri are nevoie Bill?

- a. 32 ✔
- b. 16
- c. 128
- d. 64
- e. 8

(256K / 32K) * (32 / 8) = 32

The correct answer is: 32

Question **2**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 2ns iar rata de hit locală este 93%
- Latența de acces din L1 în L2 este de 8ns iar rata de hit locală este de 94%
- Latența de acces din L2 în memoria RAM este de 300ns

Care este timpul mediu de acces la memorie?

- a. 0.382ns
- b. 38.2ns
- c. 3.82ns ✔
- d. 382ns
- e. 3820ns

0.93 * 2ns + 0.07 * (0.94 * 10ns + 0.06 * 310ns) = 3.82ns

The correct answer is: 3.82ns

Question **3**

Correct

Mark 1.00 out of 1.00

Un sistem cu cache L1 si L2 are un hit rate de 80% din L1 și de 90% din L2. Care este miss rate-ul global?

- a. 20%
- b. 9.8%
- c. 98%
- d. 2% ✔
- e. 0.02%

1 - 0.8 - 0.2 * 0.9 = 0.02

The correct answer is: 2%

Question **4**

Correct

Mark 1.00 out of 1.00

Citirea secvențială dintr-un fișier exploatează:

- a. Ambele localități
- b. Localitatea temporală
- c. Nu se poate determina
- d. Localitatea spațială ✔
- e. Niciuna

Citire secvențială înseamnă că, dacă am accesat o adresă, este probabil că vom dori să accesăm și adresele următoare.

The correct answer is: Localitatea spațială

Question **5**

Correct

Mark 1.00 out of 1.00

O memorie cache cu linii de 32 octeți lungime este organizată în 32 de seturi a câte 32 linii. Dacă adresarea se face la nivel de octet, cu adrese pe 32 de biți, care este dimensiunea în biți a tag-ului pentru fiecare linie de cache?

- a. 18
- b. 21 ✔
- c. 17
- d. 19
- e. 20

~~32 - log2(32 * 32 * 32) = 17~~

32 - log2(32 * 32) = 22. Cum nu ati primit aceasta varianta, orice raspuns e corect!

The correct answers are: 20, 21, 19, 18, 17

Question **6**

Correct

Mark 1.00 out of 1.00

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before:

```
for (i = 0; i < N; i++)
 for (j = 0; j < N; j++)
  B[j*N+i] = A[i*N+j];
```

After:

```
int BS = 8;
for (i = 0; i < N; i+=BS)
 for (j = 0; j < N; j++)
  for (k = 0; k < BS; k++)
    B[j*N+(i+k)] = A[(i+k)*N+j];
```

- a. Da
- b. Nu ✔

Your answer is correct.

Prima variantă este mai rapidă fiindcă matricea A este accesată complet în ordinea în care este stocată în memorie. În a doua variantă niciună dintre matrici nu este accesată în aceasta ordine.

The correct answer is:
Nu

Question **7**

Correct

Mark 1.00 out of 1.00

Care dintre următoarele rămâne neschimbat la translatarea unei adrese virtuale într-o adresă fizică?

- a. Toate variantele
- b. TLB tag
- c. Page offset ✔
- d. Cache tag
- e. Page number

Offset-ul în cadrul paginii rămâne același, chiar dacă adresa virtuală a paginii este diferită față de adresa fizică.

The correct answer is: Page offset

Question **8**

Correct

Mark 1.00 out of 1.00

Avem un sistem de calcul cu două niveluri de cache (L1 și L2). Dacă înlăturăm nivelul L2 de cache, cum se va modifica L1 hit time?

- a. Scade
- b. Crește
- c. Nu se poate calcula
- d. Nu se modifică ✔
- e. Depinde de L2 hit time

L1 hit time este timpul de acces la o adresă ce este deja in L1 cache, deci nu este influențat de L2 cache.

The correct answer is: Nu se modifică

Question **9**

Correct

Mark 1.00 out of 1.00

Magistrala de comunicație serială RS232:

- a. Are o schemă de adresare pe 8 biți
- b. Permite conectarea punct-la-punct a două dispozitive de calcul ✔
- c. Folosește baud rate pentru a calcula distanța optimă de transmisie
- d. Stă la baza tuturor rețelelor de calculatoare
- e. Implementează un mecanism CSMA-CD

RS232 este un protocol de comunicare punct-la-punct.

The correct answer is: Permite conectarea punct-la-punct a două dispozitive de calcul

Question **10**

Correct

Mark 1.00 out of 1.00

O locație de memorie și una mapată într-un periferic pot fi accesate simultan de către un procesor.

- a. Fals ✔
- b. Adevărat

Perifericele și memoria sunt mapate în același spațiu de adresă, deci nu pot fi accesate simultan.

The correct answer is: Fals

◄ Colocviu

Jump to...

Examen - Partea II ►

Started on	Friday, 19 February 2021, 8:02 AM
State	Finished
Completed on	Friday, 19 February 2021, 8:41 AM
Time taken	38 mins 58 secs
Grade	**5.00** out of 10.00 (**50**%)

Question **1**

Incorrect

Mark 0.00 out of 1.00

Bill dorește să construiască un cip de memorie de 256K x 32, dar are doar cipuri de 32K x 8. De câte cipuri are nevoie Bill?

- a. 32
- b. 8
- c. 64
- d. 16 ✖
- e. 128

(256K / 32K) * (32 / 8) = 32

The correct answer is: 32

Question **2**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 5ns iar rata de hit locală este 80%
- Latența de acces din L1 în L2 este de 12ns iar rata de hit locală este de 90%
- Latența de acces din L2 în memoria RAM este de 400ns

Care este timpul mediu de acces la memorie?

- a. 0.154ns
- b. 15.4ns ✔
- c. 1540ns
- d. 154ns
- e. 1.54ns

0.8 * 5ns + 0.2 * (0.9 * 17ns + 0.1 * 417ns) = 15.4ns

The correct answer is: 15.4ns

Question **3**

Correct

Mark 1.00 out of 1.00

Un sistem cu cache L1 si L2 are un hit rate de 95% din L1 și de 98% din L2. Care este miss rate-ul global?

- a. 99%
- b. 0.1% ✔
- c. 1%
- d. 0.01%
- e. 0.99%

1 - 0.95 - 0.05 * 0.98 = 0.001

The correct answer is: 0.1%

Question **4**

Incorrect

Mark 0.00 out of 1.00

Decrementarea unui pointer și accesarea memoriei către care acesta pointează exploatează:

- a. Ambele localități
- b. Nu se poate determina
- c. Niciuna
- d. Localitatea temporală
- e. Localitatea spațială ✖

Your answer is incorrect.

Decrementarea presupune scrieri succesive la aceeași adresă. Asta înseamnă că, dacă am accesat o adresă, este probabil că vom dori să o accesăm din nou în curând. De aici localizarea temporală.

Accesarea memorie printr-un pointer pe care îl decrementăm rezultă într-un acces secvențial. Asta înseamnă că, dacă am accesat o adresă, este probabil că vom dori să accesăm și adresele precedente. De aici localizarea spațială.

Deci sunt ambele tipuri de localizare!

The correct answer is:
Ambele localități

Question **5**

Incorrect

Mark 0.00 out of 1.00

Un cache mapat direct este organizat intern pe 4096 linii cu lățime de 32 octeți. Adresele cu care lucrează memoria cache sunt pe 32 de biți. Care este lungimea în biți a tag-ului?

- a. 18
- b. 15
- c. 14
- d. 17 ✖
- e. 16

32 - log2(4096 * 32) = 15

The correct answer is: 15

Question **6**

Incorrect

Mark 0.00 out of 1.00

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before:

```
for (i = 0; i < N; i++)
 for (j = 0; j < N; j++)
  B[j*N+i] = A[i*N+j];
```

After:

```
int BS = 8;
for (i = 0; i < N; i+=BS)
 for (j = 0; j < N; j++)
  for (k = 0; k < BS; k++)
    B[j*N+(i+k)] = A[(i+k)*N+j];
```

- a. Da ✖
- b. Nu

Your answer is incorrect.

Prima variantă este mai rapidă fiindcă matricea A este accesată complet în ordinea în care este stocată în memorie. În a doua variantă niciuna dintre matrici nu este accesată în aceasta ordine.

The correct answer is:
Nu

Question **7**

Correct

Mark 1.00 out of 1.00

Problema cu un cache ce folosește o politică Write-Through este:

- a. Nu poate fi folosită în sisteme multi-procesor, chiar dacă CPU-urile fac snooping pentru un update
- b. Scade performanța cache-ului ✔
- c. Nici o variantă
- d. Necesită un overhead suplimentar în cache pentru a contabiliza blocurile care au fost modificate
- e. Riscă să lase memoria principală invalidă pentru o perioada lungă de timp

Faptul că la fiecare scriere cache-ul trebuie să facă o scriere și în RAM înseamnă că nu poate fi la fel de rapid ca un cache care nu trebuie să facă asta.

The correct answer is: Scade performanța cache-ului

Question **8**

Incorrect

Mark 0.00 out of 1.00

Avem un sistem de calcul cu un singur nivel de cache (L1). Dacă adăugam un al doilea nivel de cache (L2), cum se vor modifica următorii parametri de funcționare: L1 hit time, L1 miss rate, L1 miss penalty?

- a. L1 hit time scade, L1 miss rate crește, L1 miss penalty scade
- b. L1 hit time scade, L1 miss rate neschimbat, L1 miss penalty scade
- c. L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty crește ✖
- d. L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty scade
- e. L1 hit time crește, L1 miss rate scade, L1 miss penalty scade

Viteza de acces în primul nivel de cache rămâne neschimbată, deci L1 hit time rămâne neschimbat. Dimensiunea și funcționarea primului nivel de cache rămân neschimbate, deci miss rate rămâne neschimbat. Cum accesul la al doilea nivel de cache este mai rapid decât accesul la RAM, miss penalty scade.

The correct answer is: L1 hit time neschimbat, L1 miss rate neschimbat, L1 miss penalty scade

Question **9**

Correct

Mark 1.00 out of 1.00

Magistrala de comunicație SPI:

- a. Are o schemă de adresare pe 8 biți
- b. Permite comunicația la distanțe mari
- c. Este o magistrală sincronă de transfer a datelor ✔
- d. Are o topologie multi-master, multi-slave
- e. Are o viteză de transmisie de maxim 9600 biți pe secundă

SPI este un protocol de comunicare sincron (i.e. folosește un semnal de ceas).

The correct answer is: Este o magistrală sincronă de transfer a datelor

Question **10**

Correct

Mark 1.00 out of 1.00

În toate sistemele multiprocesor, timpul de acces la memorie este identic pentru fiecare procesor.

- a. Fals ✔
- b. Adevărat

În cazul sistemelor NUMA (Non-Uniform Memory Access) timpul de acces la memorie diferă de la un procesor la altul.

The correct answer is: Fals

◄ Colocviu

Jump to...

Examen - Partea II ►

Început pe	vineri, 19 februarie 2021, 08:19
Status	Finalizat
Completat pe	vineri, 19 februarie 2021, 08:48
Timp de parcurgere test	28 min 53 secunde
Notă obținută	**10,00** din 10,00 (**100**%)

1 întrebare

Corect

Marcat 1,00 din 1,00

Bill dorește să construiască un cip de memorie de 128K x 32, dar are doar cipuri de 8K x 16. De câte cipuri are nevoie Bill?

- a. 32 ✔
- b. 64
- c. 128
- d. 256
- e. 16

(128K / 8K) * (32 / 16) = 32

Răspunsul corect este: 32

2 întrebare

Corect

Marcat 1,00 din 1,00

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 5ns iar rata de hit locală este 80%
- Latența de acces din L1 în L2 este de 12ns iar rata de hit locală este de 90%
- Latența de acces din L2 în memoria RAM este de 400ns

Care este timpul mediu de acces la memorie?

- a. 0.154ns
- b. 154ns
- c. 1540ns
- d. 15.4ns ✔
- e. 1.54ns

0.8 * 5ns + 0.2 * (0.9 * 17ns + 0.1 * 417ns) = 15.4ns

Răspunsul corect este: 15.4ns

3 întrebare

Corect

Marcat 1,00 din 1,00

Un sistem cu cache L1 si L2 are un hit rate de 90% din L1 și de 95% din L2. Care este miss rate-ul global?

- a. 0.5% ✔
- b. 95%
- c. 9.5%
- d. 5%
- e. 0.05%

1 - 0.9 - 0.1 * 0.95 = 0.005

Răspunsul corect este: 0.5%

4 întrebare

Corect

Marcat 1,00 din 1,00

Parcurgerea unei liste înlănțuite alocate contiguu în memorie exploatează:

- a. Niciuna
- b. Localitatea temporală
- c. Localitatea spațială ✔
- d. Ambele localități
- e. Nu se poate determina

Lista este alocată contiguu în memorie, iar parcurgere rezultă în acces secvențial. Asta înseamnă că, dacă am accesat o adresă din listă, este probabil să vrem să accesăm și adresele următoare sau precedente.

Răspunsul corect este: Localitatea spațială

5 întrebare

Corect

Marcat 1,00 din 1,00

O memorie cache cu linii de 32 octeți lungime este organizată în 32 de seturi a câte 32 linii. Dacă adresarea se face la nivel de octet, cu adrese pe 32 de biți, care este dimensiunea în biți a tag-ului pentru fiecare linie de cache?

- a. 17
- b. 19
- c. 18 ✔
- d. 20
- e. 21

~~32 - log2(32 * 32 * 32) = 17~~

32 - log2(32 * 32) = 22. Cum nu ati primit aceasta varianta, orice raspuns e corect!

Răspunsurile corecte sunt: 20, 21, 19, 18, 17

6 întrebare

Corect

Marcat 1,00 din 1,00

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before:

```
static long factorial_table[] = {1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880};
```

```
long long factorial(int i) {
    if(i < 10) return factorial_table[i];
    else return i * factorial(i - 1);
}
```

After:

```
long long factorial(int i){
    if (i == 0)
        return 1;
    else
        return i * factorial(i - 1);
}
```

- a. Da
- b. Nu ✔

Your answer is correct.

Prima variantă este mai rapidă fiindcă are primii 10 termeni ai șirului factorial pre-calculați.

Răspunsul corect este:
Nu

7 întrebare

Corect

Marcat 1,00 din 1,00

Presupuneți că acum 20 de ani ați scris un cod pentru o arhitectura x86 care petrecea jumătate din timp cu accese la memorie și cealaltă jumătate făcând calcule. Dacă am rula același cod astăzi pe o arhitectură x86-compatible, cel mai probabil el ar:

- a. Petrece mai mult timp cu accesul memoriei ✔
- b. Nu se poate determina
- c. Exporta calculele in Cloud
- d. Petrece mai mult timp făcând calcule
- e. Menține aceeași proporție 50-50% a timpilor de rulare

Viteza procesoarelor a crescut mai mult decât viteza memoriilor.

Răspunsul corect este: Petrece mai mult timp cu accesul memoriei

8 întrebare

Corect

Marcat 1,00 din 1,00

Un TLB:

- a. Are un număr limitat de acționări pe ciclu de memorie
- b. Nicio opțiune
- c. Ajută la reducerea timpului de acces la o locație din memoria principală ✔
- d. Nu poate fi folosit în sisteme multi-procesor, pentru că produce inconsistențe la accesul memoriei partajate
- e. Poate fi folosit fără probleme și pentru accesul la locații de I/O

TLB-ul ne ajută să facem mai repede translatarea dintr-o adresă virtuală într-o adresă fizică, deci face accesul la memoria principală mai rapid.

Răspunsul corect este: Ajută la reducerea timpului de acces la o locație din memoria principală

9 întrebare

Corect

Marcat 1,00 din 1,00

O magistrala de comunicație serială:

- a. Poate transfera date doar asincron
- b. Permite doar comunicarea punct la punct
- c. Necesita un control mai complex decât o magistrală paralelă ✔
- d. Are limitări puternice de lățime de bandă
- e. Necesită în general un număr mare de linii de comunicație

Un protocol de comunicare serial necesită un control mai complex pentru, spre exemplu, determinarea biților transmiși și a numărului acestora.

Răspunsul corect este: Necesita un control mai complex decât o magistrală paralelă

10 întrebare

Corect

Marcat 1,00 din 1,00

În toate sistemele multiprocesor, timpul de acces la memorie este identic pentru fiecare procesor.

- a. Fals ✔
- b. Adevărat

În cazul sistemelor NUMA (Non-Uniform Memory Access) timpul de acces la memorie diferă de la un procesor la altul.

Răspunsul corect este: Fals

◄ Colocviu

Sari la...

Examen - Partea II ►

Examen CN2. Partea I - 10.02.2017 Nume:..... Grupa:.....

1. Ce fel de arhitectura de memorie este mai potrivita pentru un sistem multiprocesor cu 1000 de procesoare?
a) UMA b) NUMA

2. Adevarat sau Fals: Atunci sistemul de fisiere aloca spatiu pentru un fisier pe disc, este mai bine ca acesta sa fie format din sectoare adiacente.

3. O memorie adresabila la nivel de octet de 4GB este conectata la un cache set asociativ 8-way de 1MB (fara tag bits) cu lungimea liniilor de 256B. Lungimea campurilor de tag, index, respectiv offset dintr-o adresa de 32 de biti este:
a) 21, 4, 7 b) 22, 4, 6 c) 20, 3, 9 d) 23, 3, 6 e) 20, 4, 8 f) 21, 3, 8

4. Presupuneti ca avem un sistem cache/memorie cu urmatorii parametri: L1 cache cu 5% miss rate, fiecare cache hit necesita un ciclu de ceas, frecventa CPU de 1GHz si un cache miss dureaza 20ns. Care este timpul mediu de acces la memorie (AMAT)?
a) 17.5nS b) 4.4nS c) 2.2nS d) 20nS e) 2.1nS f) 1.95nS

5. Pentru un hard-disk cu 4 platane, timpul necesar pentru a citi un fisier de 100Mb este de 10ms seek time plus 80ms transfer time. De cate ori este imbunatatita performanta daca numarul platanelor este dublat (presupuneti ca pentru fiecare dublare se injumatateste timpul de transfer)?
a) 2x b) 1.8x c) 3x d) 1.3x e) 1.9x f) nu se poate calcula

6. Un cache hit dureaza un ciclu de ceas iar penalizarea la miss este de 20ns pentru fiecare din arhitecturile cache de mai jos. Care din ele ofera un timp mediu de acces la memorie (AMAT) mai bun?
a) cache mapat direct – 10% miss rate @ 1GHz b) 2-way set asociativ – 5% miss rate @ 800MHz

7. Care este rolul unui TLB si de ce trebuie acesta sa ruleze rapid (1-2 cicli de ceas)?.
Rolul TLB este de memorie cache pentru tabela de pagini ce se afla in memoria RAM. Fara TLB, pentru accesarea unei locatii din RAM sunt necesare doua accese la memorie (unul in tabela de pagini si altul la adresa efectiva din RAM). Folosind TLB, putem reduce timpul de acces pentru ca o parte din tabela de pagini este acum in cache. Evident, accesul la TLB trebuie sa fie rapid pentru ca acest sistem sa functioneze.

8. Descrieti de ce este necesara mentinerea coerentei cache-urilor pentru sistemele multiprocesor.
Aici puteti sa acceptati variatiuni pe tema: Pentru ca memoria principala este partajata intre mai multe procesoare, iar acestea pot scrie sau citi in mod concurent din ea. Fiecare procesor are cache propriu, deci este foarte usor sa ajungem intr-o situatie in care se poate pierde coerenta datelor din cache-ul propriu cu cele din memoria principala si cele din cache-urile altor procesoare.

9. Descrieti un avantaj pe care il are o magistrala sincrona de date fata de o magistrala asincrona. De asemenea, descrieti un avantaj pe care o magistrala asincrona il are fata de o magistrala sincrona.
Asincron vs. Sincron: Mai simplu de implementat (mai putine linii de date/comenzi), cost redus
Sincron vs. Asincron: Sincronizarea permite rate de transfer mai mari, latime mai mare de banda, mai putine erori de transmisie

10. Care este beneficiul principal pe care il aduce tehnica de loop unrolling?
Elimina instructiunile conditionale din executie, ceea ce mareste viteza de executie (reduce branch penalty, etc.), dezavantajul fiind o amprenta mai mare a programului in memorie. De asemenea, daca anumite linii de cod din bucla sunt independente, codul poate fi executat in paralel, ceea ce mareste si mai mult viteza de executie. Argument de nota 11: De asemenea, poate fi implementat dinamic, daca numarul de elemente din bucla nu este cunoscut la compile time (e.g. Duff's Device)

Started on	Friday, 19 February 2021, 8:17 AM
State	Finished
Completed on	Friday, 19 February 2021, 8:46 AM
Time taken	29 mins 36 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Bill dorește să construiască un cip de memorie de 256K x 64, dar are doar cipuri de 16K x 8. De câte cipuri are nevoie Bill?

- a. 16
- b. 8
- c. 64
- d. 128 ✔
- e. 32

(256K / 16K) * (64 / 8) = 128

The correct answer is: 128

Question **2**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 3ns iar rata de hit locală este 95%
- Latența de acces din L1 în L2 este de 9ns iar rata de hit locală este de 98%
- Latența de acces din L2 în memoria RAM este de 600ns

Care este timpul mediu de acces la memorie?

- a. 4.05ns ✔
- b. 405ns
- c. 0.405ns
- d. 40.5ns
- e. 4050ns

0.95 * 3ns + 0.05 * (0.98 * 12ns + 0.02 * 612ns) = 4.05ns

The correct answer is: 4.05ns

Question **3**

Correct

Mark 1.00 out of 1.00

Un sistem cu cache L1 si L2 are un hit rate de 95% din L1 și de 98% din L2. Care este miss rate-ul global?

- a. 0.99%
- b. 0.01%
- c. 1%
- d. 99%
- e. 0.1% ✔

1 - 0.95 - 0.05 * 0.98 = 0.001

The correct answer is: 0.1%

Question **4**

Correct

Mark 1.00 out of 1.00

Parcurgerea unei liste înlănțuite alocate contiguu în memorie exploatează:

- a. Localitatea spațială ✔
- b. Ambele localități
- c. Localitatea temporală
- d. Niciuna
- e. Nu se poate determina

Lista este alocată contiguu în memorie, iar parcurgere rezultă în acces secvențial. Asta înseamnă că, dacă am accesat o adresă din listă, este probabil să vrem să accesăm și adresele următoare sau precedente.

The correct answer is: Localitatea spațială

Question **5**

Correct

Mark 1.00 out of 1.00

O memorie cache cu linii de 8 octeți lungime este organizată în 128 de seturi a câte 4 linii. Dacă adresarea se face la nivel de octet, cu adrese pe 32 de biți, care este dimensiunea în biți a tag-ului pentru fiecare linie de cache?

- a. 23 ✔
- b. 21
- c. 22
- d. 20
- e. 19

~~32 - log2(128 * 4 * 8) = 20~~
32 - log2(128 * 8) = 22. Cum facusem o greseala initial, orice raspuns va fi punctat!

The correct answers are: 21, 20, 19, 22, 23

Question **6**

Correct

Mark 1.00 out of 1.00

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before

```
for (i = 0; i < M; i++) {
 for (j = 0; j < N; j++) {
        x[i*N+j] = 2 * x[i*N+j];
 }
}
```

After

```
for (j = 0; j < N; j++) {
 for (i = 0; i < M; i++) {
        x[i*N+j] = 2 * x[i*N+j];
 }
}
```

- a. Da
- b. Nu ✔

Your answer is correct.

Prima variantă este mai rapidă fiindcă accesează matricea x în ordinea în care este stocată în memorie.

The correct answer is:
Nu

Question **7**

Correct

Mark 1.00 out of 1.00

Presupuneți că acum 20 de ani ați scris un cod pentru o arhitectura x86 care petrecea jumătate din timp cu accese la memorie și cealaltă jumătate făcând calcule. Dacă am rula același cod astăzi pe o arhitectură x86-compatible, cel mai probabil el ar:

- a. Menține aceeași proporție 50-50% a timpilor de rulare
- b. Petrece mai mult timp făcând calcule
- c. Petrece mai mult timp cu accesul memoriei ✔
- d. Nu se poate determina
- e. Exporta calculele in Cloud

Viteza procesoarelor a crescut mai mult decât viteza memoriilor.

The correct answer is: Petrece mai mult timp cu accesul memoriei

Question **8**

Correct

Mark 1.00 out of 1.00

Care funcție de mapare cache are cele mai puține șanse să producă thrashing (adică oferă șanse minime ca două blocuri să concureze la stocarea în aceeași linie)?

- a. Complet-asociativă ✔
- b. Toate trei
- c. Nu se poate determina
- d. Set-asociativă
- e. Directă

Cum orice linie de memorie poate intra în orice linie de cache, un cache complet-asociativ oferă cea mai bună șansă de a evita thrashing.

The correct answer is: Complet-asociativă

Question **9**

Correct

Mark 1.00 out of 1.00

Magistrala de comunicație SPI:

- a. Are o topologie multi-master, multi-slave
- b. Are o schemă de adresare pe 8 biți
- c. Este o magistrală sincronă de transfer a datelor ✔
- d. Permite comunicația la distanțe mari
- e. Are o viteză de transmisie de maxim 9600 biți pe secundă

SPI este un protocol de comunicare sincron (i.e. folosește un semnal de ceas).

The correct answer is: Este o magistrală sincronă de transfer a datelor

Question **10**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul ce are doar un L1 cache de 128kB este **întotdeauna** mai rapid decât un sistem ce are L1 și L2 cache cu dimensiunea totală de 128kB.

- a. Fals ✔
- b. Adevărat

Nu putem zice că un sistem cu un nivel de cache este **întotdeauna** mai rapid ca un sistem cu două niveluri de cache. Chiar dacă primul sistem are mai mult cache rapid (L1), există cazuri în care asta nu îl ajută să atingă o performanță mai mare ca al doilea sistem. Spre exemplu, un program poate să nu acceseze memoria deloc, caz în care cele două sisteme sunt la fel de rapide.

The correct answer is: Fals

◄ Colocviu

Jump to...

Examen - Partea II ►

Started on	Friday, 19 February 2021, 8:11 AM
State	Finished
Completed on	Friday, 19 February 2021, 8:39 AM
Time taken	27 mins 35 secs
Grade	**6.00** out of 10.00 (**60**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Bill dorește să construiască un cip de memorie de 128K x 32, dar are doar cipuri de 32K x 16. De câte cipuri are nevoie Bill?

- a. 32
- b. 16
- c. 8 ✔
- d. 64
- e. 128

(128K / 32K) * (32 / 16) = 8

The correct answer is: 8

Question **2**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 3ns iar rata de hit locală este 95%
- Latența de acces din L1 în L2 este de 9ns iar rata de hit locală este de 98%
- Latența de acces din L2 în memoria RAM este de 600ns

Care este timpul mediu de acces la memorie?

- a. 4050ns
- b. 0.405ns
- c. 40.5ns
- d. 405ns
- e. 4.05ns ✔

0.95 * 3ns + 0.05 * (0.98 * 12ns + 0.02 * 612ns) = 4.05ns

The correct answer is: 4.05ns

Question **3**

Correct

Mark 1.00 out of 1.00

Un sistem cu cache L1 si L2 are un hit rate de 95% din L1 și de 98% din L2. Care este miss rate-ul global?

- a. 1%
- b. 0.01%
- c. 0.1% ✔
- d. 0.99%
- e. 99%

1 - 0.95 - 0.05 * 0.98 = 0.001

The correct answer is: 0.1%

Question **4**

Correct

Mark 1.00 out of 1.00

Incrementarea unei variabile exploatează:

- a. Nu se poate determina
- b. Ambele localități
- c. Localitatea spațială
- d. Niciuna
- e. Localitatea temporală ✔

Incrementarea unei variabile presupune scrieri repetate la aceeași adresă. Deci, dacă am accesat acea adresă este probabil că vom dori să o accesăm din nou în curând.

The correct answer is: Localitatea temporală

Question **5**

Correct

Mark 1.00 out of 1.00

Un cache mapat direct este organizat intern pe 16384 linii cu lățime de 64 octeți. Adresele cu care lucrează memoria cache sunt pe 32 de biți. Care este lungimea în biți a tag-ului?

- a. 11
- b. 13
- c. 12 ✔
- d. 15
- e. 14

32 - log2(16384 * 64) = 12

The correct answer is: 12

Question **6**

Incorrect

Mark 0.00 out of 1.00

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before:

```
for (i = 0; i < N; i++)
 c[i] = a[i] + b[i];
```

After:

```
int P = 1;
for (i = 0; i < N; i++) {
        prefetch(&a[i+P]);
        prefetch(&b[i+P]);
        prefetch(&c[i+P]);
        c[i] = a[i] + b[i];
}
```

- a. Da ✖
- b. Nu

Your answer is incorrect.

Prima variantă este mai rapidă fiindcă vectorii a, b și c sunt accesați în ordinea în care sunt stocați in memorie. În varianta a doua ei sunt accesați în aceeași ordine, elementele lor vor putea fi găsite în cache, deci apelurile către prefetch vor adăuga doar overhead.

The correct answer is:
Nu

Question **7**

Incorrect

Mark 0.00 out of 1.00

Care dintre următoarele este adevărată?

- a. Un cache set-asociativ cu o singură linie per set este un cache mapat direct
- b. Un cache set-asociativ cu o singură linie per set este un cache complet-asociativ
- c. Un cache set-asociativ cu un singur set este un cache mapat direct ✖
- d. Un cache set-asociativ cu două seturi este un cache complet-asociativ
- e. Toate variantele sunt adevărate

Între seturi maparea se face în mod direct. Dacă seturile au o singură linie, înseamnă ca avem mapare directă între toate liniile, deci este un cache mapat direct.

The correct answer is: Un cache set-asociativ cu o singură linie per set este un cache mapat direct

Question **8**

Incorrect

Mark 0.00 out of 1.00

Care funcție de mapare cache are cele mai puține șanse să producă thrashing (adică oferă șanse minime ca două blocuri să concureze la stocarea în aceeași linie)?

- a. Set-asociativă
- b. Complet-asociativă
- c. Directă ✖
- d. Nu se poate determina
- e. Toate trei

Cum orice linie de memorie poate intra în orice linie de cache, un cache complet-asociativ oferă cea mai bună șansă de a evita thrashing.

The correct answer is: Complet-asociativă

Question **9**

Correct

Mark 1.00 out of 1.00

Magistrala de comunicație serială RS232:

- a. Stă la baza tuturor rețelelor de calculatoare
- b. Are o schemă de adresare pe 8 biți
- c. Folosește baud rate pentru a calcula distanța optimă de transmisie
- d. Permite conectarea punct-la-punct a două dispozitive de calcul ✔
- e. Implementează un mecanism CSMA-CD

RS232 este un protocol de comunicare punct-la-punct.

The correct answer is: Permite conectarea punct-la-punct a două dispozitive de calcul

Question **10**

Incorrect

Mark 0.00 out of 1.00

Pentru a maximiza performanța, TLB-urile sunt organizate de obicei sub forma unor cache-uri mapate direct.

- a. Fals
- b. Adevărat ✖

TLB-urile sunt de obicei organizate sub forma unor cache-uri set-asociative sau complet-asociative.

The correct answer is: Fals

◄ Colocviu

Jump to...

Examen - Partea II ►

Started on	Friday, 19 February 2021, 8:00 AM
State	Finished
Completed on	Friday, 19 February 2021, 8:40 AM
Time taken	39 mins 28 secs
Grade	**7.00** out of 10.00 (**70**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Bill dorește să construiască un cip de memorie de 128K x 32, dar are doar cipuri de 32K x 16. De câte cipuri are nevoie Bill?

- a. 16
- b. 32
- c. 8 ✔
- d. 64
- e. 128

(128K / 32K) * (32 / 16) = 8

The correct answer is: 8

Question **2**

Correct

Mark 1.00 out of 1.00

Un sistem de calcul are două niveluri de cache cu următoarele specificații:

- Latența de acces la L1 este de 3ns iar rata de hit locală este 95%
- Latența de acces din L1 în L2 este de 9ns iar rata de hit locală este de 98%
- Latența de acces din L2 în memoria RAM este de 600ns

Care este timpul mediu de acces la memorie?

- a. 4.05ns ✔
- b. 405ns
- c. 0.405ns
- d. 4050ns
- e. 40.5ns

0.95 * 3ns + 0.05 * (0.98 * 12ns + 0.02 * 612ns) = 4.05ns

The correct answer is: 4.05ns

Question **3**

Correct

Mark 1.00 out of 1.00

Un sistem cu cache L1 si L2 are un hit rate de 95% din L1 și de 98% din L2. Care este miss rate-ul global?

- a. 0.01%
- b. 99%
- c. 0.1% ✔
- d. 0.99%
- e. 1%

1 - 0.95 - 0.05 * 0.98 = 0.001

The correct answer is: 0.1%

Question **4**

Correct

Mark 1.00 out of 1.00

Parcurgerea unei liste înlănțuite alocate contiguu în memorie exploatează:

- a. Localitatea spațială ✔
- b. Localitatea temporală
- c. Nu se poate determina
- d. Ambele localități
- e. Niciuna

Lista este alocată contiguu în memorie, iar parcurgere rezultă în acces secvențial. Asta înseamnă că, dacă am accesat o adresă din listă, este probabil să vrem să accesăm și adresele următoare sau precedente.

The correct answer is: Localitatea spațială

Question **5**

Correct

Mark 1.00 out of 1.00

Un cache mapat direct este organizat intern pe 4096 linii cu lățime de 32 octeți. Adresele cu care lucrează memoria cache sunt pe 32 de biți. Care este lungimea în biți a tag-ului?

- a. 18
- b. 14
- c. 16
- d. 15 ✔
- e. 17

32 - log2(4096 * 32) = 15

The correct answer is: 15

Question **6**

Incorrect

Mark 0.00 out of 1.00

Nicușor optimizează programul în C de mai jos pentru o execuție mai rapidă. A procedat corect?

Before:

```
for (i = 0; i < N; i++)
 c[i] = a[i] + b[i];
```

After:

```
int P = 1;
for (i = 0; i < N; i++) {
        prefetch(&a[i+P]);
        prefetch(&b[i+P]);
        prefetch(&c[i+P]);
        c[i] = a[i] + b[i];
}
```

- a. Nu
- b. Da ✖

Your answer is incorrect.

Prima variantă este mai rapidă fiindcă vectorii a, b și c sunt accesați în ordinea în care sunt stocați in memorie. În varianta a doua ei sunt accesați în aceeași ordine, elementele lor vor putea fi găsite în cache, deci apelurile către prefetch vor adăuga doar overhead.

The correct answer is:
Nu

Question **7**

Correct

Mark 1.00 out of 1.00

Problema cu un cache ce folosește o politică Write-Through este:

- a. Nu poate fi folosită în sisteme multi-procesor, chiar dacă CPU-urile fac snooping pentru un update
- b. Scade performanța cache-ului ✔
- c. Necesită un overhead suplimentar în cache pentru a contabiliza blocurile care au fost modificate
- d. Riscă să lase memoria principală invalidă pentru o perioada lungă de timp
- e. Nici o variantă

Faptul că la fiecare scriere cache-ul trebuie să facă o scriere și în RAM înseamnă că nu poate fi la fel de rapid ca un cache care nu trebuie să facă asta.

The correct answer is: Scade performanța cache-ului

Question **8**

Correct

Mark 1.00 out of 1.00

Cum o să fie afectați parametrii hit time, miss rate și miss penalty dacă adăugăm capabilitatea de prefetching unui cache?

- a. Hit time neschimbat, miss rate scade, miss penalty neschimbat ✔
- b. Hit time scade, miss rate scade, miss penalty crește
- c. Hit time scade, miss rate crește, miss penalty neschimbat
- d. Hit time creste, miss rate crește, miss penalty crește
- e. Hit time neschimbat, miss rate crește, miss penalty crește

Viteza de acces rămâne neschimbată, deci hit time si miss penalty ramân neschimbate. Cum funcționalitatea de prefetching va aduce date în cache înainte de a fi folosite, miss rate va scădea.

The correct answer is: Hit time neschimbat, miss rate scade, miss penalty neschimbat

Question **9**

Incorrect

Mark 0.00 out of 1.00

O magistrala de comunicație serială:

- a. Poate transfera date doar asincron
- b. Permite doar comunicarea punct la punct ✖
- c. Necesită în general un număr mare de linii de comunicație
- d. Necesita un control mai complex decât o magistrală paralelă
- e. Are limitări puternice de lățime de bandă

Un protocol de comunicare serial necesită un control mai complex pentru, spre exemplu, determinarea biților transmiși și a numărului acestora.

The correct answer is: Necesita un control mai complex decât o magistrală paralelă

Question **10**

Incorrect

Mark 0.00 out of 1.00

O locație de memorie și una mapată într-un periferic pot fi accesate simultan de către un procesor.

- a. Fals
- b. Adevărat ✖

Perifericele și memoria sunt mapate în același spațiu de adresă, deci nu pot fi accesate simultan.

The correct answer is: Fals

◄ Colocviu

Jump to...

Examen - Partea II ►

Started on	Thursday, 27 January 2022, 1:25 AM
State	Finished
Completed on	Thursday, 27 January 2022, 1:28 AM
Time taken	3 mins
Marks	4.00/12.00
Grade	**3.33** out of 10.00 (**33**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Știm că la adresa 69 dintr-o memorie SSRAM se afla valoarea 47. Se considera următoarele valori pentru porturile acestei memorii:
CS = 1
WE = 1
OE = 0
ADDR = 69
DATA = 12 (intrare/ieșire).
Ce se va afla la adresa 69 după acest ciclu?

- a. 47, fiindcă se face o operație de citire, nu de scriere.
- b. 12, fiindcă se face o operație de scriere. ✔
- c. 47, fiindcă nu este selectat chipul de memorie.

The correct answer is: 12, fiindcă se face o operație de scriere.

Question **2**

Correct

Mark 1.00 out of 1.00

Ce registru are rolul de contor?

- a. TCNT0 ✔
- b. TIMSK
- c. TCCR0A
- d. OCR0A

The correct answer is: TCNT0

Question **3**

Incorrect

Mark 0.00 out of 1.00

Folosind întregul set de instrucțiuni AVR, scrieți o secvență de cod care:
- Configurează portul A al unui microcontroller ATtiny20 ca pini de ieșire
- Setează valoarea pinilor PA4, PA5 și PA6 să fie 0 (Low)
- Setează valoarea celorlalți pini să fie 1 (High)

Considerăm:
PINA equ 0x00
DDRA equ 0x01
PORTA equ 0x02

a.
```
ldi R16, 0b11111111
out DDRA, R16
ldi R16, 0b10001111
out PORTA, R16
```

b.
```
ldi R16, 0b00000000
out DDRA, R16
ldi R16, 0b01110000
out PORTA, R16
```

c.
```
ldi R16, 0b10001111
out DDRA, R16
ldi R16, 0b10001111
out PORTA, R16
```

d.
```
ldi R16, 0b01110000
out DDRA, R16
ldi R16, 0b01110000
out PORTA, R16
```

The correct answer is: ldi R16, 0b11111111
out DDRA, R16
ldi R16, 0b10001111
out PORTA, R16

Question **4**

Correct

Mark 1.00 out of 1.00

Scrieți un program în avrasm care:
- configurează Timer/Counter0 în modul Fast PWM, TOP == OCRA;
- setează ceasul Timer/Counter0 să folosească un prescaler de 8;
- configurează Timer/Counter0 să schimbe starea pinului OC0B astfel: Set OC0B on Compare Match, clear OC0B at BOTTOM(inverting mode);
- setează valoarea lui OCR0B la 34;
- setează valoarea lui OCR0A la 130;
- generează o întrerupere de comparație pe canalul B.

Considerăm:
TCCR0A equ 0x19
TCCR0B equ 0x18
OCR0A equ 0x16
OCR0B equ 0x15
TIMSK equ 0x26

a.
```
ldi R16, 0b11000011
out TCCR0A, R16

ldi R16, 0b00000100
out TCCR0B, R16

ldi R16, 34
out OCR0B, R16

ldi R16, 130
out OCR0A, R16

ldi R16, 0b00000100
out TIMSK, R16
sei
```

b.
```
ldi R16, 0b00110011
out TCCR0A, R16

ldi R16, 0b00000010
out TCCR0B, R16

ldi R16, 34
out OCR0B, R16

ldi R16, 130
out OCR0A, R16

ldi R16, 0b00000100
out TIMSK, R16
sei
```

c. ✔
```
ldi R16, 0b00110011
out TCCR0A, R16

ldi R16, 0b00001010
out TCCR0B, R16

ldi R16, 34
out OCR0B, R16

ldi R16, 130
out OCR0A, R16
```

ldi R16, 0b00000100
out TIMSK, R16
sei

d. ldi R16, 0b11000011
out TCCR0A, R16

ldi R16, 0b00001010
out TCCR0B, R16

ldi R16, 34
out OCR0B, R16

ldi R16, 130
out OCR0A, R16

ldi R16, 0b00000010
out TIMSK, R16
sei

The correct answer is: ldi R16, 0b00110011
out TCCR0A, R16

ldi R16, 0b00001010
out TCCR0B, R16

ldi R16, 34
out OCR0B, R16

ldi R16, 130
out OCR0A, R16

ldi R16, 0b00000100
out TIMSK, R16
sei

Question **5**

Incorrect

Mark 0.00 out of 1.00

Ce valoare are SP după execuția codului de mai jos, știind că valoarea sa inițială este 204?

```
main:
   ldi R20, 15
   ldi R21, 0
loop:
   ldi R22, 1
   and R22, R20
   brbc 1, skip
   push R20
skip:
   add R21, R20
   push R21
dec R20
   brbc 1, loop
```

- a. 182
- b. 174 ✖
- c. 181
- d. 234
- e. 227
- f. 226

The correct answer is: 182

Question **6**

Incorrect

Mark 0.00 out of 1.00

Instructiunea SBI 7, 2:

- a. Setează pe 1 bitul 7 la adresa 2.
- b. Setează pe 0 bitul 7 la adresa 2.
- c. Setează pe 1 bitul 2 la adresa 7.
- d. Setează pe 0 bitul 2 la adresa 7. ✖
- e. Stochează la adresa 2 valoarea 7.
- f. Stochează la adresa 7 valoarea 2.

The correct answer is: Setează pe 1 bitul 2 la adresa 7.

Question **7**

Incorrect

Mark 0.00 out of 1.00

Ce registru ar trebui modificat pentru a configura un pin ca ieșire?

- a. OCRA
- b. DDRA
- c. PINA
- d. PORTA

The correct answer is: DDRA

Question **8**

Incorrect

Mark 0.00 out of 1.00

Știind că în R20 se afla valoarea 58, iar în R21 se afla valoarea 197, ce valoare va avea flagul Z după ce se executa un AND între cele doua registre?

- a. 1
- b. 0

The correct answer is: 1

Question **9**

Correct

Mark 1.00 out of 1.00

Care dintre următoarele afirmații este adevărată?

- a. Instrucțiunea RETI activează întreruperile la nivel global
- b. Instrucțiunea SEI dezactivează întreruperile la nivel global
- c. Instrucțiunea CLI activează întreruperile la nivel global
- d. Instrucțiunea CALL_ISR activează întreruperile la nivel global

The correct answer is: Instrucțiunea RETI activează întreruperile la nivel global

Question **10**

Incorrect

Mark 0.00 out of 1.00

Știind că valoarea din Stack Pointer este 0xb2, ce valoarea se va afla în el după ce se execută o instrucțiune RET?

- a. 0xb1
- b. 0xb4
- c. 0xb3
- d. 0xb0

The correct answer is: 0xb4

Question **11**

Incorrect

Mark 0.00 out of 1.00

Care dintre următoarele instrucțiuni modifică Stack Pointer?

- a. BRBC
- b. RJMP
- c. RCALL
- d. BRBS

The correct answer is: RCALL

Question **12**

Incorrect

Mark 0.00 out of 1.00

Ce dimensiune are registrul general R16 al procesorului implementat de noi?

- a. 4 octeți
- b. 32 biți
- c. 8 biți
- d. 16 biți

The correct answer is: 8 biți

◄ Link Demo Colocviu Simulare

Jump to...